

CodexRC — Documentación Maestra

De punta a punta: qué es, qué hace, cómo está armado y en qué versión está hoy

Generado el 06/10/2026 a partir del código fuente real del repositorio (README.md, ARCHITECTURE.md, AGENTS.md, CHANGELOG.md de 2222 líneas y los 102 módulos de `core/`). No contiene información inventada: cada dato sale de un archivo existente, citado donde corresponde.

1. Qué es CodexRC, en una frase

Motor de auditoría y caza de vulnerabilidades web, con backend local, evidencia determinista, veredictos falsables, y campañas masivas sobre plugins de WordPress y blancos autorizados. No es un escáner que dice "podría ser vulnerable": construye una cadena de evidencia (`SOURCE -> FLOW -> AUTH -> SANITIZATION -> SINK -> CORRELATION`) evaluada por "abogados" deterministas (nunca una IA que opina), y solo reporta cuando esa cadena está completa y es reproducible.

Las 5 reglas de oro (filosofía, de `README.md`)

1. **Sin evidencia no hay hallazgo.** Toda cadena se evalúa, nunca se infiere por intuición.
2. **Mejor 4 hallazgos que ejecutan de verdad que 20 que solo se reflejan.** VERITAS re-verifica cada candidato en navegador real con un canario inerte antes de sellarlo.
3. **El ruido muere en el motor, no a mano.** Guardas semánticas, refutación por dominancia y corpus de regresión eliminan falsos positivos solos, con registro del porqué.
4. **La tolerancia del edge no es vulnerabilidad.** Solo el *mismatch* observable edge/origin escala; la escalera de impacto nunca se salta (DETECTED sin reproducción no avanza).
5. **La caza respeta el blanco.** Sondas read-only A->B, canarios inertes, jitter, presupuestos de requests declarados por módulo, y parada al primer impacto confirmado. En vivo jamás se envenena la respuesta de un usuario tercero.

2. Arquitectura general (6 subsistemas)

Fuente: `ARCHITECTURE.md` . El layout físico es:

<code>backend/app.py</code>	Único entrypoint HTTP (Flask): rutas <code>/api/*</code> , <code>jobs</code> , <code>watchdog</code> , <code>auth</code> de Arsenal.
<code>core/* .py</code>	102 módulos, una responsabilidad cada uno (sin subcarpetas: organización por archivo plano).
<code>frontend/* .html</code>	3 UIs autocontenidas (<code>index</code> , <code>hunter</code> , <code>arsenal</code>).
<code>hechos/</code>	Memoria de "ya auditado" – NUNCA se vacía o se re-audita todo desde cero por error.
<code>reports/</code>	Informes JSON/PDF/TXT generados por job.
<code>docs/</code>	Especificaciones y esta documentación.

2.1 Motor de caza (corazón)

`hunter.py` es una **fachada** que re-exporta Spider, DeepHunter, XSSHunter, BlindXSS y XSSPro desde sus módulos `hunter_*.py` . Los consumidores (backend, CLI, `async_lane`) solo conocen la fachada, nunca los módulos internos directamente. `brain.py` , `self_tune.py` y `pipeline.py` orquestan decisiones.

2.2 Autenticación (`auth.py` , ~60KB)

Matrix universal: combina bases × rutas × payloads. La sesión heredada se guarda por origen (nunca pisa sesiones de otro blanco). Incluye `ghostgate.py` (evasión de Cloudflare vía navegador real, nombre fijado por el operador) y `waf_guard.py` (jitter + cooldown con memoria persistente).

2.3 Verificación anti-falsos-positivos (lo que hace confiable al motor)

Cadena de evidencia en orden: - `taint_trace.py` — TAIN-T-TRACE: sigue fuente->sink intra-archivo. - `gates_audit.py` — GATES-AUDIT: dictamina si un handler AJAX/REST está protegido (nonce/capability). - `evidence.py` — EVIDENCE-CHAIN: arma la cadena completa y la evalúa con jueces deterministas (nunca un LLM). - `fp_autoclose.py` — cierra falsos positivos ya conocidos. - `pattern_match.py` — CVE-MATCH con patrones ponderados contra una baseline histórica. - `veritas.py` — navegador real contra falsos positivos, con degradación elegante si no hay Chromium. - `observe.py` / `regress.py` — CODEX-OBSERVE / CODEX-REGRESS: el corpus de regresión (141 casos a la fecha de este documento). - `escalada.py` — sigue el hallazgo crítico con 4 pasos automáticos, solo sobre piezas verificadas.

2.4 Caza masiva (plugins de WordPress)

`wide_corpus.py` arma el universo (todo plugin de wordpress.org con

=5.000 instalaciones). `hunt_wide.py` es el runner incremental (crash-safe, nunca re-audita lo ya hecho).

`diff_hunt.py` audita solo código nuevo entre versiones. `plugin_batch.py`, `retro_hunt.py`, `vendor_farm.py` y

`cve_matcher.py` completan la batería. `dossier_hunt.py` es el driver más reciente (Q4-2026), enfocado en las clases nuevas POI-REACH y MAGIC-CONFUSION.

2.5 Ingeniería inversa (binarios/APKs)

`bin_audit.py` (secretos en ELF), `re_engine.py` (parseo puro Python de DEX/JAR/ELF) y `decompile.py` (bytecode DEX -> pseudocódigo). Todo en Python puro para correr sin compilar en entornos como Termux/armv7l.

2.6 Backend + UI

Rutas Flask con jobs bajo lock, watchdog de procesos zombie (`zombies.py`). Seguridad: bind solo a loopback por defecto, logs sin secretos (passwords/tokens/cookies nunca se registran), Arsenal con modo EXTREMO que exige contraseña y token de desbloqueo temporal.

3. La escalera cero-falsos-positivos (cómo se decide un veredicto)

Principio transversal a todos los módulos nuevos (v0.85+): **cada peldaño exige el anterior**, nunca se salta un escalón. Ejemplo genérico de la familia desync/HTTP:

```
BENIGN -> X-DETECTED -> X-DESYNC -> X-STATE-EFFECT -> [HALLAZGO] SECURITY-IMPACT-DEMO
```

Y para Object Injection (`poi_reach.py`):

```
POI-DETECTED (unserialize con taint sin sanitizar)
-> POI-UNAUTH (el sink vive en superficie anónima)
-> POI-CHAIN (hay gadget con método mágico alcanzable)
-> POI-REACH ([HALLAZGO] candidato reportable)
```

Sin alcance anónimo, el veredicto honesto queda en **POI-AUTH**: es una pista, no un hallazgo reportable.

Solo dos símbolos están reservados para parar todo y avisar al operador: **[HALLAZGO]** (hallazgo explotable real) y **[ALERTA]/[CRITICO]** (hallazgo crítico). Todo lo demás es ruido que el motor filtra solo.

4. Línea de tiempo de versiones (resumen)

CodexRC lleva **78 versiones documentadas** en el CHANGELOG (desde v0.40 "Verificación del sistema" hasta v0.96.1). El detalle completo, versión por versión, está en [version_timeline.md](#) (generado automáticamente del CHANGELOG real). Hitos mayores:

Versión	Hito
v0.40 – v0.53	Cimientos: verificación, vendor-farm, plugin-batch, gates-audit, diff-hunt, evidence-chain, VDP-1300, UI-Renaissance
v0.57.x	AUTH matrix universal, auditoría de seguridad propia, triaje masivo de falsos positivos (BAC/SQLi)
v0.62.x	REVERSE-WEB, MULTI-IA (coordinación entre operadores), VDP-FRESH watcher cada 6h
v0.67 – v0.81	BAC-PROOF (IDOR dinámico en WP-LAB), ADAPTIVE-HUNT (capa epistémica), ARSENAL de 2 niveles
v0.82 – v0.84	Grafo de experimentos, selección por EDV, correlación cross-layer
v0.85 – v0.96.1	La corona (corona) : cadena completa de detección de desync HTTP (CL.0 -> H2 -> multi-conexión -> cross-request -> reproducción -> impacto -> FP-elimination -> evidence package), integración al Hunter (<code>crown_chain.py</code>), y el dossier Q4-2026 (POI-REACH, MAGIC-CONFUSION)

4.1 La corona (v0.85 -> v0.96.1), paso a paso

Esta es la cadena de detección de HTTP request smuggling / desync que se construyó módulo por módulo, cada uno con su propio corpus de regresión y validación en LAB antes de integrarse:

1. **v0.85 PIPE-CROSS** — nodo `cross_layer` en el pipeline de escaneo.
2. **v0.86 WCD-CACHE-KEY** — envenenamiento de caché vía clave mal normalizada.
3. **v0.87 CL.0-SINGLE-TIER** — smuggling de un solo salto.
4. **v0.88 H2-TRANSLATION** — edge HTTP/2 real traducido a H1 interno; oráculo de atribución por stream.
5. **v0.89 MULTI-CONNECTION** — una conexión envenena el pool y otra (TCP distinto) hereda el eco.
6. **v0.90 CROSS-REQUEST CORRELATION** — dos smuggles en la misma conexión, desplazamiento `k` medible y consistente.
7. **v0.91 REPRODUCTION ENGINE** — exige firma y `k` idénticos en dos pases independientes (A y B) antes de llamarlo reproducible.
8. **v0.92 IMPACT CORRELATION** — ¿el desync daña a un usuario *distinto* del atacante? Recurso simulado, atribución ALIGNED / SMUGGLED-PROTECTED / MISPAIRED-BENIGN.
9. **v0.93 FP-ELIMINATION** — batería de 4 cebos que nunca deben reportar + control positivo obligatorio.
10. **v0.94 EVIDENCE PACKAGE** — paquete falsable con hash SHA-256 canónico, comandos de reproducción y límites declarados.
11. **v0.95 INTEGRACIÓN AL HUNTER** — `crown_chain.py` orquesta los 5 peldaños (DETECTION -> REPRODUCTION -> STATE-EFFECT -> CROSS-CONNECTION -> SECURITY IMPACT) sin tocar los módulos originales.
12. **v0.96.0 / v0.96.1 DOSSIER Q4-2026** — tres clases nuevas: POI-REACH (Object Injection con alcance), MAGIC-CONFUSION (confusión de extensión/contenido hacia Imagick), y el blindaje que cerró 2 falsos positivos reales encontrados en el primer barrido en vivo contra `wefirms`.

5. Catálogo completo de módulos

Los 102 archivos de `core/` están indexados con su versión de origen y descripción real (extraída del encabezado de cada archivo) en [modulos_index.csv](#) — ábrilo en Excel/Sheets para filtrar por versión o buscar por palabra clave.

Agrupados por función (ver sección 2 para el detalle narrativo):

- **Detección de desync/HTTP:** `cl0_probe`, `h2_probe`, `mc_probe`, `crc_probe`, `edgesync`, `wcd_bait`, `wcd404_probe`, `cache_*` (bait, baseline, controls, correlation, fingerprint, semantic).
- **Reproducción/impacto:** `repro_engine`, `race_proof`, `race_trace`, `chain`, `crown_chain`, `impact_probe`, `state_correlation`.
- **Anti-FP / evidencia:** `fp_elimination`, `fp_autoclose`, `fp_memory`, `evidence`, `evidence_package`, `veritas`, `ghostgate`, `waf_guard`.
- **Autenticación / IDOR / BAC:** `auth`, `bac_proof`, `gates_audit`, `poi_reach`, `normalization_audit`.

- **Inyección:** `sqli_bait`, `ssti_bait`, `graphql_bait`, `magic_confusion`.
 - **XSS:** `hunter_xss`, `hunter_xsspro`, `hunter_blind`, `cspt_scan`.
 - **Caza masiva WordPress:** `wide_corpus`, `hunt_wide`, `diff_hunt`, `plugin_batch`, `dossier_hunt`, `retro_hunt`, `vendor_farm`, `cve_matcher`.
 - **Ingeniería inversa:** `bin_audit`, `re_engine`, `decompile`.
 - **Investigación/estrategia:** `research_memory`, `experiment_catalog`, `experiment_graph`, `experiment_selector`, `hypothesis_graph`, `adaptive_hunt`, `self_tune`, `brain`.
 - **Infraestructura/soporte:** `pipeline`, `zombies`, `cfg`, `connection_state`, `tech_detect`, `recon`, `taint_trace`, `report_export`, `vision_gate`, `semantic_*`.
-

6. Estado actual del motor (a la fecha de este documento)

- **Versión:** v0.96.1 "BLINDAJE POI-REACH".
 - **Corpus de regresión:** 141/141 casos, 0 fallos (certificado hoy).
 - **Auditoría de 588 plugins:** ejecutada completa (0 [HALLAZGO] confirmados, 6 candidatos POI-AUTH detectados).
Nota de transparencia: el JSON detallado con esos 6 candidatos se perdió en una limpieza de archivos temporales; el motor que los generó sigue intacto y puede re-correr el barrido cuando el operador dé la orden "LestGo".
 - **Lo que el motor NO hace todavía de forma automática:** no dispara ataques de IDOR/lógica de negocio por sí solo. Hoy detecta candidatos (Fase 1: lectura estática de endpoints vía `gates_audit / poi_reach`) y valida BAC/IDOR dinámicamente contra un WP-LAB local (`bac_proof.py`, con usuarios Admin/Suscriptor/Anónimo), pero no arma automáticamente los payloads de lógica financiera (montos negativos, decimales truncados, sobreescritura de estado de pago). Esa pieza (el "Destructor de Lógica Financiera") es la que se definió construir a continuación.
-

7. Roadmap inmediato (decidido en esta conversación)

Objetivo: hacer que CodexRC sea mejor que Burp Suite Pro en control de acceso (IDOR) y lógica de negocio, su único punto débil real.

1. Ya tenemos: extracción estática de endpoints (`gates_audit / poi_reach`) + disparo multi-rol dinámico (`bac_proof.py`, Admin/Suscriptor/Anónimo contra WP-LAB).
 2. Falta: módulo de **inferencia de parámetros** (leer qué variables lee cada handler: `$_POST['ticket_id']`, `$_GET['user_id']`, etc.) y autocompletarlas con valores de prueba.
 3. Falta: **Destructor de Lógica Financiera** — detectar variables tipo `price / amount / qty / discount / total` y aplicarles la batería de ataques (negativos, decimales truncados, arrays, sobreescritura de estado `status=paid`).
-

8. ANEXO A — CHANGELOG COMPLETO, sin resumir (78 versiones, texto íntegro)

Esta sección vuelca el archivo `CHANGELOG.md` completo (2222 líneas), tal cual está en el repositorio, sin condensar ni resumir.

v0.96.1 — BLINDAJE POI-REACH: DOS FP ELIMINADOS (141/141 PASS)

Causa: la primera corrida LestGo del DOSSIER-HUNT regalo un POI-REACH en weforms (10k installs) que era FP doble del propio modulo.

- FP 1 (RC-000269): unserialize con `allowed_classes => false` esta NEUTRALIZADO (no instancia objetos): el sink se descarta de raíz. weforms trae el patch explícito en `class-ajax.php:524`.
- FP 2 (RC-000270): nopriv de OTROS metodos del archivo NO otorga alcance; queda como pista `nopriv_file`, jamas como POI-UNAUTH.
- Fix adicional: `_line_in_handler` ahora mide el cuerpo REAL del callback contando llaves (la heurística de 800 lineas se comia funciones vecinas y regalaba UNAUTH ajeno).

- dossier_hunt: presupuesto de 150s por plugin (SIGALRM); gigantes como booking-manager se cortan honestos con TIMEOUT.

Corpus: 139 -> 141 casos, 2 nuevos, 0 FAIL. Motor listo para re-auditar los 588 blancos del dossier con numeros honestos.

v0.96.0 — DOSSIER Q4-2026: TRES CLASES NUEVAS (139/139 PASS)

Investigacion de calibre integrada al motor (decision del operador: "Pon los 3 de una vez"). Todo nacio en LAB; nada toco al Hunter ni a los modulos existentes. Corpus: 131 -> 139 casos, 8 nuevos, 0 FAIL.

- core/poi_reach.py — POI-REACH (leccion CVE-2026-2599): Object Injection con ALCANCE y GADGET. Escalera: POI-DETECTED (taint de TAINT-TRACE en unserialize) -> POI-UNAUTH (handler dictaminado por GATES-AUDIT como superficie anonima) -> POI-CHAIN (POP-CANDIDATE: clase con metodo magico y propiedades en el mismo plugin). Veredicto POI-REACH exige los tres peldanos; sin alcance = POI-AUTH honesto. Superficie export/download/csv anotada como prioridad.
- core/magic_confusion.py — MAGIC-CONFUSION (leccion CVE-2026-65640, WP Core <= 7.0.3): extension vs contenido. Propagacion punto fijo de variables desde primitivas de entrada (\$_FILES, wp_upload_bits, download_url...) hasta sinks Imagick (readImage/readImageBlob/ping); gate de contenido = wp_check_filetype_and_ext/finfo/getimagesize. Sin gate: MAGIC-CONFUSION. Con gate: CONFUSION-GUARDED (cero FP).
- labs/wcd404_lab.py + core/wcd404_probe.py — WCD-404-LEAK (status code miente): 4 escenarios lab (lie_cached, lie_nocache, honest_404, lie_expired) + probe con escalera BENIGN -> STATUS-LIE-DETECTED (4xx cuyo cuerpo contiene marcadores DECLARADOS de la cuenta propia) -> WCD-404-LEAK (cache HIT cross-user observable, <=8 reqs read-only). Sin HIT: LEAK-NO-CACHE honesto (caso linktr.ee). Blanco muerto: conn-dead sin crash.
- Control negativo obligatorio en las 3 clases: los fixtures sanitizados/gated/limpios NUNCA generan veredicto (RC-000262, RC-000265, RC-000267).
- Flakes ambientales del sandbox (puertos): corrida unica regla reafirmada; huérfanos limpiados antes de certificar.

v0.95.0 — INTEGRACION AL HUNTER (corona montada)

Decision del operador cumplida: con la corona en mano, la cadena completa queda disponible desde la fachada del Hunter.

- core/crown_chain.py: orquestador de la escalera completa con los motores del roadmap, SIN tocar ningun modulo existente: rung 1 DETECTION (cl0+h2+mc+crc, veredictos propios, cero-FP cada uno) -> rung 2 REPRODUCTION (2 genealogias x pase A/B) -> rung 3 STATE-EFFECT (firma k consistente) -> rung 4 CROSS-CONNECTION (contaminacion entre conexiones PROPIAS).
- rung 5 SECURITY IMPACT: LIMITE HONESTO en vivo: NO se envenena la respuesta de usuarios de terceros. Rung demostrada SOLO en lab (impact_probe, recurso protegido simulado). La escalera viva jamas supera CROSS-CONNECTION-DEMO.
- Modo lab: ensambla el paquete completo (CHAIN-COMPLETE-LAB, k=2, 37/40 reqs). Modo live: read-only, presupuesto declarado 60 reqs, sin senal = NO-DESYNC honesto (la escalera no se abre).
- Fachada core/hunter.py exporta CrownChain (PEP: imports existentes intactos). Dependencias minimas del CLI (requests, beautifulsoup4) restauradas en el sandbox.
- RC-000258..260 integrados (3 casos). Corpus: 131.

v0.94.0 — FP-ELIMINATION + EVIDENCE PACKAGE (CORONA)

Las dos ultimas etapas del roadmap, de una vez: la garantia cero-FP se vuelve VERIFICABLE y la cadena completa se entrega FALSABLE.

v0.93.0 FP-ELIMINATION (core/fp_elimination.py)

- TABLA DE CLASES DE SENAL: 8 clases observables del oraculo, cada una con su regla y su estatus reportable. Senal sin regla = FP potencial: el modulo NO aprueba.

- BATERIA DE CEBO: quiet_drain, benign_pin, pool_shift (mispairing benigno), flaky_swap (1/2): NINGUNO puede producir veredicto reportable. Si alguno reporta: FP-LEAK (falso positivo REAL).
- CONTROL POSITIVO: pool_swap DEBE reportar; sin el, la bateria es vacua: UNSTABLE, sin claim.
- Veredicto FP-ELIMINATED: 0/4 cebos reportables + control positivo activo + reglas de no-escala del disparo unico presentes en crc/repro/impact. Presupuesto 25/26 reqs.

v0.94.0 EVIDENCE PACKAGE (core/evidence_package.py)

- Paquete falsable de la cadena COMPLETA: rung 1+2 deteccion+k (CRC-STATE-EFFECT k=2) y reproduccion (2 genealogias x pases A/B, firma identica); rung 3 impacto cross-user (SECURITY-IMPACT-DEMO 2/2 con control limpio); rung 4 cita FP-GUARANTEE (RC-000254).
- LIMITES declarados: read-only, modo lab, recurso protegido SIMULADO, presupuestos (37/40), escalera sin saltos, cero-FP verificado.
- Falsabilidad mecanica: comandos exactos de reproduccion + hash sha256 del payload canonico; verify_package() re-computa y DETECTA tampering.

LECCION CRITICA (huerfano): un lab cuyo spawn fallo

en el bind no se entera nadie: _port_up exita contra CUALQUIER instancia que ya escuche el puerto, aunque sea huerfana de otra corrida. Los tests manuales con `$!` del subshell dejaron un pool_swap vivo en 19621 y las auditorias auditaron una INSTANCIA AJENA con estado acumulado (ronda 1 ALIGNED). Regla nueva en los 3 motores + corpus: si el puerto YA escucha antes del spawn, es contaminacion: UNSTABLE/FAIL ruidoso. Nunca se audita lo que uno no spawnó.

- RC-000254..257 integrados (4 casos). Corpus: 128.

v0.92.0 — IMPACT CORRELATION (etapa de IMPACT)

El roadmap cierra el circulo: el desync reproducido (v0.91) hace dano a un usuario DISTINTO del atacante?

Mecanica v0.92 (herencia del pool v0.89 + recurso protegido SIMULADO del lab): - el atacante (conn A) hace POST CL.0 con un request smuggleado y CIERRA tras leer su propia respuesta: la respuesta del smuggle queda PENDIENTE en la conn del pool; - la victima (conn V, TCP DISTINTO) hereda la conn del pool LIFO: su primer request recibe la respuesta del smuggle del atacante (contaminacion cross-user).

Oraculo observable: QUE respuesta recibe la victima. ECHO -> ALIGNED SECRET-IMPACT-* -> SMUGGLED-PROTECTED (recibio la respuesta de un recurso protegido que jamas pidio) ECHO /OK -> MISPAIRED-BENIGN

Escalera cero-FP: BENIGN -> IMPACT-CANDIDATE (mispairing benigno, o swap protegido 1/2: probable, NO reportable) -> SECURITY-IMPACT-DEMO (swap protegido 2/2 rondas Y control limpio).

- Control previo obligatorio: victima limpia antes de envenenar; sin control alineado no hay claim.
- Presupuesto: 1 control + 2 rondas x (1 POST + 1 GET victima) = 5 reqs (techo declarado 6). Read-only: el recurso protegido /admin/secret es dato SIMULADO del lab, jamas un recurso de terceros.
- LECCION FISICA del desync real: la cola deriva +1 por ronda (la respuesta propia de la victima queda pendiente); un pool sin limite de reuso acumula deriva entre rondas y el swap 2/2 se vuelve imposible. Solucion realista: conn heredada SUCIA se cierra tras su uso (limite de reuso tras estado anomalo, estilo keepalive_requests). El lab modela nginx, no una fantasia.
- RC-000248..253 integrados (6 casos). Corpus: 124.

v0.91.0 — REPRODUCTION ENGINE (etapa 21/22)

Ultima etapa de MODULO antes de IMPACT / FP-ELIMINATION / EVIDENCE PACKAGE. Responde UNA pregunta: la firma de un candidato STATE-EFFECT se reproduce a demanda?

Escalera de impacto (nunca se salta): DESYNC OBSERVED -> REPRODUCIBLE (2 genealogias independientes) -> STATE EFFECT -> CROSS-CONNECTION -> SECURITY IMPACT.

- Genealogia = estado del servidor INDEPENDIENTE: modo lab instancia FRESCA de crc_lab por genealogia (proceso, pool y puerto propios, 19511/19512); modo vivo secuencia completa de conexiones nuevas.
- Cada genealogia ejecuta pase A (deteccion) + pase B (replay sobre la MISMA genealogia). El disparo unico (flaky) NO escala.
- Veredictos conservadores: REPRODUCIBLE (firma y k identicos en A y B de TODAS las genealogias), NON-REPRODUCIBLE (senal que no sostiene: no avanza), NO-CANDIDATE (benign reproducible), UNSTABLE (instancia muerta/control caido: sin claim).
- Presupuesto: 2 genealogias x 2 pases x 8 reqs = 32 max. genealogies>2 se rechaza ANTES de gastar.
- FIX RC-CONN-DEAD (lecciones live v0.90.1): el _connect de _phase_conn en crc_probe estaba fuera del try; un blanco muerto a mitad de fase crasheaba el probe. Ahora devuelve "conn-dead" honesto y el finally quedo blindado (s=None).
- RC-000242..247 integrados (6 casos: reproducible, flaky-no-escala, benign, edge-strict, puerto-tumba, invariantes). Corpus: 118 casos.
- Validacion: puerto-tumba -> la genealogia viva detecto k=2 pero la muerta pesa: UNSTABLE honesto, nunca se promedia una instancia muerta.

v0.90.1 — VALIDACION EN VIVO (linktr.ee) + TLS

- Validacion en vivo de los 4 probes desync contra linktr.ee (read-only, 23 requests totales, LestGo del operador):
- cl0 (v0.87) -> BENIGN: el edge parseo el smuggle como pipelining benigno (S-REDIR). Oraculo de timing funciona en vivo SIN reflejo.
- h2 (v0.88) -> UNSTABLE, mc (v0.89) y crc (v0.90) -> UNKNOWN: los oraculos de eco requieren que el blanco REFLEJE datos del request; sin reflejo se detienen conservadores. NINGUN falso positivo contra el edge real (CF): cero-FP confirmado en campo.
- TLS en probes para vivo: h2 con ALPN h2; mc/crc con ssl wrap por esquema https; cl0 ya lo tenia.
- FIX RC-PUERTO-ESQUEMA: mc/crc conectaban al puerto 80 aunque la URL fuera https -> UNREACHABLE falso en vivo. Ahora port = 443 si use_tls, 80 si no (el lab usa puertos explicitos: sin cambio de comportamiento).
- Corpus re-verificado con el codigo final: 112/112 PASS (los flakes repetidos eran labs muertos por carga del sandbox; resueltos con cache intra-corpus).
- Limitacion documentada: los oraculos de eco (h2, mc, crc) solo dan veredicto definitivo contra blancos que reflejan datos del request en la respuesta (search, 404 con path, API echo). Sin reflejo su techo es el conservador UNKNOWN/UNSTABLE.

v0.90.0 — CROSS-REQUEST CORRELATION

Reconstruccion de la cola del origin por CORRELACION entre requests (etapa 21/22). Un POST CL.0 lleva DOS requests smuggleadas (T1, T2: GETs propios con tokens unicos): si el origin procesa el residual como cola, la coleccion de respuestas de los followups queda desplazada por k MEDIBLE:

k=0 [ECHO-fb, ECHO-fc, ECHO-fd] alineado k=1 [SMUGGLE-T1, ECHO-fb, ECHO-fc] shift 1 k=2 [SMUGGLE-T1, SMUGGLE-T2, ECHO-fb] shift 2

- labs/crc_lab.py: 5 escenarios deterministas (quiet_drain, edge_strict, seq_desync k=2, partial_drain k=1, flaky_forward sin repro). Edge con apareamiento UNA respuesta por request; origin H1 interno con parser residual de cola.
- core/crc_probe.py: auditor crc_audit() con oraculo de PERMUTACION: la firma del desync es la correlacion CONSISTENTE (eco propio desplazado EXACTAMENTE por el numero de ecos del smuggle: fi == si_count). Patron disperso NO escala. k queda medible en evidencia.
- Auto-control por atribucion (sin baseline): eco propio en pos 0 = control de fase; sin eco propio ni del smuggle -> UNKNOWN (cero-FP).
- Escalera: BENIGN -> CRC-DETECTED (1/2, NO reportable) -> CRC-DESYNC (repro 2/2) -> CRC-STATE-EFFECT (k consistente y reproducible).
- Contratos pre-registrados (ExperimentGraph), 1 variante x 2 fases x 4 reqs = max 8 requests, read-only (smuggles = GETs propios).
- RC-000236..241 (6 casos). Corpus: 112/112 PASS (0 FAIL). Etapa 21/22 del roadmap a la corona.
- Leccion de implementacion: la correlacion es ENTRE REQUESTS de la MISMA conexion (no entre conexiones como v0.89): la fase completa (POST + 3 followups) corre en UNA conexion. Un flake de sandbox (lab muerto bajo carga)

se resolvio con el cache intra-corpus, sin tocar el codigo.

v0.89.0 — MULTI-CONNECTION

Contaminacion CRUZADA entre conexiones como etapa MULTI-CONNECTION del roadmap (efecto de segundo orden). El edge mantiene un POOL LIFO de conexiones keep-alive al origen (como nginx upstream keepalive): la conexion atacante A hace POST CL.0 con prefijo smuggleado y CIERRA; la origin conn envenenada vuelve al pool y la conexion victima B (TCP distinto) la HEREDA: su primer followup recibe el eco del smuggle de A.

- labs/mc_lab.py: 5 escenarios deterministas (benign_pin, quiet_drain, pool_desync, edge_strict, flaky_pool). Origin H1 interno con parser residual (consume content-length y procesa el smuggle como request invisible). Edge con apareamiento UNA respuesta por request reenviado: las respuestas pendientes del smuggle quedan en la origin conn, no se drenan al cliente que cerro. Flaky: reusar UNA conn sucia en la vida del lab (salud intermitente).
- core/mc_probe.py: auditor mc_audit() con oraculo de ATRIBUCION por CONEXION (nada inferido): B recibe su eco en pos 0 -> alineado; B recibe SMUGGLE-ECHO de material ajeno -> el origin proceso bytes de OTRA conexion; ademas eco propio desplazado -> coleccion desplazada (STATE-EFFECT). La contaminacion cuenta CUALQUIER eco del smuggle (una fase puede heredar el eco stale de la anterior: igual observable).
- Auto-control por atribucion (sin baseline separado): el eco propio del followup en pos 0 es el control de cada fase; sin eco propio ni del smuggle -> UNKNOWN (no BENIGN, cero-FP).
- Escalera determinista: BENIGN -> MC-DETECTED (1/2, probable, NO reportable) -> MC-DESYNC (repro 2/2) -> MC-STATE-EFFECT (ademas coleccion desplazada).
- Contratos pre-registrados (ExperimentGraph), 1 variante x 2 fases x 4 reqs (A: 1 POST smuggleado; B: 3 GETs) = presupuesto max 8 requests, read-only (smuggle = GET propio con token unico).
- RC-000229..235 (7 casos): pinning 1:1, drenaje, pool LIFO -> STATE-EFFECT 2/2, RST estricto, flaky sin repro -> DETECTED (no reportable), invariantes de presupuesto, y typo h2 (Informe -> informe: NameError latente si el ExperimentGraph estaba activo).
- Lecciones de implementacion (fase LAB): (1) el buffer de recepcion del origen viaja CON la conexion del pool, no con el handler: los bytes mas alla de una respuesta mueren con el handler y la poison desaparece (leccion v0.88 aplicada al pool); (2) las conexiones de health-check TAMBIEN entran al pool: con FIFO, B hereda una limpia y la determinismo se rompe; LIFO (reuso mas reciente primero) hace la herencia determinista; (3) grace de ciclo de vida 0.15s tras cerrar A antes de que B herede (no es oraculo de timing: es dejar que el edge libere la conn al pool).
- Corpus: 106/106 PASS (0 FAIL). Etapa 20/22 del roadmap a la corona.

v0.88.0 — H2-TRANSLATION

Parser differential h2->h1 como etapa PARSER DIFFERENTIAL del roadmap. Un edge HTTP/2 (prefacio + HPACK literal + frames sobre sockets crudos, sin librerias) traduce a un origen H1 interno; dos bugs de traduccion clasicos: h2.CL (reenvio de DATA mas alla del content-length) e inyeccion H1 por HPACK leniente (valor con CRLF copiado verbatim).

- labs/h2_lab.py: 5 escenarios deterministas (benign_strict, quiet_benign, h2cl_desync, hdr_injection, flaky_translation). La coleccion de respuestas del origen se mapea FIFO a los streams H2: con desync, el stream del followup recibe el eco del smuggle.
- core/h2_probe.py: auditor h2_audit() con bateria h2cl|hinj. Oraculo observable: ATRIBUCION de respuestas por stream (nada inferido): fb recibe su eco -> alineado; fb recibe SMUGGLE-ECHO -> el origin proceso bytes que el edge no contabiliza; RST/GOAWAY del edge -> frontera estricta (BENIGN, no error de red).
- Escalera determinista cero-FP: BENIGN -> H2-DETECTED (1/2, probable, NO reportable) -> H2-DESYNC (repro 2/2) -> H2-STATE-EFFECT (coleccion desplazada).
- Contratos pre-registrados (ExperimentGraph), 2 baselines (C0 control, C1 POST vacio alineado), presupuesto max 8 requests, read-only (smuggle = GET propio con token unico).
- RC-000223..228 (6 casos): frontera estricta BENIGN- EDGE, drenaje benigno, h2.CL -> STATE-EFFECT, inyeccion HPACK -> STATE-EFFECT, flaky sin repro -> DETECTED (no reportable), invariantes de framing.
- Lecciones de implementacion (fase LAB): el header del frame H2 son 9 bytes (length 3 + type 1 + flags 1 + sid 4): un sid de 3 bytes corrompe TODO el framing; el buffer de lectura debe ser PERSISTENTE por conexion (HEADERS+DATA llegan en un mismo segmento TCP y ningun byte se descarta).
- Corpus: 99/99 PASS (0 FAIL). Etapa 19/22 del roadmap a la corona.

v0.87.0 — CL.0-SINGLE-TIER

Request smuggling CL.0 como etapa DESYNC DIFFERENTIAL del roadmap, con la disciplina heredada de la linea DESYNC: contratos pre-registrados en el ExperimentGraph, juez determinista, cero-FP por diseno y presupuesto de 8 requests.

- `core/cl0_probe.py`: auditor `cl0_audit()`. Tecnica: POST con 'Content-Length: 0' cuyo stream lleva un prefijo smuggleado inofensivo (GET propio con token unico, jamas recursos de terceros); si el front no contabiliza el prefijo y el backend lo procesa, la cola de respuestas queda desplazada.
- Oraculo observable (nada inferido), tres fases: T1 ventana de silencio tras POST+smuggle: una respuesta temprana demuestra que el FRONT parseo el prefijo (pipelining benigno, NO desync). T2 eco tras el followup: la respuesta del smuggle aparece solo cuando llega el request legitimo -> MISMATCH de contabilidad observable. S doble followup con tokens distintos: si la ultima respuesta porta el token del PRIMER followup, la cola esta desplazada (STATE EFFECT observable).
- Regla cero-FP explicita: el eco del token SOLO no es desync. Sin diferencial de timing observable, el veredicto no escala.
- Escalera determinista: BENIGN -> CL0-DETECTED (eco 1/2, probable, NO reportable) -> CL0-DESYNC (repro 2/2) -> CL0-STATE-EFFECT (shift observable 2/2). (+ UNSTABLE / UNREACHABLE / UNKNOWN). RST tras el POST = frontera del edge estricta -> BENIGN con precedencia sobre el error de red.
- Presupuesto: baseline C1 (POST vacio + followup secuencial, 2 reqs; sin par legible -> UNSTABLE, conclusiones prohibidas) + 1 variante x (P + R) x 3 reqs = max 8. Bateria de 2 candidatos (S-REDIR '/', S-404 '/cl0-nonexistent-404'); MAX_VARIANTS=1 dispara solo el primero viable.
- Clases de evidencia: E-CL0-SIGNAL / E-CL0-STATE con genealogia (run_id, pid, fase) en el grafo.
- `labs/cl0_lab.py`: 5 escenarios deterministicos sobre sockets crudos simulando el stack front+origin EN UN PROCESO con cola de respuestas: `benign_strict` (pipelining durante T1), `edge_safe_rst` (RST con SO_LINGER), `cl0_desync` (raw-forwarder con cola off-by-one), `flaky_desync` (desync solo en la primera conexion con smuggle), `quiet_benign` (drain residual). Deteccion de 'glued' (smuggle pegado al POST en el mismo paquete) para no confundir pipelining legitimo con smuggle.
- Bug real detectado en fase LAB (RC-000217): `_post_cl0` formateaba bytes con %s e inyectaba el repr literal b" como prefijo smuggleado, corrompiendo el framing de TODA la sonda. Fix: serializacion explicita a str antes de formatear. Es exactamente el tipo de defecto que el corpus existe para impedir que vuelva.
- Validacion: matriz 5/5 escenarios lab; corpus 93/93 (RC-000217..222, invariantes de presupuesto, precedencia RST, escalera y bytes-repr).

v0.86.0 — WCD-CACHE-KEY

Web Cache Deception como familia nativa del motor, con la disciplina de la linea DESYNC: contratos pre-registrados en el ExperimentGraph, juez determinista, cero-FP por diseno y presupuesto de 8 requests.

- `core/wcd_bait.py`: auditor `wcd_audit()`. Bateria de 4 disfraces (D-SEMI ';' + .css, D-DDOT '././', D-QM '%3F', D-HASH '%23') sobre un endpoint autenticado; presupuesto 2 baseline + 2 variantes x 3 = max 8 reqs.
- Contrato del baseline: A0 sesion@base debe contener el marcador autenticado (sin marcador: NO-MARKER, no hay linea base observable); D0 anon@base NO debe verlo. Si D0 ya lo ve -> OPEN-ENDPOINT: candidato BAC, familia distinta, NO es WCD y el presupuesto no se gasta en variantes.
- Escalera determinista por variante: A sesion+disfraz (leak?), B sesion+disfraz (HIT repro?), C anon+disfraz (contaminacion?). BENIGN -> LEAK-NO-CACHE (leak sin almacenamiento positivo: probable pero NO reportable) -> WCD-CACHE (HIT/Age observable) -> WCD-DEMO (contaminacion anon observable + controles PASSED). Vary sobre Cookie o Set-Cookie en la respuesta descalifican el DEMO.
- Clases de evidencia: E-WCD-LEAK / E-WCD-CACHE / E-WCD-CONTAM con genealogia en el grafo (run_id, pid, conexion) y journal append-only.
- `labs/wcd_lab.py`: 5 escenarios deterministicos con `ThreadingHTTPServer` simulando edge+origin (cache keyed por path normalizado, router confuso que recorta en delimitadores): `benign` (19184), `leak_nocache` (19185), `cache_vary` (19186, HIT con Vary: Cookie), `contamination` (19187, cache que ignora cookies), `open_endpoint` (19188).
- RC-000211..216: disfraz sin leak -> BENIGN; leak sin almacenamiento no reportable; HIT con Vary: Cookie no escala a DEMO; contaminacion anon con controles PASSED -> WCD-DEMO; endpoint abierto NO es WCD (reqs=2, presupuesto intacto); invariantes de bateria/presupuesto. Corpus: 87 casos.
- Robustez del corpus: RC-000161/RC-000162 migrados de sleep fijo (1.0s/1.2s) a la espera activa `_esperar_labs` ya usada por los casos DESYNC nuevos; bajo carga el lab podia no escuchar todavia cuando la auditoria conectaba

(ConnectionRefused) y el corpus abortaba antes de las WCD-RC. Corrida completa: 87 casos, 0 fallos.

FASE LAB: probado solo contra laboratorios locales. La integracion al Hunter (nodo del pipeline + UI) queda pendiente de la aprobacion del operador tras esta fase.

En vivo (linktr.ee, 05/10/2026, LestGo del operador, cuenta ninja via mail.tm + GHOSTGATE para el CF del login Descopes): endpoint autenticado /admin/links, marcador = username. Baseline STABLE (A0 con marcador, D0 anon -> login). Bateria: D-SEMI/D-QM/D-HASH -> BENIGN (SPA 404 y body crudo sin marcador via view-source). D-DDOT ('/admin/links/.../x.css') -> LEAK observable 2/2 (el origin sirve el dashboard autenticado ante el disfraz, confusion de router real) pero SIN almacenamiento positivo en el edge: anon + disfraz recibe login (sin contaminacion) y las respuestas admin via 'cache-control: private, no-store' + 'x-lt-cache: E-PASS'. VEREDICTO: LEAK-NO-CACHE (probable, NO reportable). Presupuesto respetado (~27 reqs en total incluido el registro). Nota GHOSTGATE: 'Just a minute' del CF se resolvió solo; el modal 'Your Privacy Choices' bloqueaba el paso intermedio del registro y hubo que cerrarlo. El reset de contexto no limpia cookies de la sesion viva: hay que stop + sesion nueva para el anon real.

v0.85.0 — PIPE-CROSS

El Hunter automatico (/api/scan) ahora invoca la auditoria CROSS-LAYER (v0.83 grafo + v0.84 veredicto) como nodo nativo del pipeline, despues de SECURITY. Cada caza deja tarjeta X-LAYER en la UI (veredicto, baseline, reqs, experimentos, run_id del grafo) y linea humana en el log del job. El presupuesto se mantiene: <= 11 reqs por disparo.

Lab: ThreadingHTTPServer (un edge real acepta conexiones concurrentes; el single-thread se bloqueaba con keep-alive de clientes previos). Regresion RC-000210 asegura el cableado backend+UI+presupuesto. Corpus: 81 casos.

v0.84.0 — CROSS-LAYER CORRELATION

Un disparo, tres capas observadas a la vez (Edge / Cache / Origin) + dimension de conexion (A, B, control). Antes cada modulo media una capa aislada; v0.84 correlaciona la MISMA perturbacion a traves de capas y conexiones en un solo experimento del grafo.

- core/cross_layer.py: cross_audit() dispara la bateria de perturbaciones (P-CASE, P-SLASH; presupuesto 11 reqs) y registra 4 observaciones por perturbacion (A conn1+reuse, B conn2, C control nuevo = contaminacion, D control fresco = semantica del origin). Capa EDGE: status + persistencia de conexion. Capa CACHE: fingerprint completo (Age, X-Cache, ETag, Vary...). Capa ORIGIN: status + hash de cuerpo.
- Clases de evidencia E-XL-MISMATCH (origin sirve distinto lo que el edge acepto), E-XL-SHARED (efecto reproducido desde 2 conexiones: estado compartido) y E-XL-CONTAMINATION (una conexion nueva recibe el efecto sin pedirlo). Todas con contratos pre-registrados.
- Escalera determinista: BENIGN (absorbida o EDGE-ONLY: tolerancia del edge NO es vulnerabilidad) -> SUSPICIOUS (mismatch no reproducido) -> SHARED-STATE -> DEMO (exige contaminacion observable + controles PASSED: Vary sensible o cookies descalifican).
- Veredictos honestos: baseline AMBIGUO o diff no observable -> UNKNOWN; UNREACHABLE si el baseline falla.
- labs/cross_layer_lab.py: 4 escenarios deterministicos (absorbed 19180, edge_local 19181, shared_state 19182, contamination 19183) con HTTP/1.1 real para medir persistencia. Ningun puerto colisiona con labs previos.
- RC-000205..209: absorbed->BENIGN, edge_local->BENIGN (EDGE-ONLY sin escalada), shared_state->SHARED-STATE (no DEMO sin contaminacion), contamination->DEMO, invariantes de presupuesto y escalera. 5/5 PASS.

v0.83.0 — EXPERIMENT-GRAPH: la investigacion como grafo

Objetivo: que codexRC construya y recorra un grafo de investigacion experimental (hipotesis, experimentos, observaciones, evidencias, resultados, siguiente experimento) para la linea DESYNC. La corona no se fabrica: se construye el camino hacia ella.

MODULOS: core/experiment_graph.py — el backbone. Nodos

HYPOTHESIS/EXPERIMENT/OBSERVATION/EVIDENCE, journal append-only (la evidencia previa nunca se sobrescribe), provenance (run_id, target, git_commit) en todo. Contratos registrados ANTES de ejecutar (re-registrar post-hoc: rechazado). Estados de hipotesis: LIVE / SUPPORTED / CONTRADICTED (terminal) / INCONCLUSIVE /

CLOSED. Sobrevivir NO demuestra: solo evidencia explicita apoya.

core/desync_hunt.py — la investigacion DESYNC sobre el grafo, consumiendo EDGESYNC (v0.74) sin reemplazarlo. Cada sonda = EXPERIMENTO con contrato; cada disparo = OBSERVACION con huellas de reproducibilidad completas (request fp, identidad de conexion, reuso, timing, respuesta, estado, cache, baseline, controles, genealogia); el contrato convierte senales en EVIDENCIA.

FILOSOFIA ANTI-FALSO-POSITIVO (la leccion del lab): El eco puede aparecer en PRE (pipeline del edge o split del backend: desde una sola conexion son INDISTINGUIBLES, atribucion honesta UNKNOWN) o en POST (la sonda recibio una respuesta que no pidio: efecto de estado OBSERVABLE). Solo el eco-post produce E-DESYNC-STATE. El lab consistente (pipelining) queda en REPRODUCIBLE sin efecto de estado; el lab desync demuestra P1 eco-state y llega a IMPACT-CANDIDATE.

ESCALERA DE IMPACTO (nunca se salta): DESYNC OBSERVED -> REPRODUCIBLE (2 genealogias independientes) -> STATE EFFECT -> CROSS-CONNECTION -> SECURITY IMPACT. CONFIRMED exige E-DESYNC-IMPACT + invariantes limpias (baseline caracterizado, multiples anomalias, reproducibilidad, impacto). El candidato nunca salta de anomalia a vulnerabilidad.

SELECCION: EDV (v0.82) sobre hipotesis LIVE; el contrato sigue siendo la autoridad; EDV 0 -> parada honesta con razon citando las vivas.

PERSISTENCIA: .codexrc/intelligence/experiments// .json — nunca se sobrescribe una investigacion; latest_run por campo created (no por nombre). Completa el dossier longitudinal.

CLI: python3 core/desync_hunt.py [--json] y --cmd active|graph|next|explain [--exp EXP-XXXX].

Validacion: RC-000195..204 (construccion, lifecycle, provenance anti post-hoc, EDV, contradiccion con historial preservado, reproducibilidad por genealogias, lifecycle del candidato, invariantes anti-FP, persistencia, explain-path). Labs: desync -> V1/V6/V7 eco-cand + P1 eco-state -> IMPACT-CANDIDATE; consistente -> V2/V9 eco-cand (pipeline) -> REPRODUCIBLE sin efecto de estado. SECURITY IMPACT: NOT DEMONSTRATED en ambos.

v0.82.0 — GRAFO DE EXPERIMENTOS: seleccion por EDV

Pregunta central: dadas las hipotesis vivas y el presupuesto, ¿cual es la siguiente prueba que mas informacion me puede dar?

MODULO: core/experiment_selector.py. El loop fijo por dependencia (v0.81) se reemplaza por un selector que elige en cada paso.

EDV (Valor de Discriminacion Esperado) por experimento: para cada RAMA POSIBLE de su contrato (union estatica pre-registrada, parte del contrato, no una prediccion): CONTRADICT sobre una viva = 2.0 (es terminal: reduce el conjunto vivo de forma determinista) SUPPORT sobre una viva = 1.0 (prepara convergencia) EDV = media de valores de rama. Sin probabilidades inventadas: ramas uniformes porque NO se conoce su probabilidad. EDV ordena; el CONTRATO decide evidencia.

DAG de dependencias respetado (INTRA -> CROSS -> VIRGIN, INTRA -> TIME): un experimento sin prerequisite no es elegible. Desempates: EDV, epsilon por discriminante en ventana previa de memoria, orden del catalogo.

PARADA HONESTA: si ningun experimento disponible puede tocar una hipotesis viva (EDV 0 en todos), el loop se detiene y declara la razon citando las vivas. El UNKNOWN conserva esa razon, no un silencio.

EFFECTO MEDIBLE: origin_dynamics: 14 req -> 10 req (convergencia en 2 experimentos, el selector para al quedar 1 viva). lb_variance: mismo veredicto y presupuesto (19 req), pero orden guiado por EDV (VIRGIN EDV 2.5 corre antes que SESSION 2.0) con tabla visible en cada eleccion.

Validacion: RC-000191 (convergencia temprana 10 req, EDV en cada eleccion), RC-000192 (parada honesta EDV=0 con razon citando H2,H4), RC-000193 (orden por EDV con tabla visible), RC-000194 (invariantes v0.81/81.1: memoria y corpus ADAPTIVE re-ejecutados PASS). Corpus 65/65.

v0.81.1 — RESEARCH-MEMORY: el motor recuerda

Pregunta central: si vuelvo a un target mañana, ¿empiezo de cero o se que se ya se probó y que quedo sin resolver?

ALCANCE: core/research_memory.py + integracion en adaptive_hunt. Dossier por host en CODEXRC_HOME/memory/research// (dossier.jsonl una linea por ventana + latest.json). Respeta CODEXRC_HOME: dos IAs no comparten dossier.

TRES REGLAS DE HONESTIDAD: 1. Solo observaciones: se guarda lo visto, lo ejecutado y lo descartado CON SU NOTA. Sin inferencias. 2. Las conclusiones CADUCAN POR VENTANA: al re-entrar, el grafo se siembra FRESCO y lo previo entra como ANOTACION ("ventana previa : SUPPORTED"). Contexto, nunca veredicto heredado. 3. Reuso determinista ÚNICAMENTE con evidencia fuerte: baseline identico (mismas huellas, mismo disparador) dentro del TTL (default 6h, --ttl). Mismo observable = mismas condiciones; baseline distinto = ventana nueva y corrida nueva, siempre.

EFFECTO MEDIBLE: sesion repetida con baseline identico pasa de 19 requests a 3 (reuso). Baseline distinto o TTL agotado: corrida fresca con historial visible.

CLI: --resume, --ttl, --history (dossier ventana a ventana).

Fix: reuso no pasaba las huellas del baseline a la sesion (TypeError, atrapado por el test manual antes de regresion).

Validacion: RC-000187 (reuso 3 req REUSADO), RC-000188 (TTL agotado -> ventana nueva 19 req + anotacion previa), RC-000189 (baseline distinto -> cero reuso), RC-000190 (invariantes v0.81: RC-000183..185 re-ejecutadas PASS). Corpus 61/61.

v0.81.0 — ADAPTIVE-HUNT: la capa epistemica

Pregunta central: no "que vulnerabilidad puedo probar" sino "que experimento me daría la mayor informacion para distinguir entre las hipotesis que tengo".

Flujo: OBSERVACION -> HIPOTESIS -> EXPERIMENTO (contrato pre-registrado) -> EVIDENCIA -> ACTUALIZAR -> ... -> VEREDICTO + GAP LEDGER. Un UNKNOWN ya no es un punto muerto: es una ruta con la evidencia que falta enumerada.

MODULOS: hypothesis_graph: H1..H5 (load_balancing, personalization, cache_variation, bot_management, origin_dynamics) con reglas deterministas: SUPPORT exige evidencia positiva, CONTRADICT exige observacion discriminante y es terminal, el resto queda UNKNOWN experiment_catalog: 5 sondas HTTP de solo lectura con CONTRATO pre-registrado (INTRA-CONN, CROSS-CONN, SESSION, VIRGIN, TIME-OFFSET); cada contrato declara ANTES de ejecutar que resultado apoya o refuta que hipotesis adaptive_hunt: orquestador con juez determinista y tres salidas validas: BASELINE-CARACTERIZADO (con soporte o por eliminacion honesta marcada BY-ELIMINATION), BASELINE-STABLE y UNKNOWN-DEMOSTRADO (razon demostrable e hipotesis abiertas enumeradas) labs/adaptive_lab.py: 3 mecanismos deterministas

FIXES DE CONTRATO (atrapados por el lab): TIME-OFFSET mezclaba dimension temporal con conexion (falso soporte H5 en lb_variance): ahora mide sobre la misma conexion CROSS re-aplicaba contratos de INTRA ya ejecutado (notas duplicadas)

Validacion: 3/3 lab (lb->H1 SUPPORTED, origin->H5 BY-ELIMINATION, transient->UNKNOWN-DEMOSTRADO con H1/H3 vivas y razon citada); corpus 57/57 (RC-000183..186, invariantes v0.80 re-ejecutadas).

Sesiones vivas linktr.ee: 2x BASELINE-STABLE (3 req cada una). La ambigüedad observada por SEMANTIC-CACHE v0.80 era intermitente: queda en el dossier longitudinal como observacion de ventana, sin inferencia.

CLI: python3 core/adaptive_hunt.py [--timeout N] [--json]. Presupuesto heredado: 30 requests.

v0.80.0 — SEMANTIC-CACHE: desacuerdos semanticos (fase 2 del estado compartido)

Pregunta central: dado un diferencial o una convergencia de cache, CUAL es el desacuerdo semantico, es reproducible en segmento virgen, y cambia lo que recibe un consumidor inocente?

EXTENSION DE v0.79 (patch aditivo: cache_correlation expone fingerprints e informe79 sin tocar su veredicto).

CUATRO MODULOS: semantic_channels: canales de observacion (respuesta directa, estado de cache, atribucion) con genealogia por sonda; dos canales solo cuentan si son mecanismos independientes (caso de header: el contrato RFC 7230 3.2 NO es canal independiente -> nunca correlation STRONG solo por el) semantic_dimension: fase 2 localiza la dimension (colision por segmento de path vs fragmentacion por header) y descarta SELF-INDUCED en path virgen; correlation OBSERVED solo si reproduce semantic_models: VerdictRecord con veredicto, atributos, condiciones, presupuesto (tope 30 req) y genealogia exportable semantic_judge: E1..E9 -> BENIGN / CONSISTENT / INCONSISTENT / SUSPICIOUS / DEMO / UNKNOWN; DEMO exige 8 condiciones conjuntas (reproducible + localizada + atribuida + controles + baseline util + self-induced descartado + correlacion + impacto reproducible)

FALSOS POSITIVOS DE v0.79 CERRADOS: collision_legitima (alias por diseno): v0.79 daba DEMO falso; v0.80 da BENIGN via control negativo (B-solo ve lo mismo legitimo, nada es ajeno) -> RC-000174 [MISS,HIT] vs [HIT,HIT]: era acuerdo en entrada compartida, no desacuerdo -> CONSISTENT -> RC-000180

LAB (labs/semantic_lab.py): 8 escenarios con cache-fingerprint de estado por respuesta + eco de segmentos. http.server normaliza el case de headers: el case crudo se lee de headers.as_string().

REGLAS: tolerancia != vulnerabilidad: INCONSISTENT sin impacto observable jamas escala (RC-000177) inconsistencia != vulnerabilidad: E7 con dos canales reales y sin impacto -> maximo SUSPICIOUS (RC-000181) baseline AMBIGUO -> UNKNOWN conservador, sin fase 2 (RC-000179)

CLINICA: `python3 core/semantic_engine.py <url> [--timeout N]`.

Validacion: 8/8 escenarios lab; corpus 53/53 (RC-000174..182, invariantes v0.72-v0.79 re-ejecutadas).

v0.79.0 — CACHE-CORRELATION: el estado compartido como evidencia

Pregunta central: ¿dos representaciones que deberian ser equivalentes generan estados de cache distintos, o dos representaciones distintas terminan compartiendo un estado que no deberian compartir?

REGLA FUNDAMENTAL: no se asume ni afirma conocer la cache key interna. CacheBindingEvidence expresa FUERZA DE EVIDENCIA EXTERNA compatible con binding/correlacion; cache_key(A) == cache_key(B) es solo hipotesis. Escalera intacta: hipotesis != observacion != evidencia != veredicto.

CINCO MODULOS (core/cache_*.py): semantic: relacion A<->B (EQUIVALENT/DISTINCT/UNKNOWN) con ficha auditable; conjunto CERRADO de transformaciones neutrales demostrables (T-IDENT, T-HEADER-CASE RFC 7230 3.2); lo no demostrable -> UNKNOWN, nunca EQUIVALENT forzado fingerprint: senales observables tri-estado (OBSERVED/ABSENT/UNKNOWN); header ausente NO es cache miss; Age/Date/timing son secundarias: NUNCA producen diferencial solas baseline: STABLE / VARIANT (solo variacion CARACTERIZADA: temporal o load-balancing) / AMBIGUO / INVALID; AMBIGUO e INVALID -> UNKNOWN conservador controls: PASSED/FAILED/NOT_APPLICABLE con evidencia real (cookies, authorization, vary, UA/accept, ttl_wait re-sonda, dynamics/bot via baseline, geo honesta NOT_APPLICABLE); nada decorativo correlation: orquestador SemanticRelation -> Fingerprint -> Baseline -> Controls -> Correlation -> Target Judge

VEREDICTOS: STABLE / BENIGN / DIFFERENTIAL / SUSPICIOUS / UNKNOWN / DEMO. Juez determinista, no aditivo, sin confidence=0.85.

EVIDENCIA E0-E6 (condiciones, no puntos): E0 observacion, E1 reproducibilidad, E2 diferencial semantico, E3 cache binding, E4 diferencial downstream, E5 impacto cross-consumer, E6 impacto de seguridad.

DEMO es dificil por diseno: exige binding STRONG + E5/E6 + controles criticos PASSED + especificidad de path demostrada + baseline no ambiguo + atribucion reproducible. NUNCA por Age diferente, HIT/MISS, ausencia de headers, body diferente, dos requests distintos, convergencia aparente, inferencia de cache key, o E0+E1.

Dos fixes surgidos de la validacion en lab y en vivo: - especificidad de path: un target que sirve contenido identico para paths distintos converge trivialmente (aparente); la convergencia solo es atribuible si el target demuestra servir contenido especifico por path - binding honesto: convergencia reproducible SIN atribucion es WEAK aparente, no STRONG; el diferencial EQUIVALENT con senales explicitas se evalua antes que la convergencia aparente - DEMO acepta baseline STABLE o VARIANT caracterizado ("no ambiguo"), no solo STABLE exacto

LAB (labs/cache_lab.py): 6 escenarios deterministas: consistent, ttl_variant, personalized, bot_ambiguous, divergent_equivalent, convergent_distinct. El sistema diferencia estabilidad / diferencial real / personalización legítima / variación temporal / baseline ambiguo / estado compartido indebido.

CORPUS: RC-000168 (consistent STABLE + ttl_variant STABLE/baseline VARIANT) / RC-000169 (divergent_equivalent SUSPICIOUS) / RC-000170 (personalized BENIGN por Vary+Cookie) / RC-000171 (bot_ambiguous UNKNOWN) / RC-000172 (convergent_distinct DEMO por E5+E6) / RC-000173 (invariantes v0.72-v0.78: re-ejecuta RC-000164..167 via el nuevo flag --only; nada retroactivamente DEMO). Corpus 44/44. Renúmero los cache en 168-173 porque RC-000167 ya existe como caso histórico v0.78 (intacto).

Robustez: import subprocess al inicio de run() (con --only los imports locales condicionales no habían corrido); _esperar_labs reutilizado para los labs de cache.

En vivo (linktr.ee): BENIGN honesto (convergencia aparente por páginas idénticas, binding WEAK aparente, sin atribución); baseline VARIANT/AMBIGUO según la corrida: sin poder discriminatorio no se concluye.

Presupuesto: 17 requests por target, secuencial, cooldown 0.4s, cuerpos limitados, sin loops agresivos.

v0.80 queda FUERA de esta versión (SEMANTIC-CACHE: collision, fragmentation, cross-semantic contamination se investigará CON la evidencia de v0.79).

v0.78.0 — STATE-CORRELATION: un veredicto por target

No agrega payloads. Une lo que v0.75 (contrato del edge), v0.76 (representación) y v0.77 (estado de conexión) observan sobre el MISMO target en una ficha de estado única: reuse, aceptadas, baseline_varianza, n_resp, b1_tragado, cross_connection_effect, reproducibilidad, cierre tras A.

CUATRO RESULTADOS (solo el último alimenta la escalera): STABLE PIPE-BENIGN sellado: re-observar el desplazamiento mil veces jamás escala (regla del falso DEMO) STATE-CHANGED reproducible sin evidencia cross-connection CROSS-CONNECTION-MISMATCH

EVIDENCE QUALITY E0-E5, anotación transversal por capa: REPRESENTATION: E2 diferencial / E3 ECO downstream (te=none ES observación downstream: reporta la normalización que llegó al origen) CONNECTION-STATE: E4 cross-connection SECURITY-IMPACT: E5 respuesta ajena entregada a una conexión inocente reproducible

JUEZ determinista, no suma: DEMO exige E4+E5 SIEMPRE; E3 sube confianza, no es requerido; PIPE-BENIGN sellado; baseline inestable -> UNKNOWN en todas las capas (sin poder discriminatorio no se concluye). La confianza sube por evidencia INDEPENDIENTE que reduce incertidumbre, no por repetición de una anomalía.

ATRIBUCION por capa: layers_con_evidencia vs unknown_layers (cache, origin, balanceador, aplicación); atribución fina por capa queda para ATTRIBUTION-CHAIN.

Regresión RC-000167: desync-pool -> CROSS-CONNECTION-MISMATCH (E4+E5 -> DEMO); eco-normaliza -> PIPE-BENIGN sellado con vector E3+E2+E0 PRESENTE y sin escalar (la prueba del sello); consistente -> STABLE (E0/E0/E0). Corpus 38/38.

Robustez: _esperar_labs (espera activa de puertos) en los 5 bloques de labs de la regresión; el fix tras un Connection Refused esporádico: 1.5s fijo no alcanza bajo carga.

En vivo (linktr.ee): UNKNOWN-BASELINE honesto (3 huellas en el propio baseline, reuse=no); atribución lista capas unknown.

v0.77.0 — CONNECTION-STATE AUDIT: el estado es el testigo

Pregunta: ¿deja una petición ambigua un estado observable que modifica la interpretación de una petición posterior sobre la misma conexión?

HALLAZGO DE DISEÑO (el más importante de la serie): un edge que honra TE (RFC 7230, CORRECTO) produce el MISMO desplazamiento observable en conexión única que un desync real. La sonda de conexión única NO puede distinguirlos: "B cambio de respuesta" no es smuggling. La diferencia verdadera es de IMPACTO

CROSS-CONNECTION.

Sonda T2 VENENO QUEUE-POISONING (self-cleaning): prefijo = POST incompleto con Content-Length EXACTO = B1 + GET inocuo. - front honra TE (correcto): B1 queda absorbido por el request del propio cliente; la conexion inocente recibe SU respuesta -> STATE-STABLE+PIPE (benigno, documentado). - front honra CL y back TE con back COMPARTIDO (pool): B1 es tragado y el GET inocuo de OTRA conexion completa el cuerpo y RECIBE la respuesta del POST ajeno: contaminacion cross-connection reproducible -> DEMO.

Control de conexion inocente (conn2): GET limpio en conexion nueva despues del tratamiento. Solo si conn2 recibe respuesta ajena (marker) o silencio hay desacuerdo de CADENA; si conn2 queda limpia, lo ocurrido es pipelining legitimo del front.

Baseline con varianza: K=3 corridas A0->B0; B1 debe caer fuera de la distribucion para contar. Si el baseline varia solo (Linktree: bot management rotando respuestas) -> BASELINE-AMBIGUO: sin poder discriminatorio no se concluye nada. Verificado en vivo.

STATE-CONTINUITY como evidencia: n_resp, estado del parser (respondio A? cuantas?), huella de respuesta de B, latencia, silencio, cierre.

Escalera: STATE-CHANGED -> reproducible? no -> UNKNOWN; si -> SOSPECHA; y si ademas la conexion inocente recibio respuesta AJENA reproducible -> DEMO.

Lab nuevo: modo desync-pool (front honra CL, back TE-strict, socket back COMPARTIDO entre clientes con cola FIFO de respuestas): topologia real de pool donde el smuggling cruza clientes. El desync por-cliente (sin pool) queda contenido: solo auto-dano, veredicto honesto PIPE.

Regresion RC-000166: desync-pool -> DEMO (conn2_marker=True); eco-normaliza -> STATE-STABLE+PIPE (el falso DEMO de la primera version quedo muerto); consistente -> STATE-STABLE. Corpus 37/37.

Presupuesto: perfil (11) + baseline (6) + T1 por variante (2) + T2 (3: A1+B1+conn2) + repro (3), tope 32, secuencial.

En vivo (linktr.ee): BASELINE-AMBIGUO honesto (3 huellas distintas en el baseline); T2 sin estado observable.

v0.76.0 — NORMALIZATION-AUDIT: que recibe el origen

Pregunta: cuando el edge ACEPTA un framing contradictorio, ¿que representacion termina recibiendo el origen?

REGLA DE ORO (del operador): la tolerancia del edge NO es una vulnerabilidad por si misma. Este modulo describe transformaciones OBSERVABLES; no convierte una transformacion en hallazgo automaticamente. Lo no observable permanece unknown.

Sonda central: BODY-DIFFERENTIAL, benigna por construccion. Cuerpo "5\r\nhello\r\n0\r\n\r\n": quien lee por TE entiende cuerpo "hello"; quien lee por CL entiende 17 bytes crudos. Las DOS lecturas consumen el mensaje completo: sin smuggle, sin leftover. El probe se contrasta contra dos canonicos limpios (cuerpo "hello" / cuerpo crudo) y contra el ECO del downstream (X-Received-*).

Veredictos por variante: NORMALIZED (ECO te=none) | CONSERVED (ECO te intacto, respuesta = canonico crudo) | MISMATCH (ECO muestra que el origen ACTUO sobre la lectura TE mientras la respuesta coincide con el canonico A: dos capas, dos interpretaciones) | REJECTED | UNKNOWN.

Doble honestidad: - Escalera MISMATCH: reproducible -> estado/conexion -> evidencia downstream -> SOSPECHA. DEMO queda para v0.77+. - Poder discriminatorio: si los canonicos A y B son indistinguibles para el target, NINGUN match del probe significa algo y el veredicto es UNKNOWN (no "aparente"). Verificado en vivo: un "NORMALIZED-APARENTE" falso se convirtio en UNKNOWN.

Regresion RC-000165: eco-normaliza -> NORMALIZED/MATRIX; eco-conserva -> MISMATCH con escalera completa hasta SOSPECHA (reproducible + ECO); rechaza -> RECHAZADO-TOTAL. Corpus 36/36.

En vivo (linktr.ee): acepta cl_te/te_doble/te_case, rechaza te_espacio; representacion downstream UNKNOWN (sin ECO y sin poder discriminatorio). Ficha honesta: MATRIX.

Presupuesto: 11 (perfil) + 3 por variante aceptada, tope 24, secuencial, lectura A->B, cuerpos <= 17 bytes.

v0.75.0 — EDGE-PROFILE: el contrato observable del edge

Primera release del roadmap v0.75-v0.80 (EDGE-DIFFERENTIAL). No busca desync y no lanza smuggles: responde "¿que transformaciones aplica cada frontera antes de entregar el mensaje a la siguiente capa?" con sondeos BENIGNOS (framing contradictorio con cuerpo VACIO, sin contenido smuggleado) y nada inferido: lo no observable queda unknown.

Ficha: version HTTP observado | reuse (3 GETs en 1 conexion) | cierre_rechazo | cl_te / te_doble / te_espacio / te_case (rejected|accepted|unknown) | normalizacion (normalized|conserved|unknown, observable solo si el downstream hace ECO) | downstream (reached|unknown) | cache (firma Age/ X-Cache entre repeticiones) | origin (unknown: no observable).

Lab: modos "eco-normaliza" (edge RECONSTRUYE: consume el cuerpo bajo su lectura y reenvia CL puro sin TE) y "eco-conserva" (edge reenvia crudo); el backend devuelve X-Received-Te/Cl/Path con lo que llego, volviendo la transformacion un hecho medible.

Regresion RC-000164: normaliza -> normalized+reached; conserva -> conserved+reached; rechaza -> rejected+cierre yes+unknowns. Corpus 35/35.

Primera ficha real (linktr.ee, edge Varnish): HTTP/1.1, reuse no (cierra tras 2 requests), cierre_rechazo yes, y el hallazgo de CONTRATO: ACEPTA cl_te, te_doble y te_case pero RECHAZA te_espacio (400+close). El siguiente paso del roadmap (v0.76 NORMALIZATION-AUDIT) debe determinar que hace con lo que acepta.

Presupuesto: 11 requests por ficha (1 conexion de reutilizacion + 4 probes de 1 conexion), secuencial, lectura A->B.

v0.74.0 — EDGESYNC-V3: SONDA-DE-CORRELACION

Salto de arquitectura: el veredicto ya no nace de una sola dimension (el eco). Cada variante se observa en TRES dimensiones sobre la misma conexion:

dim1 framing respuesta al mensaje ambiguo (eco/conteo) dim2 persistencia la conexion sobrevive al ambiguo, o el edge la mata? un cierre activo del edge significa algo muy distinto a una conexion que sigue con estado inconsistente dim3 estado la sonda conserva su estado esperado, o llega desplazada/envenenada?

La pregunta que responde: ¿el comportamiento observado demuestra que DOS componentes interpretaron el mismo mensaje distinto? Escalera de veredictos: RECHAZO-EDGE (el edge corto el framing: postura activa, nada observable, distinto de un blanco limpio) | SIN-DESYNC | SOSPECHA (una dimension discrepa, eco y sonda anomala exigen REPRODUCCION) | DESYNC-DEMO.

Nuevos payloads POISON: P1 (CL.TE) y P2 (TE.CL), prefijo de request SIN terminar: el back lo pega a la sonda y la respuesta de la sonda sale por el camino del veneno. Detecta desync por desplazamiento de la sonda SIN eco clasico (dimension 3 pura).

TOPOLOGY-PRE: el baseline se parsea (Server/Via/CF-Ray/Age/ X-Cache) y clasifica el blanco: cdn-blindado -> bateria reducida (V1, V2, P1); proxy-intermedio o directo -> bateria completa. La topologia queda en la evidencia y alimenta UNIVERSAL-RECON.

Lab: modo "rechaza" (edge 400+close aunque el back sea vulnerable): antes indistinguible de un blanco limpio, hoy veredicto propio. Regresion RC-000163: P1 demuestra desync por poison contra desync, calla contra consistente, RECHAZO-EDGE contra rechaza; P2 limpio contra consistente-te; bateria vs rechaza = RECHAZO-EDGE; clasificador de topologia 3/3. Corpus 34/34.

Prueba en vivo (linktr.ee): clasificado proxy-intermedio (Varnish doble via + cache MISS/HIT). Rechaza 400 en V2/V4/V8/V9/P2, cierra la conexion tras responder en V1/V3/V5/V6/V7 (dim2: postura defensiva), y P1 deja sonda anomala REPRODUCIBLE (SOSPECHA honesta: algo en la cadena pego o rechazo el veneno; diferencial manual pendiente, sin demo).

Limitacion documentada (heredada): front que honre TE estricto puede reflejar pipelining como eco CL.TE; los ecos TE.CL y P2 siguen siendo candidatos.

v0.73.0 — EDGESYNC-HUNT V2: bateria de 9 framings

Una sola sonda CL+TE no basta: cada parser resuelve la ambigüedad a su manera. La bateria ahora dispara NUEVE framings por target y para al primer eco reproducido.

Familia CL.TE (smuggle dentro de la ventana CL, invisible al front CL; un back TE lo ejecuta): V1 clasico | V3 TE duplicado | V4 espacio antes de ":" | V5 identity+chunked | V6 tab tras ":" | V7 mayusculas ofuscadas | V8 mixto (espacio+duplicado) Familia TE.CL (smuggle dentro del data de un chunk, invisible al front TE; un back CL lo ejecuta): V2 invertido (CL corto) | V9 extension de chunk + hex mayus

Veredictos: DESYNC-DEMO exige eco REPRODUCIBLE (familia CL.TE); los ecos TE.CL quedan como TECL-CANDIDATO (desync o pipelining: ambiguo sin prueba cruzada, no dictamina DEMO); conteos anormales sin eco = SOSPECHA; rechazo activo del front (400/405/501 al framing ambiguo) = buena postura, se registra como front_rechazos y NO es sospecha. Presupuesto: 12 pruebas max (baseline + 9 + reproduccion), 1 conexion secuencial por variante, cooldown, sin flood/RST, cuerpos <= 4KB.

Lab v2 (labs/desync_lab.py): 5 modos (desync, consistente, tecl, consistente-te, lenient) con parsers strict (rechaza TE duplicado/espaciado) y lenient (los honra). La matriz de regresion RC-000162 exige que cada variante distinga el parser que le toca: V3/V4 callan al strict y gritan al lenient; V6/V7 atraviesan el strict; V2/V9 dejan candidato solo contra tecl y limpio contra consistente-te. Corpus 33/33.

Prueba en vivo (Linktree/Cloudflare): SIN-DESYNC. Front rechaza activamente V2/V4/V8/V9 (400+close) y resuelve consistente V1/V3/ V5/V6/V7. Postura solida; el modulo queda listo para edges menores y VDP self-hosted.

Limitacion documentada: un front que honre TE estricto puede reflejar pipelining como eco CL.TE; en blancos CDN (front normalizado) la senal es solida, en fronts TE puros se exige diferencial manual.

v0.72.0 — EDGESYNC-HUNT: detector de desync edge->back

Primer modulo de capa transporte del engine (hasta ahora eramos capa app PHP/JS). Automatiza la metodologia de request smuggling (Kettle): hacer que front y backend discrepen sobre donde termina un request.

TECNICA DEL ECO (core/edgesync.py): POST ambiguo con Content-Length que cubre TODO el request smuggleado + Transfer-Encoding: chunked. El front lee CL -> el smuggle es "cuerpo" (nunca un request para el). Un back que prefiera TE ejecuta el smuggle como request propio. El ECO: el cliente recibe una respuesta extra que nunca pidio, con un marker unico (X-Edge / path x-edgesync-sonde-).

Veredictos: DESYNC-DEMO (marker presente: bug demostrado, lectura A->B), SOSPECHA (conteo de respuestas anormal), SIN-DESYNC (todo cuadra). CLI: python3 core/edgesync.py http://target[:puerto]/ [--json].

SEGURIDAD ANTI-DOS (no negociable): max 3 requests por corrida (baseline + probe ambiguo + sonda), 1 conexion secuencial, cooldown 0.4s, cuerpo <= 4KB, sin loops ni flood ni RST. La bomba HTTP/2 (Rapid Reset) hace lo opuesto: miles de streams que se cancelan sin pagar; aqui el objetivo es el eco, no la exhaustion. Ademas: RACE-PROOF ahora clampa count a 30 (tope duro anti-DoS).

LAB VALIDACION (labs/desync_lab.py): edge CL -> backend TE (modo desync) o CL/CL (modo consistente), todo Python puro Termux-friendly. Bugs reales encontrados durante la validacion: (1) edge reenviaba raw_head.encode() sobre bytes -> AttributeError tragado por except silencioso (el request nunca llegaba al back); (2) parser chunked no actualizaba el buffer tras el chunk 0. Resultado determinista: lab desync = DESYNC-DEMO (3 respuestas, GET smuggleado ejecutado invisible al front), lab consistente = SIN-DESYNC (2 respuestas).

Regresion: RC-000161 levanta labs efimeros en puertos libres y exige el par exacto [DESYNC-DEMO, SIN-DESYNC]. Corpus 32/32.

v0.71.0 — ARSENAL-EXPANSION: RACE-PROOF + CSPT-SCAN + SAML-DEFENSE

Tres modulos nuevos pedidos por el operador (tecnicas calientes 2025-2026 que el engine no tenia):

RACE-PROOF (core/race_proof.py): ejecutor dinamico de carreras TOCTOU. El brazo que le faltaba a RACE-TRACE (v0.70.0): dispara N requests simultaneos (barrera de conexiones pre-abiertas HTTP/1.1, o single-packet HTTP/2 si hay lib h2 y el server habla h2) y dictamina RACE-DEMO / SIN-RACE contando exitos sobre los permitidos. Fix real durante validacion: http.client lanza "CannotSendRequest: Idle" si el request se envio crudo por socket; se pasa a lectura cruda del socket (status + Content-Length). Validado en vivo: 20/20 exitos contra lab TOCTOU, SIN-RACE contra endpoint endurecido con lock.

CSPT-SCAN (core/cspt_scan.py): Client-Side Path Traversal (clase 2025-2026). Detecta FUENTE (location.search/hash, URLSearchParams) -> SINK (fetch/axios/xhr) con path concatenado en .js del plugin/tema. Veredictos CANDIDATO-CSPT (alta/media) y MITIGADO-ENC (encodeURIComponent). Fix real: la concatenacion debe estar EN la linea del sink (evaluar la ventana completa producida FP en fetch de ruta fija vecino). Fixture 2/2 sin FP; ruido casi cero en plugins reales (astra: 1 candidato real).

SAML-DEFENSE (core/saml_audit.py): bypasses SAML/XML 2025 ("The Fragile Lock"). Detecta en PHP: XXE (parseo de XML con input usuario sin LIBXML_NONET), SAML-WSW (processResponse sin chequeo posterior de errores/validez), SAML-STRICT (strict=>false) y XInclude. Fix real: ->process() es omnipresente en PHP no-SAML (31 FP criticos contra DeskPro); WSW solo se evalua en archivos que usan un toolkit SAML real. DeskPro core verificado: limpio (su adapter valida getErrors + isAuthenticated correctamente).

Integracion PLUGIN-BATCH: rec trae cspt_candidatos/cspt_resumen y saml_candidatos/saml_resumen; resumen agrega los 3 contadores. Regresion: RC-000158 (race-proof con lab TOCTOU efimero), RC-000159 (cspt), RC-000160 (saml). Corpus 31/31 PASS.

v0.70.0 — RACE-TRACE: TOCTOU en estado persistente

Tecnica nueva incorporada al arsenal estatico: la clase de bug que se gana con single-packet HTTP/2 (limites, cupones, stock, saldos). Ningun plugin WP aguenta una carrera cuando hace leer -> decidir -> escribir sin lock.

core/race_trace.py: - Empareja read->write de estado persistente por funcion: get_option/update_option, _user_meta, _post_meta, transients, \$wpdb SELECT->UPDATE, ->get_stock_quantity/set_stock_quantity, ->get_usage_count/increase_usage_count, ->get_total/set_total. - VENTANA: detecta el guard (if) entre read y write que usa la variable leida = check-then-act clasico. - Veredictos: CANDIDATO-RACE / RACE-ATOMICO (SQL col=col+1, descartado) / MITIGADO-TRANSIENT (lock con transient) / MITIGADO-LOCK (flock/LOCK_EX/get_lock) / DESCARTADO-LECTURA (version/cache/flag sin valor) / DESCARTADO-VENTANA (contador RMW sin guard). - Severidad: critica si el handler es nopriv (carrera ANONIMA), alta para dinero/stock/cupon, media para limites. - Filtros de ruido v0.70.1: estado "otro" (versiones, caches, flags) descartado; transients de cache sin clave sensible descartados. astra 92->7 candidatos, beehive 16->3.

gates_audit.py fix (v0.70.0): - _resolve_callback: array('Cls','m') devolvia 'Cls' (la clase) como callback; el metodo se perdia y los handlers nopriv no levantaban severidad. Ahora resuelve el ULTIMO string del array. Impacto previo no cuantificado.

Integracion PLUGIN-BATCH: rec["race_candidatos"] + rec["race_resumen"] en el escaneo de cada slug. Regresion RC-000157: 4/4 veredictos + nopriv boost, corpus 28/28.

v0.69.0 — VISION-GATE: clasificador visual de challenges (RC-000155)

Gemini como SENSOR, no como conductor. Mismas reglas del JUEZ aplicadas a lo visual: el LLM clasifica, REG-BOT decide, y el modelo jamas toca el navegador.

core/vision_gate.py: - Entrada: screenshot opcional + DOM redactado (regex tapa emails/telefonos/tokens). Cero credenciales, cookies o secretos hacia el modelo. El screenshot solo en checkpoints REG-BOT. - system prompt VISION-GATE: no inventar elementos, no afirmar exito sin evidencia, salida EXCLUSIVAMENTE JSON. Few-shot E1-E5

(recaptcha v2, registro normal, SMS->descartar, interstitial CF, geetest ambiguo). - response_schema contractual (challenge_detected, challenge_type, confidence, state, action, reason) con temperatura 0. - Taxonomia: no_challenge | supported_checkpoint | unsupported_checkpoint | ambiguous | requires_human. - decide() determinista (SIN LLM): CONTINUE pasa solo; DISCARD solo con confidence >= 0.8 (regla telefono/KYC); GHOSTGATE solo sugiere; el resto cae a la cola de handoff. Sin clave o API caida -> ERROR y la caza conserva su comportamiento original.

Integracion reg_bot.py: el submit con CAPTCHA-PENDING primero clasifica; falso positivo de heuristica -> continua solo; el veredicto queda en identity["vision"] con resumen en el log.

Modelo por defecto gemini-2.0-flash (configurable via VISION_MODEL).

v0.68.0 — REG-BOT + AB-DIFF universal: identidades no-WordPress (RC-000154)

El concepto de la capa 4 (diff de sesiones) portado a CUALQUIER web. AUTHZ-PROOF ya no es exclusivo de WordPress: consume objetos Identity estandarizados y no sabe si la cuenta nacio en WP, Laravel, Django, Express o una app propia.

NUEVO core/reg_bot.py — PROVEEDOR UNIVERSAL DE IDENTIDADES: - FORM-DISCOVERY: lee el formulario (hints, patrones, honeypots), no lo adivina; descarta blancos que exigen telefono/KYC (regla permanente); CAPTCHA -> CAPTCHA-PENDING (cola de handoff al operador, nunca un sistema para vencer CAPTCHA) - CONSTRAINT-SOLVER + IDENTITY-GENERATOR: identidad plausible y unica por sitio; password siempre cumple politicas estrictas; alias con sufijo numerico y reintento condicionado a respuesta REAL del servidor (hasta 5) - REGISTRATION-FLOW: multi-paso, TODOS los campos password reciben la misma clave (confirm), formulario de verificacion LEIDO de la respuesta del registro - VERIFICATION-FLOW: codigo 4-8 digitos O enlace magico, polling corto (6s) porque los codigos caducan en minutos, reenvio hasta 3 ante expiracion - SESSION-HANDLER: captura cookies/csrf/tokens en el Identity - RECIPE-MEMORY: receta por sitio (no re-aprender nunca mas); identidades y recetas viven en CODEXRC_HOME (privado, jamas git) - Buzones: mail.tm (API real) o MockMailProvider (laboratorio) - Identity = {id, credentials, cookies, csrf, tokens, verification_state, registration_recipe, capabilities}

NUEVO core/ab_diff.py — DIFF DE SESIONES UNIVERSAL: - Niveles AUTOMATICOS: 1) registro libre -> ANON + A(duena de X) + B(independiente); 2) credenciales propias -> ANON + OWN; 3) nada -> ANON solo (fugas sin autenticacion) - Relacion A-duena-de-X / B-independiente: el diff es CONCLUYENTE (B ve los datos de A = BAC/IDOR demostrado, B bloqueado = REFUTADO con gate verificado) - Veredictos: DEMO-UNAUTH / DEMO-BAC / REFUTADO con evidencia por endpoint; presupuesto MAX_CALLS=60 (solo lectura A->B, cero payloads de exploit) - CLI: python3 core/ab_diff.py [--own email:pass]

NUEVO core/lab_ab_site.py — laboratorio no-WP local (filosofia WP-LAB): app "Laravel-like" con registro+verificacion por codigo, login, /api/user/ SIN owner check (IDOR a detectar) y /api/user//notes CON owner check (caso a refutar). Validacion end-to-end 100% local, cero terceros.

BACKEND: POST /api/ab_diff {url, own_email?, own_pass?} -> job consultable en GET /api/ab_diff/ (log en vivo).
VERSION 0.68.0.

Bugs reales encontrados durante el desarrollo (case_real RC-000154): (1) _parse_forms solo devolvía formularios con password -> el form de verificacion (solo codigo) jamas se veia: todos los registros morian en timeout 180s; (2) login() hacia POST a la accion relativa -> MissingSchema. Ambos fixeados y regresados.

v0.67.0 — AUTHZ-PROOF capa 4: BAC-PROOF dinamico (RC-000153)

La joya de la corona del BAC 2.0: validacion DINAMICA con ejecucion real. core/bac_proof.py levanta WordPress completo en local (SQLite via sqlite-database-integration, php -S, wp-cli, SIN MySQL), instala el plugin bajo prueba, siembra un post victima del admin con canario en TODAS las claves de meta que el plugin lee y dispara cada accion candidata como:

```
ANONIMO -> SUSCRIPTOR -> ADMIN
```

Difiere las respuestas y emite evidencia de ejecucion: - DEMO-UNAUTH-DINAMICO: el anonimo obtiene el canario (nopriv). - DEMO-BAC-DINAMICO: el suscriptor obtiene lo mismo que el admin sobre el objeto ajeno. - REFUTADO-DINAMICO: el admin obtiene y el suscriptor es bloqueado (gate/owner funciona en ejecucion) -> alimenta FP-MEMORIA con huella dinamica (capa 5).

Todo local: cero interaccion con terceros (regla mVDP).

Bug real del desarrollo (case_real): core install con el drop-in sqlite agrega sufijo /wp al siteurl; las cookies de sesion viajan con path /wp y no llegan a /wp-admin (todo el A/B veia usuarios como anonimos). Fix: Lab.ensure() fuerza siteurl/home raiz.

Validado en vivo (sandbox): idor sin gate -> DEMO-BAC-DINAMICO (sub 200 con canario = admin); handler con owner check -> REFUTADO-DINAMICO (sub 'no es tuyo', admin canario) y la FP-MEMORIA aprendio la refutacion. Corpus 24/24 PASS.

Requisitos: php + pdo_sqlite (Termux: pkg install php php-sqlite3).

v0.66.0 — AUTHZ-PROOF capa 5: FP-MEMORIA semantica (RC-000152)

"El falso positivo se paga una sola vez, nunca mas." fp_autoclose cubria reglas FIJAS; la capa 5 agrega aprendizaje: cuando un hallazgo es refutado (por el OPERADOR en el triaje, o por la DEFENSA con prueba en DIFF-HUNT), su HUELLA ESTRUCTURAL queda guardada y cualquier hallazgo identico se auto-cierra.

Huella semantica (core/fp_memory.py): type, veredicto de gates, nopriv, rol exigido, caps, nonce, sensible, owner_check, sinks de objeto, marcadores del nombre de accion (notice/review/media...) y del codigo alrededor (prepare/cast/escape/menu_page/settings). SIN nombres de archivo ni plugin: dos hallazgos con la misma huella son la misma familia aunque vivan en plugins distintos.

- Matching EXACTO anti-ruido: una huella solo cierra identicos.
- Dedupe: la misma huella no se aprende dos veces.
- Memoria en .codexrc/intelligence/fp_memory.jsonl dentro del repo: se propaga por git, cada instancia que hace pull hereda lo que las demas ya pagaron en triaje.
- diff_hunt cierra por memoria ANTES de correr la cadena de evidencia (ahorra abogados) y auto-aprende de los DESCARTADO probados por la DEFENSA.
- CLI: --stats / --list.

Validacion: repro aprender->cerrar con plugin B estructuralmente identico (distinto nombre/clase/archivo) cerrado FP-MEMORIA, hallazgo distinto NO cerrado, dedupe OK (corpus RC-000152, regresion 23/23 PASS, memoria aislada en el test).

v0.65.0 — AUTHZ-PROOF capa 2: OBJECT-OWNER, IDOR estatico (RC-000151)

El hueco admitido del BAC: GATES-AUDIT no miraba de QUIEN es el objeto. Un handler que hacia get_post_meta(absint(\$_POST['post_id'])) sin comparar post_author/current_user_id pasaba inadvertido: IDOR horizontal invisible para el triaje aunque el rol exigido fuera bajo y el nonce solo probara identidad.

Ahora cada handler anota: - object_access: sinks WP donde un id TAINTED (\$var = ...\$GET/\$_POST/\$_REQUEST, renames hasta 3 hops) llega (get_post, get_post_meta, get_userdata, wp_update_post, wp_delete*...). - owner_check: si el cuerpo compara dueño (post_author cmp, get_current_user_id cmp, current_user_can('edit_post', ...)).

Veredicto nuevo CANDIDATO-IDOR: id controlado por el usuario toca objetos, sin owner check, con rol exigido menor a editor (editor/ administrator legitiman acceso ajeno; nopriv ya es CANDIDATO-BAC). diff_hunt prioriza el nuevo estado.

Validacion: repro sintetico 4/4 (corpus RC-000151, regresion 22/22 PASS) + eRoom 1.7.1 estable (0-day CANDIDATO-BAC preservado, 12 PROTEGIDO intactos) + rc149 6/6 + rc150 4/4 sin cambios.

v0.64.0 — AUTHZ-PROOF capa 1: ROLE-SOLVER (RC-000150)

GATES-AUDIT trataba cualquier `current_user_can` como "protegido" sin importar QUE privilegio exige ni QUE hace el handler: un `update_option` accesible con `current_user_can('read')` (cualquier suscriptor logueado) pasaba como PROTEGIDO. El nonce prueba IDENTIDAD, no AUTORIZACION.

Ahora cada handler anota: - `caps_req`: capabilities/roles que exige de verdad (extraccion de `current_user_can/user_can/author_can/wc_current_user_has_role/ is_super_admin`). - `rol_minimo`: rol estandar WP mas bajo que pasa el chequeo (tabla `cap->rol` jerarquica `administrator>editor>author>contributor>subscriber`). - `sensible`: si el cuerpo hace acciones sensibles (`update_option`, `wpdb->write`, borrado de `users/posts/terms`, `filesystem`, `exec...`).

Veredicto nuevo PRIVILEGIO-DEBIL: `caps bajas (subscriber/ contributor/author) + accion sensible = candidato de escalada`, aunque tenga nonce. Handlers solo-nonce quedan anotados (`solo_nonce`) para el triaje. `diff_hunt` prioriza el nuevo estado.

Validacion: repro sintetico 4/4 (corpus RC-000150, regresion 21/21 PASS) + eRoom 1.7.1 estable (12 PROTEGIDO con caps de admin intactos, 0-day CANDIDATO-BAC preservado) + rc149 6/6 sin cambios.

v0.63.0 — AUTHZ-PROOF capa 3: resolucion de hooks dinamicos (RC-000149)

GATES-AUDIT dejaba CALLBACK-NO-RESUELTO (o directamente no veia) los handlers registrados con nombre de hook o callback dinamico — justo el patron que usan los plugins ofuscados o con loaders complejos. Ahora se resuelven 6 clases:

- `add_action($hook, ...)` con `$hook` por constante, concatenacion (`'wp_ajax_' . $this->prefix . '_save'`) o interpolacion (`'wp_ajax_nopriv_x'`): propagacion de constantes lite por archivo (`vars + $this->props`, ultima asignacion previa al uso).
- `add_action('hook', $cb)` con callback en variable.
- `array($this, $m_var)` con metodo en variable.
- Closures en linea: el cuerpo se analiza directamente (PROTEGIDO si tiene nonce/caps dentro del closure).
- `_action_args` corta comas SOLO a profundidad 0 (`array($this, 'm')` ya no se rompe).
- [HALLAZGO] BUG HEREDADO corregido en `FUNC_RE`: el prefijo consumia el salto de linea previo (off-by-one) y firmas de funciones SIN docblock precedidas por `'}` devolvian cuerpo VACIO -> caps/nonce falsos -> veredictos sin haber leído el handler. Latente desde v0.43.0; invisible en plugins con docblocks (estandar WP) pero activo en codigo sucio (mid-band). Fix: `In = src[:m.end()]`.

Validacion: repro sintetico 6/6 (corpus RC-000149, regresion 20/20 PASS) + eRoom 1.7.1 real: 17 hooks, 0-day `nopriv_stm_zoom_meeting_sign` sigue CANDIDATO-BAC, 12 PROTEGIDO intactos.

v0.62.8 — FP-FILTER de reflejo en pagina de bloqueo WAF

Leccion `greenlightdispensary (03/10)`: el Arsenal reporto 16 "XSS REFLEJADO MEDIA" que eran puro eco de la URL en el HTML de bloqueo de Cloudflare (estados 403/429). El WAF eco la URL completa (con el payload) en su pagina de "Access denied", y el motor contaba eso como reflexion del sitio.

- Arsenal (`frontend/arsenal.html`): `confirno()` descarta el reflejo si el estado es 403/429/503; log explicito "reflejo descartado: pagina de bloqueo WAF".
- `core/hunter_xss.py` (test_param, camino clasico): reflejo en 403/429/503 = DESCARTADO con log.
- `core/async_lane.py` (SLIPSTREAM): mismo filtro; ademas `_probe` devuelve el OBJETO respuesta, asi el `WafGuard.observe` vuelve a ver status/headers (antes recibia solo .text y no detectaba NINGUN bloqueo en el carril async).

v0.62.2 — capa UI/UX del pipeline y la navegacion

Presentacion unicamente; sin cambios en UNIVERSAL-ENGINE, TAIN-TRACE, EVIDENCE-CHAIN ni ningun modulo de caza.

- Backend: helper `_frontend_page` con Cache-Control no-store para las TRES paginas (`/`, `/hunter.html`, `/arsenal.html`): Android ya no conserva un HTML viejo.

- Nav: activa por location.pathname en los tres HTML (initMobileNavigation), nunca mas hardcodeda; al navegar se cierra el menu movil (nav-open).
- Pipeline movil: el flex-wrap muere. Grid de 2 columnas minmax(0,1fr); los .conn no participan del layout; los nodos llenan su celda -> CVE/REPORT quedan alineados.
- Luces de esquina: running = ambar con pulso alterno (corner-pulse-a/b sobre ::before/::after); success = verde fija. Solo el nodo RUNNING se anima; los success quedan en luz estable (baratissimo en un telefono).

• **matrix oculto en movil (<=820px): cero trabajo constante**

de canvas en telefonos viejos.

- prefers-reduced-motion: animaciones y transiciones a 0.001ms.

v0.62.1 — HUNTER anonimo con potencia completa

- MODO SIN SESION: nueva bateria _descubrir_acceso_anon. Sin credenciales no hay contraste A->B con sesion, pero una ruta tipicamente PRIVADA (/dashboard, /admin, /wallet...) que responde 200 a un visitante anonimo ES un BAC directo: se reporta como hallazgo 'alta' y alimenta la arana como semilla.
- Fingerprint anti-shell corregido: los SPA reales sirven HTML byte-identico para toda ruta; la ventana fija de 120 bytes tragaba paginas privadas cortas (bug hallado con lab local, RC-000145). Ahora: comparacion byte-identica, tolerancia <10.
- Validado: lab con /dashboard + /admin anon -> 2 hallazgos y 2 seeds; httpbin y vellius (SPA real) -> 0 falsos. Regresion 18/18.

v0.62.0 — DIFF-HUNT v2: la fuente cambio, el motor se adapta

- wp.org sello el historial de plugins (oct 2026): la API ya no sirve el mapa de versiones, los zips viejos dan 404 y los SVN tags estan vacios. El diff entre dos zips murio EN LA FUENTE (RC-000144).
- get_versions tolerante a dict y list.
- prepare() fallback FULL-CODE: sin historial -> descarga la version ACTUAL y audita todo el codigo vivo (dir_old=None).
- _senales_changelog(): el readme publico sigue diciendo QUE parchearon (XSS, SQLi, hardening...) y DONDE; cada corrida adjunta las senales de seguridad de las ultimas 3 versiones para guiar la caza del parche incompleto.
- Validado en vivo: cola de 29 blancos 29/29 OK en 24s, 37 hallazgos crudos, 2 SQLi de swift-performance-lite pasaron a EVIDENCE-CHAIN -> PROBABLE 3/4 (uno con proof COMPLETE pero alcance admin sin resolver, otro INCOMPLETE honesto).

v0.61.0 — Corpus fresco: VDP-FRESH + prioridad mid-band

PALANCA 1 (corpus fresco automatico): - core/vdp_fresh.py: snapshots del mapa VDP (GOLDEN LIST) y diff semanal -> vdp_nuevos.json. Detecta ALTAS (VDP recién agregado: blanco sin competencia, se caza ya), BOUNTY-NUEVOS (sube prioridad) y BAJAS (dejar de cazar). - hunt_wide.py --vdp-nuevos: las altas+ounty entran a la cola sin filtro de 90 dias ni corpus (incluso slugs fuera de >=5k) y se ordenan al tope con etiqueta NUEVO-VDP.

PALANCA 2 (mid-band 5k-50k): - Orden de cola: (1) VDP-FRESH, (2) pagables, (3) banda 5k-50k (medianos: menos blindaje y competencia que los top), (4) resto por installs. Configurable --band, visible en --dry-run. - --dry-run: imprime la cola ordenada sin cazar.

Razon: DIFF-HUNT mostro que el codigo nuevo de los TOP sale endurecido; los medianos y los VDP recién agregados son donde la ventana de ser primeros sigue abierta. Regresion 16/16.

v0.60.0 — SSA consume la evidencia: FISCAL exige prueba def-use

- ssa.py: def_use_proof() — prueba estructurada {proof, value, lineage, source, unknown_ancestors, transformations}. SSA no dictamina vulnerabilidad: entrega genealogia del valor. Estados: DEF_USE_COMPLETE / INCOMPLETE /

NO_SOURCE / UNRESOLVED.

- evidence.py: build_chain adjunta chain['def_use']. FISCAL: "taint alcanza el sink" ya no es probado=True incondicional; exige DEF_USE_COMPLETE (o fuente directa en el sink). DEFENSA: DEF_USE_NO_SOURCE = refutacion fuerte demostrada. JUEZ: DEF_USE_INCOMPLETE (ancestros desconocidos) nunca sostiene DEMOSTRADO-ESTATICO (cap a PROBABLE, como INTEGRITY).
- RC-000142 (hallado por muestreo antes de concluir seguridad): la refutacion NO_SOURCE era inservida con asignaciones condicionales (la limpia dentro de un if no mata la version con \$_REQUEST en el merge). Ahora exige dominancia CFG sobre el sink. Y el pipeline crudo filtraba mal las vars sin version.
- Medicion (5 plugins): DESCARTADO 34->37, PROBABLE 115->92, CONTESTADO 0->20; 60/149 proofs COMPLETE. Pipeline 231 taint: 4 refutados con dominancia, 225 con cadena. Regresion 15/15.

v0.59.0 — Fase 1.5: SSA-lite en Taint-Trace

- core/ssa.py: grafo de versiones def-use intra-funcion. Cada asignacion crea una version (kind: SOURCE/SANITIZED/PREPARED/ CONCAT/COPY/CONST/UNKNOWN-FN/UNKNOWN-VAR) con padres y linea.
- El sink ya no reporta "\$q aparece cerca": reporta la CADENA (\$sql <- COPY <- \$term <- SOURCE @L11), evidencia directa para la capa FISCAL/JUEZ.
- Refutacion conservadora (SSA-REFUTADO): solo cuando el BFS sobre TODOS los padres prueba que ninguna version alcanzable trae fuente, taint ni valor desconocido. Llamadas no resueltas y vars sin version BLOQUEAN la refutacion (jsst SSRF vuelve a vivo).
- Bugs de solidez encontrados y corregidos por muestreo real: := no matcheaba la regex de asignacion; vars interpoladas sin version contaban como limpias; refutacion caminaba solo el primer padre del DAG.
- Integrado en plugin_batch.py y diff_hunt.py (refutados filtrados, cadenas en el output).
- Medicion N1-VDP: 231 taint -> 10 refutados, 219 con cadena, 2 sin vars resueltas. Regresion 14/14.

v0.58.3 — Triage de los 29 cerrado: 0 candidatos, 2 clases semanticas nuevas

- RC-000139 (cmp_router): comparacion dominada POR un gate = corre solo tras autenticar = logica interna, no control de acceso. js-support-ticket 28 -> 0 (canadddfile x28: form_request router dentro de rama nonce-verificada, copiado en 28 controllers).
- RC-000140: comparacion sin sinks dependientes + operando no credencial = router de display. Sinks escapados (esc_html/esc_attr/ wp_kses/sanitize_) no cuentan como efecto sensible. revisionary admin notices 1 -> 0 (el "contexto AUTH" venia del texto HTML del aviso, no del codigo).
- Credenciales (password/token/session/...) quedan SIEMPRE fuera de estas reglas: 6875 sigue vivo.
- Triage congelado en hechos/triage_loosecmp_v0583.json: 29 -> 0 candidatos reales. La cola vuelve a cero y SSA-lite arranca con el corpus limpio.
- Regresion 12/12 PASS.

v0.58.2 — SEMANTIC CORE: RC-000132 resuelto en el patron BAC

- cfg.py: all_protected(gate_rx, sens_rx) — True/False/None: ¿todos los sinks estan dominados por un gate?
- pattern_match.finalize: el handler nopriv resuelto ya NO se descarta por "gate visible": el gate limpia solo si DOMINA todas las escrituras. Si una rama publica alcanza la escritura sin pasar por el gate, el hallazgo se conserva con anotacion RC-000132 (candidato real). Fallback a la regla anterior si el CFG no parsea.
- La pregunta abierta de la auditoria ("rama publica por diseno vs alcanzable por salto de flujo") queda respondida: dominancia.
- Mediciones: pods FP muerto con prueba de dominancia; N1-VDP 0 BAC recuperados y 0 FP resucitados; regresion 11/11.
- Hallazgo colateral: 28 loose-cmp vivos NO TRIADOS en js-support- ticket (vivos ya en v0.58.1; nuevos para la cola de triaje).

v0.58.1 — SEMANTIC CORE Fase 1: CFG + dominancia en produccion

- core/cfg.py (Python puro, ~650 lineas): tokenizer PHP-lite (strings/comentarios/heredoc), parser de statements (if/elseif/else encadenados, if sin llaves, alt-syntax : ... endif, loops, switch, try, HTML embebido ?> ... <?php), CFG

intra-funcion y dominadores iterativos.

- Consultas: `gate_dominates_sink` (el gate protege el sink?), `cmp_router` (la comparacion solo rotea hacia region protegida?), `router_noise`.
- Integrado en `loose-auth-cmp` (RC-000137): la AUTORIDAD es el CFG; las ventanas de texto quedan como fallback si el parseo falla. Credenciales en operando exigen capability gate (nonce no salva un `==` sobre password).
- Medicion: SNAP 8/8 loose-cmp muertos por dominancia (incluidos los 2 residuos a 2-3 lineas que la ventana +-1 no veia). Gate a 30+ lineas detectado en lab. N1-VDP: 48 -> 1 vivo (familia distinta, ya cerrada a mano en RC-000134). Lab 6875 SIGUE VIVO.
- Caches por src (funciones y CFG): timeout 300s -> 1.2s en SNAP.
- Regresion: 10/10 PASS.

v0.58.0 — SEMANTIC CORE (nivel 0): identidad canonica + integridad

Inicio del Semantic Core (recomendacion experta: el motor pasa de decidir por proximidad textual a decidir por propiedad del programa).

- `core/semantic_core.py`: FileId = root canonico + ruta relativa normalizada. El basename NO es identidad:
- match por ruta exacta -> INTEGRITY OK (hash sha256-16 de testigo)
- match solo por basename con UNA coincidencia -> AMBIGUO
- basename con varias coincidencias (`Wpil/Error.php` vs `Wpil/Table/Error.php`) -> INTEGRITY FAILURE: NO se elige ninguno (antes el bug tomaba el ultimo del `os.walk`)
- EVIDENCE-CHAIN: `build_chain` resuelve via `semantic_core` y expone `chain.file {file_id, content_hash, integrity}`; el JUEZ bloquea DEMOSTRADO-ESTATICO si `integrity != OK` (invariante RC-000127: evidencia de archivo equivocado nunca alcanza veredicto alto).
- Cache de indice por root (200 resoluciones: 0.10s).
- Regresion: 10/10 PASS con el nuevo modulo integrado.
- Lab: caso `Wpil` duplicado resuelto por FAILURE (no elige ninguno), hash equivocado -> INTEGRITY FAILURE, veredicto alto bloqueado.

Proxima fase: CFG intra-funcion + dominancia (`gate_dominates_sink`) para RC-000132 y cierre de raiz de RC-000136.

v0.57.10 — RC-000136: gate adyacente mata el loose-cmp

- `_GATE_ADJ` generalizado (sin hardcodear funciones de plugin): `current_user_can | user_can( | is_user_logged_in | _can_access | wp_die | die( | exit` en la misma linea o la anterior a la comparacion senalada => esa llamada es el control de acceso real; el loose-cmp es router/filtro detras del gate.
- Se evalua ANTES que los tokens AUTH de RC-000135 (los 6 de SNAP volvian por el token 'auth' del operando sin pasar por el gate).
- Medicion: los 6 vivos de RC-000135 (todos `$_GET['auth']==$ntInfo['lcode']` en SNAP) MUEREN. Lab 6875 real (`==` sobre password, sin gate adyacente) SIGUE VIVO.
- Familia loose-cmp N1 n VDP: 29 -> 0 vivos. Cerrada.

v0.57.9 — RC-000135: loose-cmp exige contexto de AUTORIZACION

- El patron loose-auth-cmp ya no dispara con cualquier `==` sobre input: exige tokens AUTH en el snippet de la comparacion senalada (+80 chars) o contexto +-5 lineas.
- tokens fuertes (`password/passwd/user_pass/nonce/capability/current_user_can/login/role/privilege/cookie/manage_options`): valen en snippet O contexto.
- tokens debiles (`auth/token/secret`): solo en el snippet, con lookbehind anti-'author' (queries de posts usan author).
- comentarios despojados: 'nonce was verified' en un comentario no autoriza nada.
- snippet tomado de la LINEA SENALADA, no del primer `==` del cuerpo (FP jsst-hooks: `checkbox == 1` dentro de funcion de registro).
- Lab: caso real 6875 (`==` sobre password) SIGUE VIVO; filtros de `term_id/checkbox/code!=` MUEREN.
- Medicion sobre el N1 n VDP: 23/29 loose-cmp mueren solos. 6 vivos, todos la misma clase (`$_GET['auth']==$ntInfo['lcode']` en SNAP: router de flujo OAuth, admin-gateado por `nxs_snap_user_can_access` en la misma linea).

v0.57.8 — TRIAJE de los 2 sobrevivientes no-loose-cmp: ambos MUERTOS

- simple-floating-menu (BAC 1177/1214): MUERTO. Cero hooks nopriv/REST en el plugin; los handlers imex cuelgan de admin_init con nonce dedicado + manage_options. Era RC-000133 puro.
- wc-multivendor-membership (pago degradable 57): MUERTO. is_valid_member_id exige member_id == usuario actual; paymode contra allow-list doble estricta (enabled n offline); online solo via IPN (vendedor ya parcheo CVE-2026-12967). RC-000134 al corpus.
- FIX del analisis del re-run: el match por ubicacion confundia TIPOS de finding distintos en la misma linea. Sobrevivientes reales del N1 n VDP: los 15 loose-cmp, nada mas.

v0.57.7 — PATRON BAC MIRA EL CUERPO DEL HANDLER (RC-000131/133)

- finalize() postergado: los hallazgos bac-ajax-nopriv esperan a tener el mapa de TODAS las funciones escaneadas, resuelven el callback del hook ('fn', 'fn', array(\$this['Cls'],'method')) y escanean el cuerpo del HANDLER en busca de gates (nonce/caps/is_user_logged_in). Handler gateado = FP descartado (pods admin_ajax: 20 sinks -> 0).
- Se exige el hook nopriv EN el cuerpo (RC-000133): el patron no dispara solo por escribe+input+bonus nonce publico (FP pods admin_save, gate en el caller l18n.php:123).
- Lab: vulnerable con nonce publico DISPARA, control gateado MUERE, pods y ad-insertar 0 BAC.
- RE-RUN N1 n VDP (12 plugins, \$48 hallazgos): 31 mueren solos, 17 sobreviven (15 loose-cmp + 1 BAC simple-floating-menu + 1 verificacion de pago wcfmvm).

v0.57.6 — TRIAJE BAC N1: pods y ad-insertar, ambos MUERTOS

TRIAJE MANUAL (protocolo BAC: handler -> gate -> impacto): - pods 3.3.9.2 (PodsAdmin.php:62, \$2.600 VDP): MUERTO. nopriv_pods_admin es intencional (forms front-end publicos). Gate interno completo: whitelist de metodos, nonce pods-method, pods_is_admin para priv, y process_form con nonce NUEVO en 3.3.9.2 vinculado a (pod, id, fields, uri, uid) que solo se acuna al renderizar un form ya autorizado. No retargeteable. - ad-insertar 2.8.19 (\$2.600 VDP): MUERTO como BAC. nopriv_ai_ajax solo expone features publicas (iframe de bloques de ads, ads.txt publico); ramas sensibles gated por REMOTE_DEBUGGING default OFF; el write real (adsense-client-id) esta en ai_ajax_backend con nonce + manage_options. Nota informativa: remote-ads-txt expone ABSPATH/paths a nopriv (divulgacion de rutas, severidad baja, tipicamente no pagable).

REGRESS: RC-000131, RC-000132 (clase: nopriv intencional con gate interno, disparada por el HOOK sin mirar el cuerpo del handler).

v0.57.5 — TRIAJE SQLI N1: 11/11 FALSOS POSITIVOS, 3 CLASES DE FP ELIMINADAS

TRIAJE MANUAL (backlog wide, SQLi en plugins con VDP pagable): - unlimited-elements (3/3), js-support-ticket (7/7), revisionary (1/1): TODOS MUERTOS. Ningun SQLi explotable. Ningun reporte.

MOTOR (core/taint_trace.py) — 3 clases de FP erradicadas: - WP-WRAPPER GUARD: ->query()/get_results() sobre WP_Query/WP_Term_Query/ WP_User_Query/WP_Comment_Query/WP_Site_Query/WP_Network_Query (new o global \$wp_query) ya no es sink SQLi: core sanitiza internamente. - PREPARE MULTILINEA: \$q = \$wpdb->prepare(\$q, \$args) limpia el taint y marca la var PREPARADA (sobrevive a acumulaciones .= posteriores). Formato con \$wpdb->prefix tolerado; placeholder visible en cualquier literal del format, no solo el primero. - SCOPE POR FUNCION: cada function con nombre resetea el taint (semantica PHP real); antes una var local no-SQL en otra function contaminaba todo el archivo. Las closures sin nombre NO resetean (patron add_action).

VALIDACION: unlimited-elements 0 SQLi (antes 3 criticas), revisionary 0 (antes 1), js-support-ticket 0 (antes 20), lab 4/4 (vulnerable sigue disparando, saneado/wrapper no, \$wpdb crudo no se pierde). NOTA FN aceptada: formato de prepare con input crudo DENTRO del string + args aparte (raro) no lo ve taint nivel 1; lo cubre SINK-SCAN/Semgrep. REGRESS: RC-000128, RC-000129, RC-000130 con repros minimos.

v0.57.4 — AUDITORIA DE SEGURIDAD DEL PROPIO PROYECTO

AUDITADO (sin cambios necesarios): - Arsenal: fire/resume/report/extreme exigian token de sesion (30 min) con 401/403 y log de auditoria con IP de origen. OK. - log_event: descarta claves password/token/cookies. safe_url: redacta query params. safe_auth_info: Authorization [redacted], cookies solo NOMBRES. - Job dicts: guardan solo opts booleanos, nunca credenciales. - login_debug: pasos con nombres/conteos/codigos, jamas valores de credenciales. - field_adapt: registra el NOMBRE del campo requerido, no su valor.

CORREGIDO: - Server escuchaba en 0.0.0.0: todo el WiFi veia jobs, informes y el Arsenal. Ahora 127.0.0.1 por defecto; LAN solo con CODEXRC_LAN=1 (con aviso impreso). - /api/auth_token devolvia el header Authorization (token de sesion vivo) sin autenticacion. Ahora rechaza peticiones no-loopback (403) con log del intento; override explicito CODEXRC_LAN_TOKENS=1 para quien lo necesite. - Aviso al arrancar si ARSENAL_PASSWORD queda en la default 'extremo'.

v0.57.3 — HIGIENE DE DEPENDENCIAS (Termux-first)

- requirements.txt reescrito: 100% instalable en Termux armv7l. Fuera rich (declarado pero jamas importado), adentro websocket-client (COV-BAIT) y fpdf2 (export PDF) que se usaban sin estar declarados.
- NUEVO requirements-pc.txt: curl_cffi sale del requirements base porque NO compila en Termux y rompia pip install -r requirements.txt completo. En PC: pip install -r requirements-pc.txt.
- Chromium: _find_browser() ahora tambien mira \$PREFIX/bin/* de Termux (repo termux-x11) antes de darse por vencido.
- /api/status devuelve deps: salud de cada dependencia opcional con su fallback documentado (httpx->hilos, websocket->WS off, fpdf->TXT/JSON, chromium->fpdf2). Verificar una instalacion de Termux = 1 request.
- Validado: pip install -r requirements.txt resuelve limpio (7/7 OK).

v0.50.0 — CODEX-OBSERVE + CODEX-REGRESS, alcance reducido (2026-10-02)

El usuario comparti una segunda especificacion tecnica: CODEX-OBSERVE (observabilidad) + CODEX-INTEL (diagnostico de anomalias) + CODEX-REGRESS (regresion), 8 fases completas (event collector, anomaly engine, differential/metamorphic/property-based/fuzz testing, regression corpus, quality gates, IA diagnostica).

Decision de alcance: implementar SOLO lo que protege plata real hoy. Para una herramienta de caza de bugs pagables, el motor de anomalias/fuzzing/metamorphic testing es inversion de ingenieria sin payoff claro (no encuentra bugs mas rapido, no cobra bounties). Lo que SI tiene valor inmediato es no repetir un bug ya corregido.

Implementado: - core/observe.py (CODEX-OBSERVE, Fase 1 reducida): un evento VERDICT_CREATED por cada cadena de evidencia resuelta (file, line, verdict, evidence_hash, module_version, git_commit, run_id). JSONL en .codexrc/intelligence/events/<fecha>.jsonl (gitignored, son logs operativos, no conocimiento versionado). Best-effort: si observe falla, la caza sigue (nunca bloquea). - core/regress.py (CODEX-REGRESS, Regression Corpus solamente): casos de defectos reales confirmados en .codexrc/intelligence/regressions/corpus.jsonl (SI versionado en git, es conocimiento permanente). Semilla: RC-000127 = el bug de resolucion de rutas de v0.49.0 (archivos homonimos resolvian al archivo equivocado), fixed_in v0.49.1. python3 core/regress.py reproduce el caso con un root sintetico (Error.php real + Table/Error.php homonimo) y falla si el motor vuelve a resolver mal. - Hook en evidence.annotate: cada veredicto generado dispara log_verdict automaticamente.

Fuera de alcance (documentado, no implementado): Anomaly Engine (contradicciones, verdict instability), Differential testing entre versiones, Metamorphic testing, Property-based/fuzz testing interno, CODEX-INTEL (correlacion/diagnostico asistido por IA), Quality Gates de release. Se revisara si en el futuro aparecen inconsistencias reales de verdict entre corridas que justifiquen construir el Anomaly Engine.

v0.49.1 — EVIDENCE-CHAIN hardening: bug de resolucion de rutas (2026-10-02)

El usuario compartio una especificacion tecnica de Evidence Chain (principios de evidencia, veredictos no monotonicos, reachability honesta). Al contrastarla contra la implementacion v0.47.0, surgio un bug real durante la revision manual de un hallazgo del dia (link-whisper).

- FIX CRITICO** core/evidence.py build_chain: la resolucion de ruta por fallback (cuando root + basename no existe) recorria os.walk SIN romper el loop externo tras encontrar un candidato por nombre. Si dos archivos comparten

basename (ej. `core/wpil/Error.php` y `core/wpil/Table/Error.php`), el motor terminaba analizando el ULTIMO archivo visitado, no el correcto. Fix: match por ruta relativa EXACTA primero; fallback a basename solo si no hay match exacto, tomando el PRIMERO (no el ultimo).

- **Saneamiento indirecto:** `_sanitization_evidence` ahora detecta `$v[] = (int)... / absint(...) / intval(...)` antes de que `$v` llegue al sink (ej. `implode(',', $v)` dentro de un `$wpdb->query`). Antes solo veía el casteo si envolvía la variable DIRECTAMENTE en la línea del sink.
- **Trazabilidad (`verdict_history` + `evidence_hash`):** cada cadena guarda un hash determinista de su evidencia normalizada y un historial mínimo del veredicto emitido, siguiendo la recomendación de la especificación de no sobrescribir silenciosamente decisiones.
- **Caso real que lo disparó:** link-whisper (`core/wpil/Error.php:674`) salía DEMOSTRADO-ESTATICO por el bug de ruta (analizaba `Table/Error.php` en vez de `Error.php`) Y porque no veía el casteo indirecto. Tras el fix: 0 hallazgos vivos (correcto, el dato real se castea a entero antes del `implode`).
- **Regresión:** 4/4 plugins previamente validados limpios (akismet, google-site-kit, wpforms-lite, woocommerce) siguen en 0 hallazgos tras el fix.
- **Nota:** este bug pudo afectar cualquier chain previa donde el archivo del finding tuviera un homónimo en otra carpeta del mismo plugin; alcance exacto no cuantificado (no se re-corrió todo el historico).

v0.48.0 — WIDE-HUNT: el corpus se multiplica (2026-10-02)

Con los abogados cerrando los falsos positivos solos (v0.47.0), limitar la caza a los VDP de Patchstack dejó de tener sentido técnico: el VDP no decide a quien cazamos, solo a quien reportamos.

- `core/wide_corpus.py` — corpus completo de wordpress.org vía API pública (browse=popular, umbral configurable, por defecto $\geq 5k$ installs): **3.260 blancos** (2.186 con $\geq 10k$, 466 con $\geq 100k$) en ~16 segundos, refrescable con `--refresh`.
- `vdp_mapa.json` — mapa de pagabilidad refrescado de `vdp.patchstack.com/api/database/vdp` (52 páginas): **1.298 VDP únicos, 710 con bounty individual** (de \$100 a \$14.400).
- `core/hunt_wide.py` — runner: cola = corpus n updates frescos (`--dias 90`) menos los ya auditados (`hechos/hunt_wide_done.txt`), DIFF-HUNT en paralelo (`--workers`), y cada hallazgo sale marcado con `vdp` y `paga`. Modo `--solo-pagables` para priorizar. Con `--limit N` para tandas cortas.
- **Fix v0.47.1:** la cadena de evidencia ya no embebe el finding vivo (referencia circular al exportar JSON); guarda copia plana.
- **Validación:** smoke 4/4 (akismet, google-site-kit, wpforms-lite, woocommerce limpios); cola real detectada: 2.108 plugins con updates ≤ 90 días, 417 pagables.

v0.47.0 — EVIDENCE-CHAIN + ABOGADOS: razonamiento de evidencia (2026-10-02)

Cambio de filosofía inspirado en RacerD/Infer/Pysa: no preguntar "¿podría ser vulnerable?" sino "¿qué evidencia tengo para afirmar que lo es?". Nuevo `core/evidence.py`:

- **Cadena de evidencia por finding:** SOURCE (¿input controlable por el atacante? con clasificación: superglobal/cookie/header-semi), FLOW (vars tainteadas y saltos), AUTH (compuertas: nonce/caps/nopriv + entrada resuelta por BFS de callers hasta 3 niveles: hook -> callers -> función del sink), SANITIZATION (sanitizadores en la ruta, incluida la línea del sink), SINK (tipo/línea/severidad), CORRELATION (qué analizadores lo vieron), DYNAMIC (slot para evidencia dinámica futura).
- **Abogados deterministas (el LLM nunca es juez):** FISCAL debe probar los 4 requisitos del ataque (source controlado, taint al sink, sin sanitización efectiva, alcance sin privilegios); DEFENSA busca refutaciones fuertes (gate protegido, sanitizador en ruta, prepare con placeholders, contexto solo-admin, alcance no resuelto) y débiles (media); JUEZ pesa con reglas fijas.
- **Veredictos en escala RacerD (se reporta solo lo demostrable):** CONFIRMED (estática completa + dinámica reproducida), DEMOSTRADO-ESTATICO (prueba estática completa), PROBABLE (flujo probado, alcance sin resolver), CONTESTADO (refutación parcial), DESCARTADO (refutación fuerte). Los DESCARTADO se filtran de los hallazgos vivos de DIFF-HUNT y se contabilizan en `descartados_defensa`.
- **GATES-AUDIT v0.43.1:** parsea hooks registrados vía loader propio (`$this->loader->add_action('wp_ajax_nopriv_x', $obj, 'metodo')`) y deduplica contra HOOK_RE clásico.
- **TAINT-TRACE:** nuevo sink `fsockopen` (red) en la familia SSRF.

- **Validación:** (1) lab 3/3 — handler nopriv sin sanitizar -> DEMOSTRADO-ESTATICO (fiscal 4/4), handler con nonce+caps -> DESCARTADO por gate, sanitizado con absint -> ni genera finding; (2) sobre RegistrationMagic real: los 4 candidatos del admin caen DESCARTADO (nonce+manage_options), el XSS de paypal.php cae DESCARTADO (esc_url en el sink), y el 0-day real del IPN downgrade (test_ipn) queda DEMOSTRADO-ESTATICO con alcance SIN AUTENTICACION y salto resuelto `validate_ipn -> callback -> paypal_ipn (wp_ajax_nopriv)`.
- **Integración:** DIFF-HUNT adjunta `_chain` y `_verdict` a cada hallazgo vivo, ordena por veredicto y expone `descartados_defensa`. CLI: `python3 core/evidence.py <plugin_root>` (cadenas de todos los findings taint, ordenadas por veredicto).

v0.46.0 — VDP-1300: directorio completo de Patchstack (2026-10-02)

- Extracción del directorio publico de VDP activos de `patchstack.com/database/vdp` (1.300 productos con bounty) via navegador real (GHOSTGATE): el endpoint `/api/database/vdp` solo responde con sesion de la app y la WP API (`wp.patchstack.com`) rechaza IPs no autorizadas.
- Cruce automatico con `api.wordpress.org`: slug, version, installs, last_updated.
- Lista maestra: `cz_hunt/vdp/vdp_full.json` (producto, installs, bounty USD, vendor, axp).
- Motivacion: el GOLDEN LIST tenia 76 plugins y el lote 90-dias completo (69 plugins) dio 0 bugs pagables; el directorio real es 17x mas grande y esta lleno de medianos (1k-100k) con bounty (\$250-\$2,600) poco cazados.

v0.45.1 — DIFF-HUNT paralelo (2026-10-02)

- `core/diff_hunt.py`: `ThreadPoolExecutor` (--workers, default 4); cada slug se diffea y escanea en paralelo, reporte de progreso en vivo con `flush`.
- Uso: `python3 core/diff_hunt.py --workers 6 --targets lista.txt`

Changelog de codexRC

Historial completo de versiones, de la más reciente a la más antigua. Los parches menores (x.y.z) también viven aquí; el README solo lista los hitos.

v0.45.0 — DIFF-HUNT

Pivote de estrategia: en vez de auditar plugins enteros (los top estan blindados), caza SOLO el codigo nuevo.

`core/diff_hunt.py`: descarga version actual + anterior de cada slug (API wp.org), calcula diff de lineas agregadas/modificadas en .php propios (sin vendor/assets), y corre Taint-TRACE + CVE-MATCH + GATES-AUDIT + FP-AUTO-CLOSE restringido a esas lineas nuevas. Logica: lo recién escrito no paso por ningun auditor ni por los bots de los demas hunters. Flujo de operacion: filtrar GOLDEN LIST por `last_updated <= 21 dias` e `installs >= 10k` (API en paralelo), cazar los diffs de todos. Validado en vivo: 37 plugins VDP frescos diffeados; cartflows resulto ser un parche de seguridad (FP), captcha-code-authentication heredo una comparacion floja solo en registro (categoria no pagada).

v0.44.0 — FP-AUTO-CLOSE

Nuevo modulo `core/fp_autoclose.py`: segunda capa de verificacion tras GATES-AUDIT. Reconoce los patrones de falso positivo que se repiten lote tras lote y los dictamina solo, para que la consola muestre solo hallazgos vivos: -

FP-GATE-PROTEGIDO: el hallazgo cae en un handler con caps y/o nonce. - FP-SQLI-PREPARE / FP-SQLI-CAST: prepare con placeholders o absint/intval en el flujo. - FP-XSS-ESCAPED: la linea del sink aplica `esc_wp_kses/sanitize`. - FP-UPLOAD-WHITELIST: la subida valida mime y/o extension (magic bytes incluidos). - FP-STRICT-IN_ARRAY: `in_array` con `strict=true` (sin type juggling). - FP-GATE-NOPRIV-LOGIN: handler nopriv cuyo callback exige sesion (dispatcher global). PLUGIN-BATCH: contador `fp-auto=N` por plugin, `criticos` cuenta solo hallazgos vivos, los autocerrados se muestran solo con `--verbose`, y las acciones publicas por diseno (notices, reviews, dismiss, formularios de visitantes) ya no cuentan como [HALLAZGO]. Validado contra los lotes reales del dia: 34 hallazgos autocerrados en 3 plugins que antes se descartaban a mano, cero vivos perdidos.

v0.43.0 — GATES-AUDIT — GATES-AUDIT

Nuevo modulo `core/gates_audit.py`: dictamina automaticamente si los handlers de un plugin estan protegidos. Mapea hooks `wp_ajax` / `wp_ajax_nopriv` / `wc_ajax` y rutas REST (`permission_callback __return_true`) a su callback, y busca compuertas (`current_user_can`, `wp_verify_nonce`, `check_ajax_referer`). Veredictos: CANDIDATO-BAC (anonimo sin caps ni nonce), REVISAR-AUTH (logueado sin caps ni nonce), PROTEGIDO, REST-ABIERTO. Integrado en PLUGIN-BATCH: cada hallazgo llega anotado con el veredicto del handler donde cae, y los candidatos van arriba. Parche en TAIN-TRACE: `implode(array_fill(...%d...))` (placeholders internos) ya no se taintea (FP de prepare).

v0.42.0 — PLUGIN-BATCH (modo agente)

Un solo comando caza plugins WordPress por slugs: descarga la última versión estable, corre TAIN-TRACE + CVE-MATCH, filtra ruido de librerías de terceros (vendor, plugin-fw, assets, codemirror...) y deja solo hallazgos en código propio, marcados con [HALLAZGO]. El flag `--vdp` anota los plugins con programa VDP activo en Patchstack y `--out` guarda el JSON completo. ~1-13s por plugin. Uso:

```
python3 core/plugin_batch.py slug1 slug2 --vdp vdp_matches.json --out resultados.json
```

v0.41.0 — VENDOR-FARM

Descubrimiento de familias de vendors WordPress con instalaciones en rango pagable (10k-200k) vía la API pública de wordpress.org, exclusión de slugs ya auditados y estimación de paga Patchstack integrada. CLI + POST `/api/vendor_farm`.

v0.40.0 — VERIFICACIÓN DEL SISTEMA

El sistema verifica los hallazgos automáticamente (contraste anónimo A->B, doble petición, CACHE-BAIT contra falsos positivos de cache como `no-store + BYPASS`) y los informes TXT/JSON/PDF exponen solo veredictos automáticos del sistema, sin guías de verificación manual para el operador.

Lo nuevo de cada entrega, de la más reciente a la más antigua:

- **v0.39.2 — SCHEME-PROBE:** la verificación de sesión ya no depende de leer el JS del sitio (que Cloudflare puede bloquear). Si el token heredado no pasa con el scheme descubierto/default, se prueban EN VIVO `Session`, `Bearer`, `Token`, `JWT`, token plano y cookies (`session_key`, `sessionid`, `session`, `token`); el primero que responde 200 queda instalado en la sesión para todo el escaneo y se registra en el debug. Validado en lab sin JS (peor caso): login -> 401 con Bearer -> probe detecta `Session` -> `api_authenticated` con perfil.
- **v0.39.1 — TOKEN-INHERIT:** SPAs sin cookies (bitevolut, apps React con localStorage) ya no pierden la sesión tras loguear. El Hunter (1) descubre el scheme de autorización leyendo el JS (`Authorization: Session ${a}`, `Bearer`, `Token`, `JWT`), (2) extrae el token del JSON de respuesta del login (`session_key`, `access_token`, `sessionid`...), y (3) lo inyecta en el header `Authorization` para toda la sesión. Antes: login exitoso + verificación 401 (`api_unauthorized_or_error`). Validado en lab: login -> token heredado -> `/api/auth/me/` 200 con perfil del usuario.
- **v0.39.0 — SPA-DISCOVERY v2:** el descubridor de login en apps JavaScript ya no queda ciego con apps modernas. (1) Extrae endpoints estilo `fetch` (strings sueltos `/api/...`) además de `axios .post()`; (2) rastrea chunks Vite/esbuild con hash (`assets/Login-BBdKUnr1.js`) además de webpack; (3) si Cloudflare bloquea la descarga de JS (403), reintenta con la sesión del escaneo (cloudscraper/GHOSTGATE-LITE hereda el pase); (4) escalera de rutas de login estándar (`/api/auth/login/`, `/api/login/`, `/api/v1/auth/login/`, etc.) cuando el auto-descubrimiento no encuentra nada, en vez de rendirse posteando a la página HTML (el 405 clásico); (5) descarta un endpoint al primer 404/405 sin quemar los 3 payloads de campo. Validado en lab con estructura Vite+fetch real: descubre login, cuenta y wallet, filtra impersonate/captcha.
- **v0.38.4 — diagnóstico transparente de auth:** cuando el auto-descubrimiento falla por bloqueos de WAF (Cloudflare 403) o el servidor rechaza el POST (405), la UI ahora muestra la razón real (`rechazo_servidor_HTTP_405`, `api_rechazo_credenciales`) en vez del confuso `no_auth_method`.
- **v0.38.3 — guardia blindado en Termux (auto_update.sh):** instancia única por lock con PID (dos watchers nunca pelean), resurrección del servidor ante muerte (OOM/Android/crash) con volcado de las últimas 15 líneas de

- `server.log` como diagnóstico, anti-cuelgue (proceso vivo pero `/health` mudo 3 ciclos -> reinicio), anti-bucle (5 caídas seguidas -> backoff progresivo 30/60/90/120s en vez de martillar el teléfono), liberación del puerto 8000 por PID con fallbacks `fuser -> ss -> lsof` (nunca `kill -f`), rotación de `auto_update.log` (>512KB conserva 200 líneas), wake-lock automático con aviso si falta Termux:API. Validado en vivo: server muerto a propósito -> detectado, diagnosticado y revivido solo; segundo watcher rechazado por el lock. Nota honesta: si Android mata Termux COMPLETO, ningún script interno revive; el ajuste "Batería -> Sin restricciones" sigue siendo obligatorio.
- **v0.38.2 — severidad honesta en cadenas XSS:** una cadena XSS+CSP débil o XSS+panel admin solo es CRÍTICA si alguna pieza XSS ejecutó en navegador real (verificado por VERITAS/kit PoC). Sin confirmación queda en ALTA "teórica" con la razón visible. Lección del lab real (velliuss, 02/10): el sink `location.hash -> innerHTML` nunca ejecutó pese al CSP ausente — el Hunter ya no grita CRÍTICA por piezas no confirmadas.
 - **v0.38.1 — parche de honestidad en ESCALADA:** si la petición con sesión no devuelve `Set-Cookie` (sesión ya activa, lo normal tras el login), el playbook ya NO afirma "cookies con HttpOnly": indica revisar la cookie a mano en DevTools (columna HttpOnly) para decidir el techo real (hijack vs. lectura en sesión).
 - **v0.38.0 — ESCALADA: qué hacer DESPUÉS del aviso crítico.** El CHAIN decía "tenés una cadena crítica" y ahí se quedaba. Ahora, ante cada cadena crítica/alta, el Hunter sigue SOLO hasta el techo con 4 pasos deterministas: (A) **VERITAS dirigido** — re-verifica en navegador real SOLO las piezas XSS de la cadena; para piezas DOM (`location.hash -> innerHTML`) arma el canario correcto (`<img onerror>`, porque por innerHTML los `<script>` no ejecutan) y lo mete en el fragmento de la URL; (B) **techo de impacto** — pide el blanco con la sesión heredada y revisa las flags de cookies: una cookie de sesión SIN `HttpOnly` + XSS ejecutable = camino a HIJACK de sesión (el techo), y sube el veredicto; (C) **kit PoC local** — genera `poc/poc_<host>.html`: página atacante local con iframe del blanco + link con hash canario + envío de canario inerte por `postMessage` en varias formas; el operador la abre EN SU MÁQUINA con su sesión y VE si el banner CANARY aterriza dentro del sitio (canario inerte: nada sale del equipo, cero exfiltración); (D) **PLAYBOOK** — pasos ordenados de lo que falta según las piezas (persistencia -> XSS almacenado, canal sin click vía `postMessage`, destino del informe). Todo con aislamiento de fallos: un paso que falla no tumba la caza. Nodo `escalada` en el pipeline tras `chain`, sección ESCALADA en el informe TXT/PDF con el playbook, y `escalaciones` en el resumen. Validado contra el lab real (velliuss.com): VERITAS cargó la shell Angular sin bloqueo de CF; sin sesión la lógica del account no corre (por eso el kit PoC es la vía), cookies y veredicto reportados honestos, PoC generado y verificado.
 - **v0.37.1 — DECOMPILE en producción (parche de topes).** Probado sobre un APK real de 12MB (F-Droid, 3 DEX, ~30,000 métodos): los topes de muestreo (20,000 strings/métodos, pensados para la araña) dejaban los nombres como "?" en apps grandes. El loader de DECOMPILE ahora levanta hasta 150,000 strings/métodos y 30,000 tipos; el APK completo desensambla en ~3 segundos con todos los nombres resueltos. Nota de caza: apps compiladas con R8 renombran la mayoría de los métodos a a/b/c — `--all` es el modo útil en blancos ofuscados; el filtro sensible brilla en apps sin ofuscar.
 - **v0.37.0 — DECOMPILE: DESCOMPILACIÓN PARCIAL (bytecode -> pseudocódigo).** Nivel 3 de ingeniería inversa: abre el `code_item` de cada método del DEX y desensambla sus instrucciones dalvik REALES resolviendo lo que referencian — `v0 = "AKIA..."` (strings), `invoke-static Lic;->verify(v0)` (métodos por clase y nombre), saltos y retornos. El resultado se lee como fuente: el flujo completo de `decryptCard` sin ejecutar nada y sin JRE/dex2jar (que no existen para armv7l). Por defecto descompila solo los MÉTODOS DE LÓGICA SENSIBLE (`decrypt/license/sign/verify/auth/checkout`); `--all` baja todo. Un secret que aparece como `const-string` dentro del flujo se reporta como crítico "SECRET EN EJECUCIÓN" (está en el camino del código, no en una string suelta). Subset honesto de ~45 opcodes (constantes, strings, fields, invokes, saltos, retornos); un opcode fuera del subset se marca y el método se corta ahí. Contenedores recursivos (APK con varios .dex). CLI: `python3 core/decompile.py <apk|dex> [--all]`. API: `POST /api/decompile`. Validado: DEX sintético con código real descompilado completo (`const-string + invoke-static + return-void`), secret AKIA detectado en el flujo, dentro de APK.
 - **v0.36.0 — REVERSE: INGENIERÍA INVERSA (nivel 2).** Va más allá de la superficie de BIN-AUDIT y lee la ESTRUCTURA interna del artefacto. **DEX (APK):** parse real del formato (`string_ids/type_ids/method_ids/class_defs` con uleb128) — lista clases, métodos y todas las strings del bytecode; sobre ellas corre el detector de secrets y endpoints (una API key dentro del bytecode DEX es un secret que nadie mira). Detecta app **OFUSCADA** (nombres a/b/c estilo ProGuard/R8) y **métodos de lógica sensible** (`decrypt/license/sign/verify/auth/checkout`). **CLASS (JAR):** parse del constant pool CAFEBABE — clase, superclase y constantes Utf8 con secrets. **ELF:** tabla de símbolos **interna** (.symtab, no solo la pública .dynsym) — funciones estáticas que revelan la organización (`verify_`, `decrypt_`), detección de binario **STRIPPED**, secciones `.debug_*` sin limpiar y **entropía por sección** (>7.2 bits/byte = empaquetado/ofuscado UPX-like). Contenedores recursivos: zip/apk/jar anidados se abren y se auditan por dentro. Nivel honesto: parse de formatos + símbolos + entropía (el desensamblado de instrucciones queda como etapa futura). CLI: `python3 core/re_engine.py <path>`. API: `POST /api/re`. Validado: DEX sintético válido parseado completo (4 clases, 2 métodos, 12 strings, secret AKIA y endpoint staging desde el bytecode), `.class` con `sk_live` y connection string del constant pool, objeto compilado con `decrypt_card`, `verify_license` en su .symtab, y detección

de STRIPPED en binarios reales del sistema.

- **v0.35.0 — BIN-AUDIT: ANÁLISIS BINARIO/NATIVO.** Va más allá del navegador y del código PHP: abre cualquier artefacto distribuido por un vendor y extrae lo que paga. **Secrets hardcoded** (crítica; Patchstack paga por "BAC sobre objetos sensibles: API keys, secrets"): claves AWS `AKIA...`, Google `AIza...`, Stripe live `sk_live_...`, GitHub PAT, Slack, JWT embebidos, claves privadas PEM, connection strings con credenciales (`mysql://user:pass@...`), pares `api_key/secret/password = valor`. **Endpoints internos** en el binario (`admin/staging/debug = alta`). **Imports peligrosos** de la tabla dinámica real de ELF (`system/execve/popen/dlopen/fork`, parse propio de `.dynsym`, 32/64 bits, LE/BE). **Fingerprint:** ELF/PE/Mach-O/zip/jar/apk/gzip, arquitectura y bitness; contenedores zip/apk/jar se abren RECURSIVO (audita cada `.so / .dex / config anidado`). Nivel 1 honesto: recon estático (strings, símbolos, secretos, endpoints); desensamblado completo fuera de alcance Python puro. CLI: `python3 core/bin_audit.py <path>`. API: `POST /api/bin`. Validado: SDK sintético con 5 secretos dentro de un zip anidado (5/5), `/bin/sh` real con `execve`, `fork` en su `dynsym` (correcto), `.so` limpio = 0 hallazgos.
- **v0.34.0 — CVE-MATCH: APRENDIZAJE DE PATRONES GLOBALES.** Identifica fallos conocidos SIN firmas fijas: cada patrón es una combinación de señales estructurales débiles (`hook wp_ajax_nopriv +3`, escritura `+3`, input de usuario `+2`, protección presente `-4`, nonce impreso público `+2` que cancela el veto) que juntas recrean el fallo histórico; al disparar cita la familia: "coincide con CVE-2023-6875 / 0-day AFFI (CZ-HUNT 2026)". 6 familias reales: BAC ajax sin auth (AFFI), type juggling en control de acceso (CVE-2023-6875, con lookbehind para no confundir `===`), IPN downgrade (CVE-2026-9242), SSRF de request, upload sin validación, rol desde request (PrivEsc). **Línea base global:** `--baseline` mide sobre todo el corpus descargado (73,401 archivos PHP) qué % de funciones parecidas SÍ se protege; cada hallazgo anota el outlier ("68% de 1,408 handlers parecidos protegen esto; este NO"). Validación honesta: replicó los 4 patrones del lab, 0 falsos positivos en controles protegidos, y **redescubrió solo el 0-day real de RegistrationMagic** (`validate_ipn` en `paypal.php:193`) solo por el patrón. CLI: `python3 core/pattern_match.py <path> [--top N] [--json]`. API: `POST /api/patterns`.
- **v0.33.0 — TAIN-TRACE: ANÁLISIS DE FLUJO DE DATOS.** Sigue el valor de un parámetro desde la FUENTE (`$_GET/$_POST/$_REQUEST/$_COOKIE/php://input/$_SERVER`) a través de asignaciones y concatenaciones hasta el SINK peligroso: base de datos (`$wpdb->query/get_var/get_row/get_results`, `mysql_query`), navegador (`echo/print/printf = XSS`), `unserialize` (object injection), `include/require` y funciones de archivos (LFI), `system/exec/eval` (RCE) y `wp_remote_*/curl` (SSRF). Encuentra la vulnerabilidad aunque la respuesta HTTP no muestre nada: **SQLi ciego**, **XSS en otra página**, **object injection diferido**. Sanitizadores que cortan el flujo: `intval/absint/sanitize_*/esc_sql/esc_attr/esc_html/esc_url/wp_kses`; `$wpdb->prepare` con placeholders y el dato SOLO en los valores = seguro (detecta además la inyección de cadena de formato). Motor por punto fijo intra-archivo, Python puro, cero dependencias (Termux OK). CLI: `python3 core/taint_trace.py <archivo_o_carpeta> [--json] [--top N]`. API: `POST /api/taint {"path": "...", "top": 20}`. Validado: 3/3 flujos en lab vulnerable (SQLi ciego, XSS, `unserialize`) y 0 falsos positivos en el control sanitizado; sobre plugins reales endurecidos con `prepare` también da 0 (correcto).
- **v0.32.0 — COV-BAIT (etapa 2): MOTOR DE MUTACIÓN CON FEEDBACK.** Fuzzing guiado por cobertura, estilo XBOW, 100% determinista. (1) **Pistas del código:** antes de mutar, el motor descarga los scripts de la app (los mismos URLs que la cobertura registró) y extrae los nombres de parámetros que el código lee (`URLSearchParams.get(...)`, `getParameter`) — el propio JS dice dónde mirar. (2) **Expansión estructurada (fase A):** BFS sobre el corpus de inputs: cada parámetro pista se sondea con valores canónicos sobre los padres más profundos; si una sonda EJECUTA rangos nuevos, el input entra al corpus y se profundiza desde ahí (hill-climbing). (3) **Mutación aleatoria (fase B):** con el presupuesto restante, añade/muta/quita parámetros con anti-bucles (hash de inputs probados) y freno por estancamiento (12 sondas sin ganancia). Presupuesto fijo de 36 sondas por caza. Reporta las COMBINACIONES que desbloquean funciones escondidas con los nombres de las funciones ejecutadas. Validado en lab: la etapa 1 encontró `secretInit` con `adminmode=1`; la etapa 2, guiada por las pistas `['level', 'adminmode']`, desbloqueó `secretDeep` con la combinación `adminmode=1&level=x` en la sonda 2. Chrome en modo bajo consumo (banderas de ahorro de RAM para entornos límite).
- **v0.31.0 — COV-BAIT (etapa 1): caza guiada por COBERTURA DE CÓDIGO.** El salto de calidad del arsenal: se enchufa al navegador real vía CDP (protocolo de depuración, cliente websocket propio en Python puro) y mide **QUÉ funciones de JavaScript ejecuta cada sonda** con `Profiler.startPreciseCoverage`. Flujo: (1) baseline — cobertura "pública" de la app al navegar la semilla; (2) sondas de parámetros ocultos (`adminmode`, `debug`, `role...`) — si un parámetro ejecuta rangos de JS que la app NUNCA ejecutó antes, hay un **camino escondido** y se reporta con los nombres de las funciones; (3) páginas descubiertas — mapea qué páginas traen lógica propia nueva. Un parámetro oculto que EJECUTA código es señal mucho más fuerte que una reacción de tamaño/status. Toggle `opt_cov` (apagado por defecto, pesado: levanta Chrome). Sin navegador: se salta con aviso y la caza sigue. Etapa 2: mutación de inputs para maximizar cobertura nueva (fuzzing con feedback) — implementada en v0.32.0.
- **v0.30.0 — BATERÍAS DEDICADAS:** las técnicas de superficie oculta se separan en baterías propias con toggle individual y cobertura expandida: **PATH-BAIT** (LFI/traversal con wrappers `php://filter` para `wp-config` y `.env`, firmas

passwd/win.ini/id_rsa, sondas genéricas en parámetros de alto valor; LFI paga en la tabla), **SSTI-BAIT** (fingerprint del motor de plantillas: Jinja2 confirmado con {{7'7}}->7777777, Twig, FreeMarker, Velocity, ERB, Smarty; SSTI = RCE en potencia), **CACHE-BAIT** (8 cabeceras unkeyed probadas en múltiples páginas + web cache deception: rutas privadas con sufijo .css que sirven los mismos datos privados con content-type cacheable; 100% pasivo, no envenena nada), **GRAPHQL*** (descubrimiento por POST y GET, introspección con mapa de tipos = mapa de IDORs, field suggestions cuando la introspección está cerrada, detección de batching con lote inerte de 2). **STEALTH-BAIT** queda con parámetros ocultos + CRLF + method tampering. Validado 9/10 en laboratorio (el 10° es el camino alternativo de GraphQL con introspección cerrada).

- **v0.29.0 — STEALTH-BAIT (caza de superficie oculta):** 7 técnicas nuevas para blancos blindados, todas en modo lectura A->B con canarios inertes: (1) **parámetros ocultos** — sondea secretos que el frontend nunca muestra (debug, is_admin, role, internal...) y detecta si el backend reacciona; (2) **PATH-BAIT / LFI** — path traversal en parámetros file/path/tpl/lang con firma de detección (passwd, wp-config), LFI paga en la tabla de recompensas; (3) **SSTI-BAIT** — canario de plantillas {{77}}: si la respuesta contiene 49 el servidor COMPUTA (RCE en potencia); (4) **CRLF-BAIT** — inyección de cabeceras verificada solo en headers de respuesta; (5) **method tampering** — X-HTTP-Method-Override y verbos alternativos; (6) **CACHE-BAIT** — detección PASIVA de entradas unkeyed (X-Forwarded-Host reflejado), candidatas a cache poisoning sin envenenar nada; (7) **GraphQL*** — introspección abierta (mapa completo de la API = mapa de IDORs). Toggle **STEALTH-BAIT** activado por defecto. Validado 7/7 en laboratorio.
- **v0.28.0 — ENCADENAR HALLAZGOS + AUTOCORRECCIÓN:** el valor real está en las **cadenas**, no en piezas sueltas. Nuevo nodo **chain** tras VERITAS: conecta hallazgos ya verificados y calcula el impacto combinado (XSS + CSP débil = ejecutable y sesiones robables; XSS o SQLi + panel admin mapeado = captura de sesión admin / toma de control; IDOR en varias identidades = filtración masiva; datos privados + endpoint de settings/token = camino a account takeover). Las cadenas van arriba en los informes TXT/PDF/JSON con su severidad combinada, listas para justificar la recompensa. Nuevo nodo **self_tune** + memoria persistente **tune_state.json** (no se committea): el Hunter registra qué parámetros produjeron hallazgos (por nombre y por tipo de blanco) y en la caza siguiente sube su presupuesto de sondas (x1.2 con 1 hallazgo, x1.5 con 2+), sin bajar el de los nuevos. La caza N aprende de la N-1, 100% determinista.
- **v0.27.0 — CEREBRO (inteligencia determinista):** nueva capa de análisis previo al gasto de sondas. **Fingerprint del blanco:** detecta WordPress (+ plugins y theme vía **wp-content**), frameworks (Angular, React, Next.js, Vue, Laravel, jQuery) y servidor por headers; si identifica un plugin de vendor con historial de CVEs (VillaTheme, WPSwings, YITH, Elementor...), lo marca como pista de vendor-farming. **Priorización de la caza:** puntúa cada objetivo (endpoint + parámetro) por palabras clave de valor (**admin-ajax**, **upload**, **user_id**, **settings**...) contra un diccionario de pesos, descarta la basura de tracking (**utm_***, **fbclid**, **assets**) y ordena la cola de caza por valor. Cada objetivo recibe un **presupuesto de sondas proporcional** a su puntaje (SQLi-BAIT ya lo respeta). 100% determinista: mismo blanco, mismas decisiones; costo cero y sin dependencias.
- **v0.26.0 — SQLi-BAIT (inyección SQL avanzada):** nueva batería con 4 tiers de detección sobre los parámetros mapeados y cabeceras clásicas (**X-Forwarded-For**, **User-Agent**, **Referer**): **error-based** (rompe la sintaxis y captura el error del motor en la respuesta, con fingerprint de MySQL/MariaDB, PostgreSQL, MSSQL, SQLite y Oracle), **boolean-based** (diferencial clásico **AND 1=1** / **AND 1=2** contra el baseline con medición de similitud), **time-based blind** (**SLEEP(4)** / **pg_sleep(4)** / **WAITFOR DELAY**, confirmación repetida 2 veces para descartar jitter de red) y **stacked queries**. Controles anti-falso-positivo: el error debe ser nuevo respecto al baseline y el control parametrizado no dispara. Presupuesto de sondas por parámetro para no martillar el blanco. Solo para blancos autorizados.
- **v0.25.2 — CAZA EN LOTE + reportes exportables:** **/api/hunt_batch** recibe una cola de blancos y los caza en serie con la misma configuración: un blanco que falla no tumba el lote, progreso en vivo por blanco (**/api/batch/<id>**) y resumen agregado al final. Cada informe se descarga en TXT (informe plano con filtraciones arriba), JSON (dump completo con leaks separados) y PDF estilizado: documento profesional impreso con el navegador real en headless (tarjetas de resumen, sección roja de filtraciones primero, hallazgos técnicos por severidad con sellos de verificado/falso positivo). Sin navegador, fallback a **fpdf2** (**pip install fpdf2**, python puro, funciona en Termux).
- **v0.25.1 — Robustez por aislamiento:** cada batería corre encapsulada: si un módulo explota en plena caza, el nodo se marca en error, el fallo se reporta en el log vivo con su causa y el pipeline CONTINUA con las demás baterías. Probado con fallo inyectado a propósito: caza terminada en finished con hallazgos y VERITAS activo. VERITAS además ahora detecta el navegador en cualquier plataforma: Linux (Chrome/Chromium), Windows (chrome.exe/msedge.exe, incluidas rutas de Program Files y LOCALAPPDATA) y macOS (Google Chrome.app, Chromium.app, Edge.app). Sin navegador, la caza sigue y los hallazgos quedan como reflexiones sin verificar.
- **v0.25.0 — VERITAS:** verificación de XSS con navegador real (Chrome headless): canario inerte document.title, veredicto por ejecución, falsos positivos descartados automáticamente, blindaje para hallazgos sin datos de caracteres crudos (nunca descarta a ciegas).

- **v0.24.0 — GHOST-SHIELD:** evasión WAF con memoria: jitter por sonda, detección de bloqueo (firmas de challenge y rate-limit), cooldown exponencial por origen persistido en disco, origen quemado tras 4 bloqueos (sondas saltadas al instante, purga de 1h) y rehabilitación automática si el origen vuelve a responder sano. Toggle en la UI; el carril async comparte la misma memoria.
- **v0.23.0 — SLIPSTREAM:** carril async para la batería GET (I/O asíncrono, hasta 64 sondas en vuelo por event loop, sesión heredada intacta). Benchmark: 219 sondas/s en el lab, 35x frente al secuencial, mismos hallazgos. Fallback automático a hilos si falta httpx.
- **v0.22.0 — OVERDRIVE + PIPE-HUNT:** sondas en paralelo (hasta 8 workers) en las baterías GET/forms y en los módulos de sonda de XSS-PRO (dangling, base tag, redirect); 5.4x más rápido con 8 workers. Modo tubería `hunt_pipe.py`: URLs desde stdin (`waybackurls sitio | python3 hunt_pipe.py --xsspro --workers 8`), vigilancia de jobs y resumen final.
- **v0.21.1 — parche de estabilidad:** índice de jobs con lock y poda, watchdog para cuelgues, recuperación de jobs al reiniciar (estado "interrupted"), sesión heredada por origen, errorhandler global JSON, `auto_update.sh` con verificación de salud.
- **v0.21.0 — XSS-PRO 2:** arsenal completo de 20 vectores XSS: DOM clobbering, prototype pollution, iframe srcdoc, base tag injection, open redirect y `javascript:` en Location, path reflection, fingerprint de sanitizadores, self-XSS escalable, sink de document.cookie.
- **v0.20.0 — XSS-PRO + GHOSTHOOK:** seis vectores avanzados (postMessage, carga dinámica de script, mXSS, dangling markup, stored, CSP bypass) y colector propio de blind XSS en Cloudflare Workers (KV, panel privado, beacon a prueba de CSP).
- **v0.19.0 — escalada de chunks:** descarga recursiva de chunks lazy de Angular para extraer la API real del sitio (`apiurl`), IDOR con soporte POST, batería BLIND opt-in, listeners de postMessage, evaluación de CSP.
- **Estabilidad:** el índice de jobs está protegido con lock (dos cazas simultáneas nunca pisan el estado de la otra), con tope de 40 jobs en memoria y log vivo limitado. Un job que supera 45 minutos es marcado como colgado por el watchdog (la UI nunca queda en "running" eterno). Al reiniciar el servidor, los jobs se recuperan del disco y una caza interrumpida aparece como "interrupted" con su explicación. La sesión heredada se guarda por origen: cazas contra sitios distintos nunca se cruzan credenciales. Cualquier error no manejado responde JSON sin tumbar el proceso, y `auto_update.sh` verifica `/health` tras cada reinicio y resucita al servidor si cayó.
- **Diagnóstico:** cada respuesta incluye `connection` y `diagnostics` (backend respondió, objetivo alcanzable, sesión verificada, usuario detectado).

v0.51.0 — RETRO-HUNT + ABILITY-SCAN (2026-10-02)

- **RETRO-HUNT (core/retro_hunt.py):** caza PRE-COOLDOWN. Cola = plugins >=5k installs cuya ultima actualizacion es anterior al gate de revision IA de WP.org (jun 2026); ese codigo distribuido nunca paso el escaneo automatico. Auditoria FULL-CODE (todo el plugin, no solo diff), orden: pagables por monto -> installs. Incremental crash-safe (hechos/retro_done.txt + retro_results.json.jsonl).
- **ABILITY-SCAN** integrado en GATES-AUDIT: detecta `wp_register_ability()` (Abilities API, WP 6.9+); `permission_callback __return_true` o ausente = ABILITY-ABIERTA. Superficie 2026 sin escanear.
- Fix: import de la cadena de evidencia en `retro_hunt` (`gates_audit -> audit`), los veredictos FISCAL/DEFENSA/JUEZ ahora anotan.
- Fix: filtro `_fp` en `retro_hunt` para que "vivo" = realmente vivo.

v0.52.0 — UNIVERSAL-ENGINE (2026-10-02)

- **UNIVERSAL-ENGINE (core/universal_engine.py):** orquestador. Descubre que es el blanco (fingerprint fuente o URL viva), modela superficie, elige analizadores segun perfil (no corre 500 pruebas siempre) y ejecuta kernel: TAIN-TRACE + CVE-MATCH + GATES-AUDIT + ABILITY-SCAN + FP-AUTO-CLOSE + EVIDENCE-CHAIN como componentes.
- **Ledger de cobertura** (clase Ledger): todo item de superficie con estado ANALIZADO / DESCUBIERTO / NO ACCESIBLE / VERIFICADO. Nada se reporta sin estado; los estados solo escalan hacia evidencia mas fuerte.
- **UNIVERSAL-RECON:** perfil de fuente (PHP/JS/Python/Java/Go/Ruby, WP plugin/tema, Laravel, Django, Flask, Express, React, Android) y de URL viva (framework, Cloudflare, rutas tipicas).

- **WP-LAB:** laboratorio WordPress local (PHP 8.2 + SQLite, sin MySQL) para AUTH-DIFF dinamico en sitio propio. Declarado como fase en el ledger; ejecucion a pedido (costo alto).
- **BROWSER-INTEL / UNIVERSAL-API / STATE-MACHINE / POLYGLOT-TRACE(js,python):** fases declaradas en el perfil; el kernel las activa por tipo de blanco.

v0.53.0 — UI-RENAISSANCE (2026-10-02)

- **Dashboard renovado con la identidad del logo:** paleta verde oliva militar extraida del banner (verde #9fce54 / logo puro #71983f / paneles #0d140a), marca "CodexRC" en el header (codex en verde, RC en blanco) con el banner como logo. Hunter con la misma identidad.
- **Responsive real iOS/Android/PC:** sidebar deslizable en movil (menu hamburguesa), touch targets de 44px, font-size 16px en inputs (evita el zoom de iOS), safe-area insets para notch, meta theme-color + apple-mobile-web-app.
- **Nuevo panel MOTOR DE CAZA en el dashboard:** progreso vivo de WIDE-HUNT y RETRO-HUNT (barras + ultimas lineas, refresco cada 30s) y runner de UNIVERSAL-ENGINE (input path o URL, muestra perfil, analizadores elegidos, ledger de cobertura ANALIZADO/DESCUBIERTO/NO ACCESIBLE/VERIFICADO y hallazgos con veredicto).
- **Backend:** GET /api/hunt_status (progreso de cazas desde hechos/) y POST /api/universal (corre el motor sobre path o URL). VERSION 0.53.0.

v0.53.1 — fix UI movil real (2026-10-02)

- **Logo roto (404):** /assets/banner.jpg no cargaba porque Flask solo servia estaticos desde frontend/, no desde la raiz del repo donde vive assets/. Nueva ruta GET /assets/<path> en el backend sirve el logo correctamente (verificado con Chrome headless + emulacion movil: naturalWidth paso de 0 a 1280).
- **Hamburguesa "se enzima" con el pipeline:** el boton de menu usaba position:fixed (flotaba libre sobre TODO el documento, sin relacion con el header). Paso a position:absolute anclado DENTRO del header (que ahora es position:sticky), asi nunca puede superponerse con el contenido de abajo.
- **"El menu desplegable se queda abajo":** el nav (escaner/hunter) se envolvía a una segunda linea dentro del header angosto. Se movio al drawer lateral (sidebar-nav, visible solo en movil, arriba del formulario de escaneo) y se oculta del header en pantallas chicas: ya no hay wrap raro, el header queda fijo en una sola fila.
- Logo + marca CodexRC agregados tambien a hunter.html (identidad consistente en ambas paginas), con header sticky en movil.
- Limpieza de CSS muerta (.prompt ya no existia tras el rebrand, quedaban reglas huérfanas).
- Validado con Chrome headless real (CDP, emulacion de dispositivo 390px y 1366px) en las 4 combinaciones pagina x ancho: logo carga, hamburguesa solo aparece en movil, nav sin overlap, pipeline nunca se tapa.

v0.53.2 — fix overflow real del panel motor de caza (2026-10-02)

- Bug confirmado con datos reales (reportado por el usuario con captura: linea de log "[1578/1615] gelato-integration-for-woocomerce" cortada en el borde de pantalla): .hunt-card no tenia min-width:0 ni overflow:hidden, asi que el texto white-space:nowrap de .hunt-log (lineas de progreso WIDE-HUNT/RETRO-HUNT) forzaba a la tarjeta a estirarse mas alla del viewport (clasico "minimo automatico" de CSS Grid con contenido nowrap). Medido con Chrome headless: antes del fix la tarjeta llegaba a 476px en una pantalla de 390px (86px afuera); despues del fix, 366px (adentro, 0 elementos desbordados).

v0.53.3 — fix denominador RETRO-HUNT (2026-10-02)

- Usuario pregunto si "2087 y 1010" eran reales (captura con RETRO-HUNT mostrando "1010/1007 auditados", matematicamente imposible). Verificado contra disco: ambos contadores SON reales (hechos/hunt_wide_done.txt = 2087 lineas, hechos/retro_done.txt = 1010 lineas unicas sin duplicados, timestamps y logs coinciden con corridas reales en tmux). El bug era el DENOMINADOR: "totalCola" de retro estaba hardcodedo en 1007 (una foto del tamaño de la cola al momento de lanzar un run), mientras "hechos" es un contador acumulado de TODAS las corridas de retro a lo largo del tiempo. Fix: totalCola ahora se calcula en vivo desde wide_corpus.json filtrando por el mismo criterio de cooldown (last_updated < 2026-06-01) que usa core/retro_hunt.py para armar su cola real. Resultado tras el fix: 1010/1010 (RETRO-HUNT efectivamente completo al 100% del universo pre-cooldown real).

v0.62.3 — drawer móvil navegable (RC-000146/147)

Presentacion unicamente; sin cambios en el motor de caza.

- RC-000146 (BUG REAL, reproducido con headless): al tocar "hunter"/"arsenal" en el menu movil NO navegaba y el menu se cerraba solo. Causa: el backdrop (body::before, z-index 30 en el contexto RAIZ) tapaba el drawer entero, porque crea su propio contexto de apilamiento (position:relative + z-index:2): el z-index:40 del .sidebar solo compete DENTRO de ; desde afuera todo vale "2". $30 > 2 \rightarrow$ backdrop sobre el drawer: el tap caia en el backdrop y solo cerraba el menu. Fix: el pseudo-elemento pasa a main::before (mismo contexto que el drawer) + body.nav-open main { z-index: 50 } para que el drawer tambien quede sobre el header sticky (z-index 5), que tapaba los primeros ~56px del menu. Al abrir: drawer 40 > backdrop 30 > header 5. Al cerrar: todo vuelve a su lugar. Cerrar = tocar la zona oscura (comportamiento estandar de drawer).
- RC-000147: el drawer (position:fixed) conservaba su scroll interno entre aperturas; si el usuario habia bajado hasta Autenticacion, al reabrir aparecia YA scrolleado con la nav fuera de pantalla ("tenia que empujar un chiquito hacia arriba"). Al abrir siempre arranca desde arriba.
- Header: fuera el "codexRC" de texto duplicado en las 3 paginas (quedaba logo + texto = 2 CodexRC visibles). Ahora solo logo, "// HUNTER"/"// ARSENAL" y la version.
- Escáner: fuera la card "Vulnerabilidades" (lista de CVEs conocidos por tecnologia, sin relación con hallazgos; eso vive en Hunter). Card + renderCve + integración eliminados.
- VERSION del backend actualizada a 0.62.3 (quedaba clavada en 0.57.4, el header mostraba version vieja).

Validado en vivo con Chromium headless viewport 390x844: navegacion real a hunter/arsenal OK, backdrop cierra, scrollTop 0 al reabrir, 0 spans duplicados, card CVE ausente, esquinas base identicas rgb(29,42,34) en escáner y hunter.

v0.62.4 — REVERSE-WEB: ingeniería inversa desde la web

Presentacion unicamente; los modulos RE no cambian.

- Los 3 modulos de ingenieria inversa (BIN-AUDIT, RE-ENGINE, DECOMPILE) existian solo por CLI/API: la web no tenia forma de subir un APK. Ahora el escáner tiene una seccion "Ingeniería inversa" en el menu lateral: elegir archivo (input file del navegador) o escribir la ruta del telefono, checkboxes por modulo (bin/re/decompile + modo --all para apps ofuscadas), y una card de resultados a lo ancho: FILTRACIONES arriba en rojo (sev critica/alta de cualquier modulo), luego las filas de cada modulo y el pseudocodigo de DECOMPILE en bloques.
- Backend: POST /api/reverse_run (corre los modulos pedidos sobre un path, aislamiento de fallos por modulo, normaliza hallazgos a lineas legibles) y POST /api/reverse_upload (recibe el archivo del navegador, sanea el nombre, guarda en /tmp/codexrc_reverse y devuelve el path).
- UX: al analizar, el drawer se cierra para que la card de resultados quede a la vista; en PC el sidebar siempre visible.

Validado en vivo con Chromium headless (390x844 y 1400x900): subida de zip con secret AKIA + endpoint staging -> detectado, seccion FILTRACIONES renderizada, /bin/sh real -> 3 hallazgos con imports ELF, drawer cierra tras analizar, seccion visible en PC sin abrir menu.

v0.62.5 — MULTI-IA: estado individual por operador + shard de cola

Una sola cosa cambio de lugar: el ESTADO. El repo guarda codigo y datos publicos (corpus, vdp_mapa); cada IA u operador guarda sus hechos en su propio workspace. Sin tocar el motor ni la arquitectura.

- Nuevo core/state.py: home() = \$CODEXRC_HOME si esta exportado, si no el repo mismo (compatibilidad total: sin la variable todo funciona exactamente como siempre). hechos() resuelve el hunt_wide_done.txt y los resultados al workspace propio.
- hunt_wide.py y plugin_batch.py escriben su avance/resultados via core/state (hechos/hunt_wide_done.txt, wide_hunt_results.json).
- Nuevo --shard N/M en hunt_wide y plugin_batch: reparto determinista del corpus por hash de slug (sha256 % M == N). Dos o mas IAs se reparten TODO el corpus sin coordinarse ni duplicar un solo blanco. Formato: --shard 0/2 y --shard 1/2.
- Estado 0 vivo del backlog y caza de 291 pagables corriendo sin cambios de comportamiento (sin CODEXRC_HOME = historico).

Validado en vivo: IA nueva con CODEXRC_HOME propio arranca con cola llena (2120) sin heredar nuestro avance; shard 0/2 + 1/2 = 1071 + 1049 = 2120 exactos sin solaparse; sin variable, cola 0 como siempre.

v0.62.6 — VDP-FRESH WATCHER: caza automática cada 6 horas

- Nuevo core/vdp_watcher.py: detecta pagables (710 con bounty) que sacaron version nueva desde la ultima auditoria y los manda SOLO a DIFF-HUNT. Determinista, Python puro, sin LLM: lo despierta un workflow del agente cada 6 horas y el agente solo triaga al final. Flags: --check (listar sin cazar), --limit N (prueba rapida), --sleep (pausa vs la API de wp.org).
- La memoria de "version auditada" (_auditadas) lee TODOS los resultados historicos del workspace de estado (wide_hunt_results y watcher_diff), respetando CODEXRC_HOME (multi-IA).
- La caza manual de los 8 pagables parcheados de hoy queda registrada como hechos/watcher_diff_20261003_manual.json para que el watcher no los repita.
- El parche fresco = mejor ventana para cazar parches incompletos: con esto dejamos de depender de acordarse de mirar.

Validado: --limit 12 sobre pagables reales (0 movidos, esquema de salida correcto); JSON de diff_hunt compatible con la memoria de versiones.

v0.62.7 — REVIVE-SOLO: Termux:Boot

boot_install.sh: instala ~/termux/boot/00-codexrc.sh para que cada reinicio de Android levante el server solo (wake-lock + auto_update). Requiere la app Termux:Boot de F-Droid, abrirla una vez, y luego bash boot_install.sh. Android puede matar Termux cuando quiera: con esto, cada arranque del telefono lo revive automaticamente.

9. ANEXO B — Docstring/cabecera completa de los 102 módulos

Para cada archivo de `core/*.py` se vuelca el bloque de comentarios o docstring de cabecera completo tal como está escrito en el código (no solo la primera línea).

```
=== __init__.py ===
# codexRC core package

=== ab_diff.py ===
#!/usr/bin/env python3
# =====
# codexRC - AB-DIFF universal (v0.68.0)
# -----
# DIFF de sesiones para sitios NO-WordPress (el concepto de la
# capa 4 de AUTHZ-PROOF portado a cualquier web). Consume
# objetos Identity de REG-BOT: jamas sabe si la cuenta nacio
# en WP, Laravel, Django, Express o una app propia.
#
# Tres niveles (decision AUTOMATICA segun el blanco):
# Nivel 1: registro libre -> sesiones ANON / A (duena de X) / B
# Nivel 2: sin registro pero credenciales propias -> ANON / OWN
# Nivel 3: nada -> ANON solo (fugas sin autenticacion)
#
# Relaciones (lo que hace el diff CONCLUYENTE):
# A = propietario del objeto X (recursos descubiertos desde la
# sesion de A, con los datos de A dentro)
# B = usuario independiente
# A->X permitido, B->X permitido = evidencia BAC/IDOR
# A->X permitido, B->X bloqueado = gate correcto (REFUTADO)
# ANON->X permitido = DEMO-UNAUTH (lo mas grave)
#
# Reglas de seguridad permanentes:
# - SOLO lectura A->B: peticiones GET, cero payloads de exploit
# - presupuesto de peticiones acotado (MAX_CALLS)
# - descarta blanco si REG-BOT dice phone/kyc
#
# CLI:
# python3 core/ab_diff.py https://sitio.com
# python3 core/ab_diff.py https://sitio.com --own email:pass
```

```
# python3 core/ab_diff.py https://sitio.com --mock http://127.0.0.1:8899
# =====

=== adaptive_hunt.py ===
"""ADAPTIVE-HUNT (v0.81.0): la capa epistemica.

No pregunta "que vulnerabilidad puedo probar" sino "que
experimento me daria la mayor informacion para distinguir
entre las hipotesis que tengo".

Flujo:
OBSERVACION (baseline) -> DISPARADOR -> HIPOTESIS ->
EXPERIMENTO (contrato pre-registrado) -> EVIDENCIA ->
ACTUALIZAR HIPOTESIS -> ... -> VEREDICTO + GAP LEDGER.

Tres salidas, todas validas:
BASELINE-CARACTERIZADO: un unico mecanismo vivo explica la
observacion (con soporte o por eliminacion honesta).
BASELINE-STABLE: el blanco no presento varianza.
UNKNOWN-DEMOSTRADO: la superficie no permite concluir HOY,
con razon demostrable e hipotesis abiertas enumeradas.

Presupuesto heredado: 30 requests max por sesion.
"""

=== arsenal.py ===
#!/usr/bin/env python3
"""ARSENAL v0.55.0 – dos niveles: BASICO (verde, sin candado) y EXTREMO (rojo, contrasena).

BASICO = lo que todo el mundo hace en un pentest rutinario: va en cajita verde,
visible siempre, para el lab y para calibrar la superficie del blanco.

EXTREMO = tecnicas avanzadas que requieren criterio y autorizacion: evasion de
WAF, exfiltracion fuera de banda (OOB), mutation XSS, DOM clobbering, phar://,
polyglots, bypass de filtros. Contraseña obligatoria + log de cada desbloqueo.

Politica que no cambia: demostracion minima, stop al primer impacto confirmado,
sin dump masivo, sin persistencia, sin destructivos (DROP/borrado jamas).
Uso CLI:
python3 core/arsenal.py --list
python3 core/arsenal.py --cat sql # nivel BASICO (verde)
python3 core/arsenal.py --cat sql --extreme # exige confirmacion escrita ACTIVO
"""

=== async_lane.py ===
# -*- coding: utf-8 -*-
"""
codexRC - SLIPSTREAM (carril async, v0.23.0)
=====
Segunda etapa del motor de velocidad OVERDRIVE. La bateria GET del corpus
(corpus/XSSHunter.test_params) disparada por un event loop: un SOLO hilo de
Python mantiene hasta `concurrency` sondas en vuelo simultaneamente via
I/O asincrono (httpx). En I/O de red el cuello de botella es el servidor,
no el CPU: el event loop no crea hilos ni cambia contexto, solo multiplexa
sockets, por lo que 64 sondas en vuelo pesan menos que 8 hilos y el
throughput sube de forma casi lineal hasta saturar al servidor objetivo.

Detalles que preservan la filosofia de la casa:
* Sesion heredada: el AsyncClient nace con las cookies y cabeceras de la
sesion autenticada del escaneo (caza logueada igual que siempre).
* Marcadores inertes: mismas sondas de lectura, ningun payload funcional.
* Politica de cortesia: cada tarea espera `delay` tras recibir respuesta
(igual que el modo secuencial), pero las esperas se solapan, por lo que
la tasa efectiva es `concurrency / delay`. En blancos delicados subir
`delay` o bajar workers baja la tasa de forma predecible.
* Fallback transparente: si httpx no esta disponible (p.ej. instalacion
minima en Termux), test_params usa el camino clasico de hilos.
"""

=== auth.py ===
"""
codexRC - Authentication Module (Termux friendly)
Supports 3 methods:
1. Session Cookies (recommended)
2. Automatic form login (username + password) with better CSRF & form detection
3. Bearer / JWT / Custom Authorization header

```

```

"""

=== bac_proof.py ===
#!/usr/bin/env python3
# =====
# codexRC - BAC-PROOF (v0.67.0, AUTHZ-PROOF capa 4)
# -----
# Validacion DINAMICA de candidatos BAC/IDOR en WP-LAB local:
# levanta WordPress (SQLite, sin MySQL) + PHP server, instala el
# plugin bajo prueba, crea un objeto victima del admin y dispara
# cada accion candidata como ANONIMO / SUSCRIPTOR / ADMIN.
# Difiere las respuestas y emite evidencia de ejecucion:
#
# DEMO-BAC-DINAMICO      el suscriptor obtiene lo mismo que el
#                        admin sobre el objeto ajeno
# DEMO-UNAUTH-DINAMICO   el anonimo obtiene lo mismo que el
#                        admin (nopriv explotado)
# REFUTADO-DINAMICO      el suscriptor/anonimo es bloqueado
#                        (gate/owner funciona en ejecucion real)
#                        -> alimenta FP-MEMORIA (capa 5)
#
# Requisitos: php + pdo_sqlite en PATH. Todo local: cero
# interaccion con terceros (regla mVDP: lectura A->B, sin impacto).
# =====

=== bin_audit.py ===
"""BIN-AUDIT: analisis binario/nativo, mas alla del navegador.

Abre cualquier artefacto que un vendor distribuye y le saca lo que
importa para la caza:

SECRETS HARDCODEADOS (critica, Patchstack paga: "BAC sobre objetos
sensibles: API keys, secrets"): claves AWS (AKIA...), Google (AIza...),
Stripe live (sk_live...), GitHub (ghp...), Slack (xox...), JWT,
claves privadas PEM, connection strings con credenciales, y en
archivos de texto pares api_key/secret/token/password = valor.

ENDPOINTS INTERNOS (alta): URLs http/ws escondidas en el binario,
incluyendo rutas de admin, staging y APIs privadas del vendor.

IMPORTS PELIGROSOS (media): simbolos dinamicos como system/execve/
popen/dlopen (superficie de RCE y carga de codigo en native code).

FINGERPRINT (info): tipo ELF/PE/Mach-O/zip/apk/jar, arquitectura,
bitness, seccion debug presente.

Formatos soportados: ELF (32/64, LE/BE, parse real de cabeceras y
.dynsym), PE (MZ), Mach-O, zip/jar/apk (recursivo: descomprime y audita
cada artefacto interno, .dex/.class/.so anidados), gzip. Cualquier otro
archivo = escaneo de strings generico.

Nivel 1 honesto: recon de binarios (strings, simbolos, secretos,
endpoints). Desensamblado completo queda fuera del alcance Python
puro; esto ya encuentra lo que paga.

Uso:
python3 core/bin_audit.py <archivo_o_dir> [--json] [--top N]
POST /api/bin {"path": "..."} (backend)
"""

=== brain.py ===
"""CEREBRO: la capa de inteligencia determinista del Hunter.

Dos funciones, cero IA (a proposito: exactitud, costo cero, depurable):

1. FINGERPRINT identifica que es el blanco ANTES de gastar sondas:
WordPress (+ plugins y theme via wp-content), Angular/React/Vue/Next,
Laravel, servidores (headers X-Powered-By / Server). Si detecta un
vendor con historial de CVEs, lo grita: pista de vendor-farming.

2. PRIORIZACION puntua cada objetivo del spider (endpoint + parametro)
y ordena la cola de caza: admin-ajax, uploads, ids, settings, api...
arriba; utm_*, fbclid, previews y assets abajo. Cada objetivo recibe
un PRESUPUESTO de sondas proporcional a su valor, para no martillar.

Las baterias consumen la cola ya rankeada: el mismo presupuesto total se

```

```
concentra donde duele. Todo determinista: mismo blanco, mismas decisiones.
"""
```

```
=== cache_bait.py ===
"""CACHE-BAIT: bateria dedicada de cache poisoning y cache deception.
```

Todo PASIVO (no se envenena la cache de nadie):

1. ENTRADAS UNKEYED: cabeceras de routing que muchos caches/CDN no incluyen en la clave (X-Forwarded-Host, X-Rewrite-URL, Forwarded...). Si se REFLEJAN sin validar, cualquiera puede fijar contenido en la cache para TODOS los usuarios. Se reporta como candidata.
2. WEB CACHE DECEPTION: pedir rutas privadas con extension de asset (/api/user/profile.css). Si el servidor responde 200 con los mismos datos privados y content-type cacheable, un proxy puede guardar paginas PRIVADAS y servir las a extraños.
3. PROBE EN MULTIPLES PAGINAS descubiertas por la arana (no solo la base).

Basado en las tecnicas de la guia practica de cache poisoning web (unkeyed inputs + cache deception), adaptadas a solo-lectura.

```
=== cache_baseline.py ===
#!/usr/bin/env python3
"""CACHE-BASELINE (v0.79.0): clasifica el baseline del target.
```

STABLE	nucleo y secundarias identicas en K corridas
VARIANT	la variacion existe PERO es caracterizable (temporal: Age/Date/Last-Modified avanzan con nucleo estable; load-balancing: Server varia con cuerpo estable)
AMBIGUO	variacion NO explicada (posible bot management, personalizacion o backend dinamico sin caracterizar)
INVALID	sin observaciones utilizables

AMBIGUO e INVALID conducen conservadoramente a UNKNOWN en el juez: sin baseline usable no hay poder discriminatorio.

```
=== cache_controls.py ===
#!/usr/bin/env python3
"""CACHE-CONTROLS (v0.79.0): explicaciones alternativas ANTES
de elevar un diferencial.
```

Cada control emite PASSED / FAILED / NOT_APPLICABLE con EVIDENCIA real que lo respalda. Nada decorativo: un PASSED siempre corresponde a una comprobacion efectiva.

Controles CRITICOS (pueden explicar un diferencial y degradarlo a BENIGN): cookies_session, authorization, vary, ttl_wait, bot_management.

```
=== cache_correlation.py ===
#!/usr/bin/env python3
"""CACHE-CORRELATION (v0.79.0): el orquestador.
```

Pregunta central: ¿dos representaciones que deberian ser equivalentes generan estados de cache distintos, o dos representaciones distintas terminan compartiendo un estado que no deberian compartir?

REGLA FUNDAMENTAL: no asumimos ni afirmamos conocer la cache key interna. CacheBindingEvidence expresa FUERZA DE EVIDENCIA EXTERNA compatible con binding/correlacion; no es un descubrimiento de la key.

Flujo: SemanticRelation -> CacheFingerprint -> Baseline -> Controls -> Correlation -> Target Judge.

EVIDENCIA E0-E6 (no son puntos, no hay confidence=0.85):

- E0 CACHE OBSERVATION
- E1 REPRODUCIBILITY
- E2 SEMANTIC DIFFERENTIAL
- E3 CACHE BINDING
- E4 DOWNSTREAM DIFFERENTIAL

E5 CROSS-CONSUMER IMPACT
E6 SECURITY IMPACT

Juez determinista. DEMO es difícil de alcanzar por diseño:
exige binding STRONG + (E5 o E6) + controles críticos PASSED +
atribución razonable + baseline no ambiguo. NUNCA se produce
DEMO por: Age diferente, HIT/MISS, ausencia de headers, body
diferente solo, dos requests diferentes, convergencia
aparente, inferencia de cache key, o E0+E1.
"""

```
=== cache_fingerprint.py ===
#!/usr/bin/env python3
"""CACHE-FINGERPRINT (v0.79.0): huella SOLO de senales
observables.
```

Cada señal es tri-estado:

OBSERVED	presente en la respuesta
ABSENT	ausente en la respuesta
UNKNOWN	no determinable (respuesta ilegible)

REGLA DEL OPERADOR: header ausente NO significa cache miss.
Age: ABSENT no puede convertirse en CACHE-MISS. Age/Date/
timing y orden de headers son señales SECUNDARIAS: nunca
producen diferencial por sí solas ("Age diferente" no es
diferencial). El diferencial real compara el núcleo:
status, body_hash, etag, last_modified, content_type,
cache_control, vary.
"""

```
=== cache_semantic.py ===
#!/usr/bin/env python3
"""CACHE-SEMANTIC (v0.79.0): relacion entre Request A y Request B.
```

Regla fundamental del operador: NO asumir ni afirmar que
conocemos la cache key interna. cache_key(A) == cache_key(B)
es solo HIPOTESIS de correlación; lo observable son huellas
externas. La escalera: hipótesis != observación != evidencia
!= veredicto.

Conjunto CERRADO y pequeño de transformaciones cuya
neutralidad es demostrable por regla RFC. Si una diferencia no
pertenece al conjunto: UNKNOWN. No se fuerza EQUIVALENT.
"""

```
=== cfg.py ===
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""SEMANTIC CORE (v0.58.1) – Nivel 1: CFG intra-función + dominancia.
```

Tokenizer PHP-lite -> árbol de statements -> grafo de control de flujo
-> dominadores (algoritmo iterativo) -> consultas semánticas:

dominates(cfg, gate_node, sink_node)	¿todo camino a sink pasa gate?
gate_dominates_sink(...)	el gate protege el sink?
router_noise(...)	gate domina todos los sinks
	sensibles => la comparación
	'auth' es solo router (FP)

Jerarquía de precisión: una consulta de este módulo SIEMPRE gana a
una ventana de texto (+-1, +-5 líneas). Si el parseo falla, el llamador
debe caer a su heurística textual previa (fallback).

Nivel honesto 1: intra-función. No resuelve includes ni herencia.
Python puro, Termux/armv7l OK.
"""

```
=== chain.py ===
"""ENCADENAR HALLAZGOS: el valor real está en las CADENAS, no en piezas.
```

Un XSS solo suele ser P3. Ese mismo XSS en un sitio con CSP débil es un
robo de sesión. Un IDOR con email + un endpoint de settings es un posible
account takeover. Patchstack y compañía pagan por IMPACTO COMBINADO.

Reglas deterministas sobre hallazgos YA verificados (corre tras VERITAS):
1. XSS + CSP débil en el mismo blanco -> XSS ejecutable, sesión robable

2. XSS + ruta admin mapeada -> el vector puede golpear al admin
3. XSS almacenado/blind + panel admin -> captura de sesion de admin
4. IDOR repetido en varios id -> filtracion MASIVA, no caso puntual
5. IDOR con datos personales + endpoint settings/token -> encaminado a account takeover
6. SQLi + panel admin en el blanco -> extraccion de credenciales
7. SQLi + BAC en el mismo blanco -> doble via a datos privados

Las cadenas no tocan los hallazgos originales: se reportan aparte con su propia severidad combinada y razon humana lista para el informe.

"""

=== cl0_probe.py ===

#!/usr/bin/env python3

-*- coding: utf-8 -*-

"""

v0.87.0 CL.0-SINGLE-TIER (Request Smuggling CL.0)

=====

Familia CL.0 con la disciplina de la linea DESYNC:
contratos pre-registrados en el ExperimentGraph, juez determinista, presupuesto bajo y cero-FP por diseno.

Mecanica CL.0: POST con 'Content-Length: 0' cuyo body declara 0 bytes pero el stream lleva un prefijo smuggleado. Si el front NO parsea el prefijo como request propio y el backend NO consume el stream residual, el prefijo es procesado por el backend como request invisible para el front: la cola de respuestas queda desplazada.

Oraculo observable (nada inferido):

- T1 ventana de silencio tras POST+smuggle:
respuesta temprana => el front parseo el prefijo (pipelining benigno, NO desync).
- T2 tras el followup: si aparece la respuesta del smuggle (eco del token) SIN atribucion del front => el backend proceso bytes que el front no contabiliza => MISMATCH observable.
- S doble followup: si la respuesta del segundo followup porta el token del PRIMERO => la cola esta desplazada (STATE EFFECT observable).

Regla heredada: el eco del token SOLO no es desync. Sin diferencial de timing o atribucion observable, el veredicto NO escala. Sin repro 2/2 no hay DESYNC.

Contratos (pre-registrados):

- baseline: C0 control GET @base (firma de respuesta);
C1 POST vacio + followup en conexion fresca (r2 debe matchear C0: si no, UNSTABLE).
- variante (endpoint candidato, redirect/404 family):
P probe: POST CL:0 + smuggle + followup1 + followup2
R repro: idem en conexion nueva

Escalera determinista:

- BENIGN -> CL0-DETECTED (1/2, probable, NO reportable)
- > CL0-DESYNC (repro 2/2) -> CL0-STATE-EFFECT
- (+ BENIGN-FRONT / UNSTABLE / UNREACHABLE / UNKNOWN)

Presupuesto: 2 baseline + 1 variante x 2 fases x 3 reqs = max 8 requests. Read-only: el smuggle es un GET propio con token unico, jamas recursos de terceros.

"""

=== connection_state.py ===

#!/usr/bin/env python3

"""CONNECTION-STATE AUDIT (v0.77.0): ¿deja una peticion ambigua un estado observable que modifica la interpretacion de una peticion posterior sobre la misma conexion?

Arquitectura: A -> B sobre UNA conexion.

- baseline: K corridas de A0 (GET limpio) -> B (GET limpio)
- tratamiento: A1 (framing aceptado / veneno) -> B identico
- comparacion: B0 == B1 ? sobre huella, conteo, cierre, latencia

REGLA (anti falso positivo): "la respuesta cambio" NO es smuggling. Primero se demuestra QUE cambio y DONDE:

- B recibio SU respuesta (identidad preservada)
- B recibio una respuesta AJENA (marcador: contaminacion cruzada demostrable)
- B no recibio nada (traga)
- hay una respuesta extra con B bien atendido (pipelining legitimo cuando el edge honra TE: RFC 7230, no hallazgo)

Estado de la conexion como evidencia (STATE-CONTINUITY):
 parser_state (respondio A? cuantas respuestas?), reuse,
 request/response counter, connection_close, cache_context,
 origin_context (ECO). Lo no observable queda unknown.

Clasificacion por tratamiento:

REJECTED	A1 recibio 400/cierre
STATE-STABLE	B atendido con huella dentro del baseline
STATE-STABLE+PIPE	idem + respuesta extra (edge honro TE: pipelining correcto, se anota, no escala)
STATE-CHANGED	B desatendido o con huella fuera de la varianza baseline
BASELINE-AMBIGUO	el baseline varia solo (sin poder discriminatorio: nada se concluye)

Escalera STATE-CHANGED: ¿reproducible? no -> UNKNOWN;
 si -> SOSPECHA; y si ademas B recibio respuesta AJENA (marcador) reproducible -> DEMO (contaminacion cruzada demostrada: la respuesta que llego a B pertenece a otro request).

Presupuesto: perfil v0.75 (11) + baseline K=3 (6) + T1 por variante aceptada (2 c/u) + T2 veneno (2) + repro (2). Tope 28, secuencial, lectura A->B, marcador = path inexistente unico por corrida.

```
=== cov_bait.py ===
"""COV-BAIT v2: caza guiada por COBERTURA DE CODIGO + MOTOR DE MUTACION.
```

Etapas 1: medir cobertura por sonda (baseline, params ocultos, paginas).

Etapas 2 (nuevo): FUZZING CON FEEDBACK estilo XBOW:

- corpus de inputs: arranca con la query de la semilla y los params ocultos que ejecutaron codigo nuevo en la etapa 1
- mutaciones deterministas (añadir param, mutar valor, quitar param)
- cada sonda se mide en el navegador: si EJECUTA rangos nuevos, el input entra al corpus como PADRE y las mutaciones siguen desde ahi (hill-climbing por cobertura)
- presupuesto fijo de sondas + freno por estancamiento (12 sondas sin cobertura nueva = para) + anti-bucles (hash de inputs ya probados)
- reporta las COMBINACIONES que desbloquean funciones escondidas, con los nombres de las funciones ejecutadas

Todo determinista: rng con semilla fija, resultados reproducibles.
 Requisitos: navegador Chromium + websocket-client (python puro, Termux ok).

```
=== crc_probe.py ===
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

v0.90.0 CROSS-REQUEST CORRELATION
 =====

Reconstruccion de la cola del origen por CORRELACION entre requests, con la disciplina de la linea DESYNC: contratos pre-registrados en el ExperimentGraph, juez determinista, presupuesto bajo y cero-FP por diseno.

Mecanica CROSS-REQUEST CORRELATION: un POST CL.0 lleva DOS requests smuggleadas (T1, T2: GETs propios con tokens unicos). Si el origen procesa el residual como cola, las respuestas de los followups quedan DESPALAZADAS por k consistente y MEDIBLE:

k=0	[ECHO-fb,	ECHO-fc,	ECHO-fd]	alineado
k=1	[SMUGGLE-T1,	ECHO-fb,	ECHO-fc]	shift 1
k=2	[SMUGGLE-T1,	SMUGGLE-T2,	ECHO-fb]	shift 2

Oraculo observable (nada inferido): la PERMUTACION de la

coleccion. La firma del desync es la correlacion
CONSISTENTE: el eco propio aparece desplazado
EXACTAMENTE por el numero de ecos del smuggle
observados (fi == si_count). Un patron disperso (ecos
sin permutacion consistente) NO escala a STATE-EFFECT.

Regla heredada (v0.87-89): el eco del token SOLO no es
desync; sin permutacion observable, el veredicto NO
escala. Sin repro 2/2 no hay DESYNC.

Auto-control por atribucion (sin baseline separado): el
eco propio del followup en la posicion 0 es el control
de cada fase; sin eco propio ni eco del smuggle:
UNKNOWN (no BENIGN, cero-FP).

Contratos (pre-registrados):
fase P POST doble-smuggleado (T1, T2) + 3 GETs
(fb, fc, fd) en conexion fresca
fase R repro idem con tokens nuevos

Escalera determinista:
BENIGN -> CRC-DETECTED (1/2, probable, NO reportable)
-> CRC-DESYNC (repro 2/2)
-> CRC-STATE-EFFECT (ademas permutacion consistente,
k medible y reproducible)
(+ BENIGN-EDGE / UNSTABLE / UNREACHABLE / UNKNOWN)

Presupuesto: 1 variante x 2 fases x 4 reqs (A: 1 POST
doble-smuggleado; B: 3 GETs) = max 8 requests. Read-only:
los smuggles son GETs propios con tokens unicos, jamas
recursos de terceros.

Aprendizajes v0.87-89: bytes crudos jamas repr(b'..');
buffers PERSISTENTES por conexion; grace de ciclo de
vida antes de heredar.
"""

```
=== cross_layer.py ===  
#!/usr/bin/env python3  
# -*- coding: utf-8 -*-  
"""
```

v0.84.0 CROSS-LAYER CORRELATION
=====

Un solo disparo, tres capas observadas SIMULTANEAMENTE
(Edge / Cache / Origin) + dimension de conexion (A, B, control).

Filosofia heredada:
- Solo evidencia observable; nada inferido queda fuera de 'unknown'.
- Tolerancia del edge NO es vulnerabilidad: si el origin nunca
ve la perturbacion, el veredicto es BENIGN (EDGE-ONLY).
- Presupuesto bajo: <= 11 requests por auditoria.
- Contratos pre-registrados en el ExperimentGraph (anti post-hoc).

Clases de evidencia (ExperimentGraph):
- E-XL-MISMATCH : el edge acepta una perturbacion que el
origin sirve DISTINTO (semantica difiere).
- E-XL-SHARED : el efecto se reproduce desde 2 conexiones
(estado compartido, clase cache).
- E-XL-CONTAMINATION: una conexion nueva (control) recibe el
efecto sin haberlo pedido.

Escalera determinista (en el informe):
OBSERVED -> REPRODUCIBLE -> CROSS-LAYER ->
SHARED-STATE -> CONTAMINATION -> DEMO

DEMO exige: MISMATCH reproducido (2 conexiones) +
CONTAMINATION observable + controles PASSED. Nada se infiere.
"""

```
=== crown_chain.py ===  
#!/usr/bin/env python3  
# -*- coding: utf-8 -*-  
"""
```

v0.95.0 CROWN CHAIN — INTEGRACION AL HUNTER
=====

Corona alcanzada (LAB 22/22, corpus 128/128): la

cadena completa se integra al Hunter como UN orquestador que sube la escalera con los motores del roadmap, sin tocar la arquitectura de ningun modulo.

rung 1	DETECTION	cl0 + h2 + mc + crc (4 probes, veredictos propios, cero-FP cada uno)
rung 2	REPRODUCIBLE	repro_engine (2 genealogias x pase A/B, firma k igual)
rung 3	STATE-EFFECT	firma k medible y consistente
rung 4	CROSS-CONNECTION	contaminacion entre conexiones PROPIAS (mc)
rung 5	SECURITY IMPACT	LIMITE HONESTO: en vivo NO se ejecuta: envenenar la respuesta de un usuario real de terceros esta fuera de toda regla. Rung validada solo en LAB (impact_probe, recurso protegido simulado). La garantia cero-FP queda citada (RC-000254).

Modos:

lab	ensambla el paquete completo via evidence_package (deteccion k=2, 2 genealogias, impacto 2/2) -> CHAIN-COMPLETE-LAB.
live	escalera contra el blanco real, read-only, presupuesto declarado 60 reqs; NUNCA supera CROSS-CONNECTION-DEMO: sin repro 2/2 o sin k consistente NO avanza.

La escalera JAMAS se salta: DETECTED sin repro no escala; repro sin firma k consistente no escala; el disparo unico (flaky) NO escala.

"""

=== cspt_scan.py ===

#!/usr/bin/env python3

"""CSPT-SCAN (v0.71.0): Client-Side Path Traversal.

Clase nueva (2025-2026): el JS del sitio concatena input del usuario en la RUTA de un fetch()/axios -> se engana al navegador autenticado para llamar endpoints que no debería ("../" en el path). Termina en SSRF-del-cliente, CSRF o XSS encadenado.

Detecta estaticamente el patron FUENTE (query/hash de URL) -> SINK (fetch/axios/xhr con path concatenado) en los .js del plugin/tema.

Uso:

python3 core/cspt_scan.py <root>

"""

=== cve_matcher.py ===

"""

codexRC - CVE Matcher Module

Correlates detected technologies/versions with known CVEs.

Uses the public CIRCL CVE Search API (no key required for basic use).

"""

=== decompile.py ===

"""DECOMPILE: descompilacion parcial nivel 3 (bytecode DEX -> pseudocodigo).

REVERSE (v0.36) leyo la ESTRUCTURA (clases, metodos, strings). DECOMPILE

baja un nivel mas: abre el CODE_ITEM de cada metodo y DESensambla sus instrucciones reales (opcode dalvik) resolviendo lo que referencian:

```
const-string v0 = "AKIA..." (strings de la tabla)
invoke-static Lcom/sdk/Lic;->verify (metodos por nombre y clase)
if-eqz, goto, return... (flujo de control)
```

Resultado: pseudocodigo legible de los METODOS DE LOGICA SENSIBLE (decrypt/license/sign/verify/auth/checkout) sin ejecutar nada y sin necesitar JRE/dex2jar (que no existen para armv7l). Un secret que aparece como const-string dentro del flujo de un metodo se reporta

como critica "secret EN EJECUCION".

Nivel honesto: desensamblado resuelto del subset comun de dalvik (~45 opcodes: constantes, strings, fields, invokes, saltos, retornos). Opcodes fuera del subset se marcan y el metodo se corta ahi. No es Java fuente 100%; es lo que un reverser lee primero.

Uso:

```
python3 core/decompile.py <apk|dex> [--all] [--top N] [--json]
POST /api/decompile {"path": "..."} (backend)
"""
```

=== deep_scan.py ===

"""

codexRC - Deep Scan Module

Auditoria profunda: deteccion de WAF/CDN, certificado TLS y mapeo de dominios.

Todo en Python puro para que corra igual en Termux (armv7l) sin binarios extra.

"""

=== desync_hunt.py ===

"""DESYNC-HUNT (v0.83.0): investigacion DESYNC sobre el
EXPERIMENT-GRAPH.

Consume EDGESYNC (v0.74: sondas de framing con 3 dimensiones) SIN reemplazarlo. Cada sonda es un EXPERIMENTO con contrato pre-registrado; cada disparo produce una OBSERVACION cruda con huellas de reproducibilidad; el contrato convierte senales en EVIDENCIA; el candidato DESYNC avanza solo con evidencia.

Escalera de impacto (nunca se salta):

```
DESYNC OBSERVED -> REPRODUCIBLE -> STATE EFFECT ->
CROSS-CONNECTION -> SECURITY IMPACT
```

La corona (CONFIRMED) exige E-DESYNC-IMPACT reproducible: efecto de seguridad fuera del propio experimento. La mera discrepancia de parsing no basta.

"""

=== diff_hunt.py ===

#!/usr/bin/env python3

=====

codexRC - DIFF-HUNT (v0.45.0)

Pivote de estrategia: en vez de auditar plugins enteros

(los top ya estan blindados), caza SOLO el codigo nuevo.

Descarga la version actual y la anterior de cada plugin con

VDP activo, calcula el diff (lineas agregadas/modificadas en

archivos .php propios del plugin) y corre Taint-Trace +

CVE-MATCH + GATES-AUDIT + FP-AUTO-CLOSE restringido a esas

lineas nuevas.

#

Logica: lo recién escrito no paso por ningun auditor ni

por los bots de los demas hunters.

#

Uso:

python3 core/diff_hunt.py slug1 slug2 ... [--vdp vdp.json]

python3 core/diff_hunt.py slugs.txt --vdp vdp.json --out r.json

Salida: solo hallazgos VIVOS sobre lineas nuevas. [HALLAZGO] = critico.

=====

=== dossier_hunt.py ===

#!/usr/bin/env python3

=====

codexRC - DOSSIER-HUNT (v0.96.1) — caza con las clases Q4-2026

Driver de caza READ-ONLY sobre fuentes publicas de plugins

WordPress (downloads.wordpress.org), con las clases nuevas del

dossier: POI-REACH (Object Injection con alcance+gadget) y

MAGIC-CONFUSION (extension vs contenido hacia Imagick).

#

Reusa la infra de plugin_batch (descarga + filtro de ruido de

librerias). Anti-FP: hallazgos dentro de vendor/lib NO se

reportan (NOISE_RE de plugin_batch).

#

Uso:

```
# python3 core/dossier_hunt.py slug1 slug2 ...
# python3 core/dossier_hunt.py --populares 40
# python3 core/dossier_hunt.py slug --version 1.2.3 # calibracion
#
# Escalera honesta: solo POI-REACH y MAGIC-CONFUSION son
# [HALLAZGO]; POI-AUTH/POI-DETECTED son pistas, CONFUSION-GUARDED
# es defensa correcta.
# =====
```

```
=== edge_profile.py ===
#!/usr/bin/env python3
"""EDGE-PROFILE (v0.75.0): caracteriza el CONTRATO del edge.
```

No busca desync, no lanza smuggles. Responde una sola pregunta:

¿Que transformaciones aplica cada frontera antes de entregar el mensaje a la siguiente capa?

Metodo: sondeos benignos (framing contradictorio con cuerpo VACIO, sin contenido smuggleado) y observacion de lo que regresa. Nada se infiere: lo no observable queda "unknown".

Ficha:

version	HTTP observado
reuse	la conexion reutilizada responde secuencial
cierre_rechazo	el rechazo viene con cierre de conexion
cl_te / te_doble	te_espacio / te_case rejected accepted unknown
normalizacion	normalized conserved unknown (observable solo si el downstream hace ECO; en blancos reales sin eco queda unknown)
downstream	reached (eco visto) unknown
cache	firma Age/X-Cache entre requests repetidos
origin	unknown (siempre: no observable)

Presupuesto: 1 conexion de reutilizacion (3 GET) + 4 probes benignos (1 conexion, probe+sonda cada uno) = ~11 requests, secuencial, lectura A->B, cuerpos <= 4KB.

```
=== edgesync.py ===
#!/usr/bin/env python3
"""EDGESYNC-HUNT (v0.74.0): SONDA-DE-CORRELACION.
```

Salto sobre v0.73: el veredicto ya no nace de una sola dimension (el eco). Ahora cada variante se observa en TRES dimensiones sobre la MISMA conexion:

dim1 framing	respuesta al mensaje ambiguo (eco / conteo)
dim2 persistencia	la conexion sigue viva tras el ambiguo, o el edge la mata? un cierre activo del edge significa algo muy distinto a una conexion que sigue con estado inconsistente
dim3 estado	el request siguiente (sonda) conserva su estado esperado, o llega desplazado/envenenado?

La pregunta que responde: "¿el comportamiento observado demuestra que DOS componentes interpretaron el mismo mensaje de forma diferente?" Escalera de veredictos:

RECHAZO-EDGE	el edge corto el framing ambiguo (400/close): postura activa; nada observable, no se gasta presupuesto extra
SIN-DESYNC	el edge acepto y todo cuadra (o mix aceptado+rechazado)
SOSPECHA	una sola dimension desacuerda, sin eco
DESYNC-DEMO	eco (o sonda envenenada) REPRODUCIBLE, con la segunda capa pillada interpretando distinto

TOPOLOGY-PRE: pasada inicial barata que lee Server/Via/CF-Ray/Age/X-Cache del baseline y clasifica el blanco (cdn-blindado | proxy-intermedio | directo). CDN blindado -> bateria reducida (V1, V2, P1); el resto -> bateria completa. La topologia queda como evidencia y alimenta a UNIVERSAL-RECON.

Variantes: 9 framings (v0.73) + 2 POISON:

```
P1 CL.TE: prefijo de envenenamiento (request sin terminar)
    dentro de la ventana CL; el back lo pega a la sonda y la
    respuesta de la sonda sale POR el camino del veneno
P2 TE.CL: el mismo prefijo dentro del data de un chunk
```

Los ecos TE.CL y P2 (familia TECL) siguen siendo CANDIDATOS (desync o pipelining: ambiguo sin prueba cruzada) y van a listas propias: tecl_candidatos y poison_candidatos.

SEGURIDAD (anti-DoS, no negociable):

- MAX_PROBES pruebas totales (baseline + 9 framings + 2 poisons + reproducciones), 1 conexion secuencial por variante
- cooldown entre probes, sin flood, sin RST, cuerpos <= 4KB
- stop al primer DESYNC-DEMO
- lectura A->B: todo lo que viaja smuggleado es de solo lectura

```
=== escalada.py ===
```

```
"""ESCALADA: que hacer DESPUES del aviso critico.
```

El CHAIN (v0.28) dice "tenes una cadena critica" y ahi se quedaba: el operador se quedaba en el blanco sin saber el siguiente paso. ESCALADA (v0.38) continua solo, con reglas deterministas y canarios inertes:

- PASO A - VERITAS DIRIGIDO: re-verifica en navegador real SOLO las piezas XSS de la cadena (las que armaron el caso), no todo.
- PASO B - TECHO DE IMPACTO: pide el blanco con la sesion heredada y revisa las flags de las cookies: sin HttpOnly + XSS ejecutable = camino a HIJACK de sesion (el techo).
- PASO C - KIT PoC: genera la pagina atacante LOCAL (poc/<host>.html) con iframe + hash canario + postMessage canario. El operador la abre en su maquina y VE si la cadena aterriza. Canariz inerte: banner visible, cero exfiltracion.
- PASO D - PLAYBOOK: pasos ordenados que faltan, segun las piezas que tenga la cadena (persistencia, admins, reporte).

Todo es lectura y render inerte: nada de datos reales, nada sale del equipo del operador. Si no hay cadenas criticas, no corre.

```
=== evidence.py ===
```

```
#!/usr/bin/env python3
```

```
# =====
```

```
# codexRC - EVIDENCE-CHAIN + ABOGADOS (v0.47.0)
```

```
# -----
```

```
# Motor de razonamiento de evidencia (estilo RacerD/Infer):
```

```
# no pregunta "¿podria ser vulnerable?" sino
```

```
# "¿que evidencia tengo para afirmar que realmente lo es?".
```

```
#
```

```
# Por cada finding construye una CADENA DE EVIDENCIA:
```

```
#
```

```
# SOURCE      ¿input controlable por el atacante?
```

```
# FLOW        ¿como llega hasta el sink? (saltos incluidos)
```

```
# AUTH        ¿que compuertas lo custodian? (nonce/caps/nopriv/REST)
```

```
# SANITIZATION ¿existe sanitizacion efectiva en la ruta?
```

```
# SINK        operacion peligrosa final
```

```
# CORRELATION ¿que analizadores lo vieron? (TAINT/GATES/PATTERNS/DIFF)
```

```
# DYNAMIC     evidencia dinamica (si existe)
```

```
#
```

```
# Sobre la cadena actuan ABOGADOS deterministas:
```

```
# FISCAL      debe PROBAR: source controlado + taint al sink +
```

```
#             sin sanitizacion + alcance sin privilegios.
```

```
# DEFENSA     busca REFUTACIONES: gate protegido, sanitizador,
```

```
#             prepare seguro, contexto admin-only, inalcanzable.
```

```
# JUEZ        pesa evidencia y dicta. NO es un LLM: reglas fijas.
```

```
#
```

```
# Veredictos (escala RacerD: se reporta solo lo demostrable):
```

```
# CONFIRMED    prueba estatica completa + dinamica reproducida
```

```
# DEMOSTRADO-ESTATICO prueba estatica completa, falta dinamica
```

```
# PROBABLE     flujo probado, alcance/dinamica sin resolver
```

```
# CONTESTADO   defensa tiene refutacion parcial
```

```
# DESCARTADO   refutacion fuerte (gate/sanitizado/admin-only)
```

```
#
```

```
# Cada cadena expone FALSADORES: que evidencia la tumbaria.
```

```
# Meta: el scanner que necesita MENOS confianza humana por finding.
```

```
# =====
```

```
=== evidence_package.py ===
```

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
v0.94.0 EVIDENCE PACKAGE
```

```
=====
```

Etapla FINAL del roadmap: convierte lo que sobrevivio a toda la escalera en el paquete FALSABLE que exige el operador: un tercero puede re-ejecutar y llegar al mismo veredicto.

Cadena que empaqueta (ninguna rung inferida, todo re-ejecutable):

```
RUNG 1 DETECTION+STATE-EFFECT  crc_lab seq_desync:
      firma CRC-STATE-EFFECT k=2, media por pase
      del reproduction engine.
RUNG 2 REPRODUCTION  repro_engine: 2 genealogias
      INDEPENDIENTES (instancias frescas) x pase
      A (deteccion) + pase B (replay): REPRODUCIBLE
      exige firma identica en A y B de TODAS.
RUNG 3 SECURITY IMPACT  impact_probe contra
      impact_lab pool_swap: la victima (TCP
      distinto) recibe la respuesta del recurso
      protegido smuggleado, 2/2 rondas, control
      limpio: SECURITY-IMPACT-DEMO.
RUNG 4 FP-GUARANTEE (cita): la bateria de cebo de
      fp_elimination verifico 0/4 cebos
      reportables con control positivo activo
      (caso de corpus RC-000257).
```

Cada rung lleva su presupuesto y el paquete declara LIMITES explicitos: read-only, modo lab, recurso protegido SIMULADO (dato del lab, jamas un recurso de terceros), presupuestos declarados, escalera completa sin saltos.

Falsabilidad: el paquete incluye los comandos exactos de reproduccion y un hash sha256 del payload canonico; verify_package() re-computa el hash y valida los campos obligatorios: un tercero re-ejecuta la cadena y compara.

Presupuesto: repro (32) + impact (5) = 37 reqs, techo declarado 40. Read-only.

```
"""
```

```
=== experiment_catalog.py ===
```

```
"""CATALOGO DE EXPERIMENTOS (v0.81.0): sondas HTTP de solo
lectura con CONTRATO pre-registrado.
```

Cada experimento declara ANTES de ejecutar que resultados apoyarian o refutarian que hipotesis. El contrato se evalua despues con el bundle de resultados acumulado: nada se racionaliza a posteriori.

Orden de ejecucion: fijo por dependencia (INTRA -> CROSS -> SESSION -> VIRGIN -> TIME). La parte adaptativa es PARAR temprano cuando el juez converge, y el gap ledger que queda si no converge.

```
"""
```

```
=== experiment_graph.py ===
```

```
"""EXPERIMENT-GRAPH (v0.83.0): grafo dirigido de
investigacion para la linea DESYNC.
```

Principio inviolable:

```
Hypothesis != Observation != Evidence != Verdict
```

Flujo obligatorio:

```
HYPOTHESIS -> EXPERIMENT -> OBSERVATION -> EVIDENCE ->
CORRELATION -> VERDICT
```

Reglas:

- Todo contrato se registra ANTES de ejecutar (anti post-hoc).

```

- Journal append-only: la evidencia previa NUNCA se
sobrescribe; se anula con evidencia nueva citada.
- Sobrevivir experimentos NO demuestra una hipotesis:
solo evidencia explicita de apoyo la marca SUPPORTED.
- Contradiccion es terminal: gana, pero el historial
conserva el apoyo previo.
- La reproducibilidad NO es "dos respuestas iguales":
exige huellas completas (request, conexion, reuso,
timing, respuesta, estado, cache, baseline, controles,
genealogia) reconstruibles.
- CONFIRMED exige E-DESYNC-IMPACT + invariantes
anti-falso-positivo limpias. La corona no se fabrica.
"""

=== experiment_selector.py ===
"""SELECTOR DE EXPERIMENTOS (v0.82): grafo de experimentos.

Responde UNA pregunta: dadas las hipotesis vivas y el
presupuesto, ¿cual es la siguiente prueba que mas
informacion me puede dar?

EDV (Valor de Discriminacion Esperado) por experimento:
para cada RAMA POSIBLE de su contrato (union estatica
pre-registrada, parte del contrato, no una prediccion):
    valor_rama = suma de pesos sobre hipotesis VIVAS
    CONTRADICT = 2.0 (contradiccion es terminal: reduce
                    el conjunto vivo de forma determinista)
    SUPPORT    = 1.0 (apoyo solo prepara convergencia)
EDV = media de valores de rama.

Sin probabilidades inventadas: las ramas se ponderan uniforme
porque NO se conoce su probabilidad; EDV es una heuristica
de ORDENAMIENTO, no de veredicto. Ningun resultado afecta
que hipotesis es apoyada o refutada: eso lo decide el
contrato.

DAG DE DEPENDENCIAS (del propio catalogo):
    INTRA-CONN -> CROSS-CONN -> VIRGIN
    INTRA-CONN -> TIME-OFFSET
Un experimento no es elegible si falta un prerequisite.

DESEMPATES (deterministas, en orden):
1. mayor EDV
2. epsilon: discriminante en la ventana previa de memoria
   (eficiencia, nunca evidencia)
3. orden de dependencia del catalogo

PARADA HONESTA: si ningun experimento restante puede
tocar una hipotesis viva (EDV == 0 en todos), el loop se
detiene y declara que NINGUN experimento disponible
discrimina las hipotesis vivas. El UNKNOWN conserva esa
razon, no un silencio.
"""

=== fp_autoclose.py ===
#!/usr/bin/env python3
# =====
# codexRC - FP-AUTO-CLOSE (v0.44.0)
# -----
# Segunda capa de verificacion tras GATES-AUDIT: reconoce los
# patrones de falso positivo que se repiten lote tras lote y
# los dictamina solo, para que el operador/agente solo vea
# candidatos reales.
#
# Veredictos automaticos (_fp):
#   FP-GATE-PROTEGIDO    el hallazgo cae en un handler con
#                       caps y/o nonce (dictamen de GATES)
#   FP-SQLI-PREPARE      el sink usa $wpdb->prepare con
#                       placeholders (%s/%d) sobre el input
#   FP-SQLI-CAST         el input pasa por absint()/intval()
#   FP-XSS-ESCAPED       la linea del echo aplica esc_*/wp_kses
#   FP-UPLOAD-WHITELIST  la subida valida mime y/o extension
#   FP-STRICT-IN_ARRAY   in_array con strict=true (no juggling)
#   FP-GATE-NOPRIV-LOGIN handler nopriv con is_user_logged_in
#                       o dispatcher que exige sesion dentro
#
#

```

```

# Nivel 1: ventanas de contexto con regex; no ejecuta nada.
# =====

=== fp_elimination.py ===
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
v0.93.0 FP-ELIMINATION
=====
Etapa 21-bis del roadmap: caza SISTEMATICA de falsos
positivos. La red conservadora de veredictos (flaky no
escala, benigno se descarta, muerto no se promedia) ya
elimina la mayoría de los FP por diseno; este modulo los
convierte en GARANTIA verificable:

1. TABLA DE CLASES DE SENAL: cada clase observable del
oraculo tiene su regla de clasificacion y su estatus
reportable. Una senal sin regla = FP potencial: el
modulo NO aprueba.

2. BATERIA DE CEBOS (FP-bait): escenarios que DEBEN
producir veredictos NO reportables. Si algun cebo
produce un veredicto reportable (SECURITY-IMPACT-
DEMO), es un falso positivo: FP-LEAK.

Cebos: quiet_drain (drenaje), benign_pin (conn
pinneada), pool_shift (mispairing benigno cruzado),
flaky_swap (swap 1/2, un solo disparo en la vida).

3. CONTROL POSITIVO: pool_swap DEBE producir
SECURITY-IMPACT-DEMO. Sin control positivo la
bateria es vacua (no puede detectar nada): UNSTABLE,
sin claim.

Veredictos:
FP-ELIMINATED todos los cebos no reportables Y el
control positivo reporta: la garantia
cero-FP queda VERIFICADA en bateria.
FP-LEAK un cebo produjo veredicto reportable:
falso positivo REAL: hay que arreglar.
UNSTABLE lab muerto o control positivo mudo:
no se puede juzgar.

Presupuesto: 5 auditorias x 5 reqs = 25 (techo 26).
Read-only. Labs propios, nada en paralelo.
"""

=== fp_memory.py ===
#!/usr/bin/env python3
# =====
# codexRC - FP-MEMORIA (v0.66.0, AUTHZ-PROOF capa 5)
# -----
# Memoria semantica de falsos positivos: cuando un hallazgo es
# refutado (por el OPERADOR o por la DEFENSA con prueba), se
# guarda su HUELLA ESTRUCTURAL. El siguiente hallazgo con la
# misma huella se auto-cierra sin gastar triaje.
#
# "El falso positivo se paga una sola vez, nunca mas."
#
# Huella (fingerprint) = estructura semantica normalizada del
# hallazgo: tipo, veredicto de gates, rol exigido, nonce, owner,
# sinks de objeto, marcadores del nombre de la accion y del
# codigo alrededor. SIN nombres de archivo, lineas ni plugin:
# dos hallazgos con la misma huella son la MISMA familia, aunque
# vivan en plugins distintos.
#
# Matching EXACTO de huella (sin wildcards): una huella cerrada
# solo cierra huellas identicas. Anti-ruido conservador.
#
# La memoria vive en el repo (.codexrc/intelligence/fp_memory.jsonl)
# y se propaga por git: toda instancia que hace pull aprende lo
# que las demas ya pagaron en triaje.
#
# CLI:
# python3 core/fp_memory.py --stats
# python3 core/fp_memory.py --list [FPM-0003]

```

```
# =====

=== gates_audit.py ===
#!/usr/bin/env python3
# =====
# codexRC - GATES-AUDIT (v0.43.0)
# -----
# Dictamina SOLO si los handlers de un plugin estan protegidos:
# mapea wp_ajax / wp_ajax_nopriv / wc_ajax / REST (permission_callback)
# a su callback, extrae el cuerpo de la funcion y busca compuertas
# (current_user_can / wp_verify_nonce / check_ajax_referer).
#
# Veredictos automaticos:
# CANDIDATO-BAC      nopriv sin caps ni nonce (superficie anonima abierta)
# REVISAR-AUTH       solo wp_ajax (logueado) sin caps ni nonce
#                    (cualquier subscriber puede llegar; vale si toca
#                    objetos sensibles: settings, users, archivos, dinero)
# PROTEGIDO          tiene caps y/o nonce verificado
# REST-ABIERTO       permission_callback __return_true (anonimo)
#
# Uso:
# python3 core/gates_audit.py <dir_plugin> [--json]
# Integrado en PLUGIN-BATCH: anota cada hallazgo con su veredicto.
# Nivel 1: regex + balance de llaves; no ejecuta nada.
# =====

=== ghostgate.py ===
"""
codexRC - Modulo GHOSTGATE (Cloudflare relay, Termux friendly)

CF-Ghost diagnostica. GhostGate accede.

Flujo integrado al login autenticado:
1. El request normal del escaneo llega a una pagina protegida por Cloudflare.
2. Este modulo clasifica la respuesta (JS_CHALLENGE / TURNSTILE / WAF_BLOCK /
BLOQUEO_IP / UNDER_ATTACK / SIN_CF).
3. Si hay challenge y curl_cffi esta instalado, reintenta con la huella TLS
exacta de Chrome 136 (impersonacion de navegador real).
4. Si pasa, entrega las cookies al requests.Session del escaneo.
5. Si no pasa (IP quemado / Turnstile), informa que se necesita GHOSTGATE
completo: delegar el acceso a un navegador real con IP limpia.

Basado en la metodologia GHOSTGATE (repo Eliezer1817/GhostGate).
"""

=== graphql_bait.py ===
"""GRAPHQL: bateria dedicada contra endpoints GraphQL ocultos.

Las SPA modernas esconden toda la logica en /graphql. Esta bateria:

1. DESCUBRE el endpoint (rutas comunes + soporte GET ?query=)
2. INTROSPECCION: si esta habilitada, lista los tipos del esquema:
el mapa completo de la API = mapa de donde buscar IDORs/BAC
3. FIELD SUGGESTIONS: con introspeccion CERRADA, los errores tipo
"Did you mean 'user'?" siguen filtrando el esquema campo por campo
4. BATCHING: detecta si acepta arrays de queries en un solo request
(bypass de rate limits) - se envia UN lote de 2 consultas inertes,
jamás un bombardeo (DoS fuera de reglas)

Todo lectura: introspeccion y sugerencias son metadatos publicos.
"""

=== h2_probe.py ===
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
v0.88.0 H2-TRANSLATION (Parser Differential)
=====
Familia h2.CL / inyeccion H1 por HPACK leniente, con la
disciplina de la linea DESYNC: contratos pre-registrados en
el ExperimentGraph, juez determinista, presupuesto bajo y
cero-FP por diseno.

Mecanica H2-TRANSLATION: el edge traduce HTTP/2 -> HTTP/1.1.
Dos bugs de traduccion clasicos:
h2.CL el edge reenvia TODOS los DATA frames ignorando el
```

```
content-length del origen: el origen consume CL
bytes y procesa el residual como request invisible.
H-INJ el traductor copia verbatim un valor de header con
CRLF: el origen parsea una request extra escondida
en el header block.
```

Oraculo observable (nada inferido): la ATRIBUCION de respuestas por stream. En H2 cada request tiene su stream:

```
A el stream del followup recibe el eco del token del
followup -> traduccion alineada (BENIGN).
B el stream del followup recibe SMUGGLE-ECHO del token
smuggleado -> el origen proceso bytes que el edge no
contabiliza -> MISMATCH observable.
S cadena: el segundo followup recibe el eco del primero
-> la coleccion queda desplazada (STATE EFFECT).
R RST_STREAM/GOAWAY del edge ante smuggle -> frontera
estricta (BENIGN, no desync).
```

Regla heredada (v0.87): el eco del token SOLO no es desync; sin desalineacion observable, el veredicto NO escala. Sin repro 2/2 no hay DESYNC.

Contratos (pre-registrados):

```
baseline: C0 GET control (firma de eco propio);
          C1 POST vacio + followup alineado (si el
          followup no matchea C0: UNSTABLE).
variante (bateria h2.CL o H-INJ, 1 endpoint):
P probe en conexion fresca (POST smuggleado + fb + fc)
R repro idem en conexion nueva
```

Escalera determinista:

```
BENIGN -> H2-DETECTED (1/2, probable, NO reportable)
-> H2-DESYNC (repro 2/2)
-> H2-STATE-EFFECT (ademas coleccion desplazada)
(+ BENIGN-EDGE / UNSTABLE / UNREACHABLE / UNKNOWN)
```

Presupuesto: 2 baseline + 1 variante x 2 fases x 3 reqs = max 8 requests. Read-only: el smuggle es un GET propio con token unico, jamas recursos de terceros.

Aprendizaje RC-000217: los payloads son BYTES crudos jamas repr(b'..'); la carga HPACK va en 0x00-literal sin huffman.

```
=== hunt_wide.py ===
#!/usr/bin/env python3
"""v0.48.0 WIDE-HUNT: caza sobre el corpus COMPLETO wordpress.org (>=5k installs).
```

La mejora EVIDENCE-CHAIN (v0.47.0) hace que los falsos positivos se cierren solo, asi que el ruido ya no justifica limitar la caza a los VDP de Patchstack. Nueva regla:

- CAZAMOS a todos (corpus ancho, refrescable con wide_corpus.py).
- REPORTAMOS donde pagan (VDP activo con bounty, mapa vdp_mapa.json); sin VDP = solo CVE credit, el hallazgo se guarda igual.
- Los ya auditados (hechos/hunt_wide_done.txt) se saltan para no repetir.

CLI:

```
python3 core/hunt_wide.py --dias 90 --workers 6
python3 core/hunt_wide.py --solo-pagables # solo VDP con bounty
```

```
=== hunter.py ===
"""
```

codexRC - Hunter (FACHADA, v0.57.5)

El modulo original de 96KB se partio en submodulos por responsabilidad:

```
core/hunter_base.py  constantes + helpers compartidos
core/hunter_spider.py Spider (mapeo)
core/hunter_deep.py  DeepHunter (caza profunda)
core/hunter_xss.py   XSSHunter (reflexion)
core/hunter_blind.py BlindXSS
core/hunter_xsspro.py XSSPro
```

Esta fachada re-exporta TODO para que los imports existentes (backend/app.py, core/async_lane.py, CLI) no cambien ni una linea.

```
=== hunter_base.py ===
```



```
protegido que jamas pidio)
ECHO <otro-tok> -> MISPAIRED-BENIGN (eco del token
del ATACANTE: mispairing cruzado
pero contenido benigno)
OK -> MISPAIRED-BENIGN (la respuesta del
POST del atacante llego a la
victima)
```

Escalera determinista (nunca se salta, cero-FP):
BENIGN -> IMPACT-CANDIDATE (mispairing benigno, o swap
protegido 1/2: probable, NO reportable)
-> SECURITY-IMPACT-DEMO (swap protegido 2/2 rondas Y
control limpio: la victima recibe a demanda la
respuesta protegida del smuggle del atacante)
(+ BENIGN-EDGE / UNREACHABLE / UNSTABLE / UNKNOWN)

Auto-control por atribucion: la fase CONTROL (victima
limpia antes de envenenar) valida que el apareamiento
funciona con la cola alineada. Sin control no hay claim.

Presupuesto: 1 control + 2 rondas x (1 POST atacante +
1 GET victima) = 5 requests max. Read-only: los smuggles
son GETs al propio lab; el recurso protegido
(/admin/secret) es dato SIMULADO del lab, jamas un
recurso de terceros.

Aprendizajes v0.87-91: bytes crudos jamas repr(b'..');
buffers PERSISTENTES por conexion; el obuf viaja CON la
conn del pool; conn muerta = error honesto, no crash;
nada en paralelo.

"""

```
=== lab_ab_site.py ===
#!/usr/bin/env python3
# =====
# codexRC - LAB-AB-SITE (v0.68.0): sitio NO-WordPress de prueba
# -----
# Laboratorio local para validar REG-BOT + AB-DIFF sin tocar a
# ningun tercero (misma filosofia que WP-LAB, pero para apps
# genericas). Simula una app "Laravel-like" con:
# - registro (username, email, password estricta, confirm)
# - verificacion por CODIGO (6 digitos) al mailbox mock
# - login por email+password
# - /api/me: datos de la sesion (id, email, links)
# - /api/user/<id>: SIN owner check <- bug IDOR a detectar
# - /api/user/<id>/notes: CON owner check <- caso a REFUTAR
# - /mockmail/<email>: buzon mock (compatible mail.tm shape)
#
# Uso:
# python3 core/lab_ab_site.py 8899 # levanta en :8899
# python3 core/ab_diff.py http://127.0.0.1:8899 \
# --mock http://127.0.0.1:8899
# =====
```

```
=== magic_confusion.py ===
#!/usr/bin/env python3
"""MAGIC-CONFUSION (v0.96.0): extension vs contenido en rutas de media.
```

CVE-2026-65640 (WordPress Core <= 7.0.3, Author+ RCE 9.1): WP confia
en la EXTENSION del archivo; ImageMagick decide por MAGIC BYTES.
Un "png" que en realidad contiene PostScript (%!, \x04%!,
\xC5\xD0\xD3\xC6, prefijo de formato tipo EPS:file.png) pasa los
checks de extension, llega a Imagick -> Ghostscript -> ejecucion.
Core parcheo 7.0.4 (commit 7daaa50) snifeando contenido ANTES de
construir el objeto Imagick. Los PLUGINS que procesan media por su
cuenta (galerias, optimizadores, importadores, cover-art) siguen
duplicando el patron.

Escalera cero-FP:

```
CONFUSION-CANDIDATE sink Imagick (readImage/readImageBlob/ping/
new Imagick) alimentado por variable que nace
de una PRIMITIVA DE ENTRADA: $_FILES,
wp_upload_bits, wp_handle_upload,
media_handle_upload, move_uploaded_file,
download_url, media_sideload, wp.downloadFile
CONFUSION-UNGUARDED el archivo del sink NO presenta gate de
```

```

                                contenido: wp_check_filetype_and_ext, finfo,
                                getimagesize, wp_get_image_mime, exif_imagetype
CONFUSION-PREFIX               ademas, el path/file llega con posible
                                prefijo de formato (variable interpolada
                                antes de ':' o file.name sin sanear):
                                riesgo de forzar el coder (EPS:evil.png)

```

Veredicto final: MAGIC-CONFUSION solo con CANDIDATE + UNGUARDED.
Demo real queda en WP-LAB (payload PostScript INERTE: demostrar el routing, nunca Ghostscript ejecutable en blanco de terceros).

Uso:

```
python3 core/magic_confusion.py <dir_plugin> [--json]
"""

```

```

=== mc_probe.py ===
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""

```

```

v0.89.0 MULTI-CONNECTION (Efecto de Segundo Orden)
=====
Contaminacion CRUZADA entre conexiones via pool de
conexiones origin, con la disciplina de la linea DESYNC:
contratos pre-registrados en el ExperimentGraph, juez
determinista, presupuesto bajo y cero-FP por diseno.

```

Mecanica MULTI-CONNECTION: el edge mantiene un POOL de conexiones keep-alive al origin. El atacante (conexion A) hace POST CL.0 con prefijo smuggleado y CIERRA: la origin conn envenenada vuelve al pool con respuestas pendientes. La victima (conexion B, TCP DISTINTO) hereda la conn del pool: su primer followup recibe el eco del smuggle de A.

Oraculo observable (nada inferido): la ATRIBUCION de respuestas por CONEXION:

```

A B recibe SU eco en posicion 0      -> alineado.
S B recibe SMUGGLE-ECHO del token de A -> el origin
  proceso bytes de OTRA conexion     -> MISMATCH
  observable cruzado.
D ademas el eco propio de B aparece DESPLAZADO (pos>0)
                                -> la coleccion
  queda desplazada (STATE EFFECT).
R RST del edge al smuggle          -> frontera
  estricta (BENIGN, no desync).

```

Regla heredada (v0.87/88): el eco del token SOLO no es desync; sin desplazamiento observable, el veredicto NO escala. Sin repro 2/2 no hay DESYNC.

Auto-control por atribucion (sin baseline separado): el eco propio del followup en posicion 0 es el control de cada fase; si no aparece ni eco propio ni eco del smuggle: UNKNOWN (no BENIGN, cero-FP).

Contratos (pre-registrados):

```

fase P  conexion A (POST smuggleado + close) y
        conexion B (3 GETs: fb, fc, fd)
fase R  repro idem con conexiones nuevas

```

Escalera determinista:

```

BENIGN -> MC-DETECTED (1/2, probable, NO reportable)
-> MC-DESYNC (repro 2/2)
-> MC-STATE-EFFECT (ademas coleccion desplazada)
(+ BENIGN-EDGE / UNSTABLE / UNREACHABLE / UNKNOWN)

```

Presupuesto: 1 variante x 2 fases x 4 reqs

(A: 1 POST smuggleado; B: 3 GETs) = max 8 requests.

Read-only: el smuggle es un GET propio con token unico, jamas recursos de terceros.

Aprendizajes v0.88 aplicados: buffers PERSISTENTES por conexion (los bytes nunca se descartan); payloads BYTES crudos jamas repr(b'..').

```

"""

```

```

=== normalization_audit.py ===

```

```
#!/usr/bin/env python3
"""NORMALIZATION-AUDIT (v0.76.0): que representacion recibe el
origin cuando el edge ACEPTA un framing contradictorio.

Pregunta: cuando el edge acepta una variante, ¿que representacion
termina recibiendo el origin?
```

REGLA DE ORO: una tolerancia del edge NO es una vulnerabilidad por si misma. Este modulo describe TRANSFORMACIONES OBSERVABLES; no convierte una transformacion en hallazgo automaticamente. Lo no observable queda unknown (nunca "probablemente").

```
Sonda central: BODY-DIFFERENTIAL (benigna). Cuerpo
5\\r\\nhello\\r\\n0\\r\\n\\r\\n
- leido como TE/chunked -> cuerpo "hello" (5 bytes)
- leido como CL          -> 15 bytes crudos
Las DOS lecturas consumen el mensaje completo: no hay smuggle,
no hay leftover, no hay request secundario. Solo dos
interpretaciones posibles del mismo mensaje, contrastables contra
dos canonicos limpios enviados aparte:
canonical A: POST CL con cuerpo "hello" (normalizador TE->CL)
canonical B: POST CL con cuerpo crudo (conservador)
```

```
Veredictos por variante (todos exigidos por evidencia):
NORMALIZED input != downstream PERO ECO muestra TE eliminado
CONSERVED   input ~= downstream, ECO muestra TE intacto y la
            respuesta coincide con el canonico crudo
MISMATCH    ECO muestra que el origin ACTUO sobre la lectura TE
            (te=chunked) mientras la respuesta coincide con el
            canonico A: dos capas, dos interpretaciones
REJECTED    el probe diferencial recibio 400/cierre
UNKNOWN     sin eco y sin diferencia observable de respuesta
```

Escalera MISMATCH (nada escala sin evidencia):
¿reproducible? -> ¿afecta estado/conexion? -> ¿evidencia
downstream (ECO)? -> SOSPECHA. DEMO queda para las capas de
estado/cache del roadmap (v0.77+).

Presupuesto: perfil v0.75 (11 req) + 3 req por variante aceptada
(probe + 2 canonicos), secuencial, lectura A->B. Tope 24.

```
=== observe.py ===
"""CODEX-OBSERVE (Fase 1, alcance reducido).
```

Registro minimo de eventos semanticos de EVIDENCE-CHAIN: a que decision
llego el motor, con que evidencia, en que version/commit. Fase 1 de la
especificacion CODEX-OBSERVE/INTEL/REGRESS del operador. NO se implementan
las fases de motor de anomalias, differential/metamorphic/fuzz testing ni
CODEX-INTEL: para una herramienta de caza de bugs pagables el ROI de esa
infraestructura es bajo comparado con el tiempo de construirla; lo que
protege plata real es el Regression Corpus (core/regress.py), no el
logging exhaustivo. Si en el futuro aparecen inconsistencias reales de
verdict entre corridas, esto se puede ampliar.

```
=== path_bait.py ===
"""PATH-BAIT: bateria dedicada de LFI / path traversal (PAGA en Patchstack).
```

Mas profunda que la version dentro de STEALTH-BAIT:

- wrappers: traversal directo, dobles puntos, barras invertidas (Windows), null byte, php://filter (base64) para wp-config
- firmas: /etc/passwd, wp-config.php, .env, boot.ini, id_rsa
- objetivos: params con nombre de archivo/ruta/plantilla Y ademas cualquier objetivo de alto valor (score >= 8) con sondas genericas
- 100% lectura: archivos de SISTEMA como prueba, cero ejecucion

```
=== pattern_match.py ===
"""CVE-MATCH: aprendizaje de patrones globales sobre codigo PHP.
```

Compara el codigo de un plugin contra PATRONES ABSTRACTOS destilados de vulnerabilidades historicas reales (CVEs verificadas + 0-days propios de CZ-HUNT). No usa firmas fijas: cada patron es una COMBINACION de senales estructurales debiles que juntas forman el fallo conocido:

ej. BAC ajax sin auth = hook nopriv (+3) + escritura (+3) + input de usuario (+2) - proteccion presente (-4) + nonce impreso al frontend (+2, cancela el veto del nonce)

Si el puntaje pasa el umbral, el hallazgo cita la familia:
"coincide con la familia del 0-day AFFI (CZ-HUNT 2026) / CVE-2023-6875".

LINEA BASE GLOBAL (miles de funciones): con --baseline se mide que porcentaje de funciones similares del corpus (cientos de plugins descargados) SI tiene la proteccion. Un hallazgo anota:
"outlier: 92% de los handlers parecidos del corpus protegen esto".
Comparar contra la poblacion es lo que separa un patron de un ruido.

Patrones incluidos (familias con referencia real):
bac-ajax-nopriv : BAC/PrivEsc en ajax nopriv (0-day AFFI, VillaTheme)
loose-auth-cmp : comparacion floja en control de acceso
(CVE-2023-6875 Post SMTP, type juggling)
ipn-downgrade : verificacion de pago degradable a sandbox
(CVE-2026-9242 RegistrationMagic, vector propio)
ssrf-from-request: SSRF con URL del usuario (familia SSRF plugins WP)
upload-unchecked : subida de archivos sin validar tipo
(familia upload arbitrario, Patchstack paga)
role-from-request: rol/capacidad escrita desde request (PrivEsc)

Uso:
python3 core/pattern_match.py <archivo_o_dir> [--top N] [--json]
python3 core/pattern_match.py <corpus> --baseline # construye stats
POST /api/patterns {"path": "..."} (backend)

=== pipeline.py ===

codexRC - Pipeline Orchestrator
Controls the flow between modules (nodes).

=== plugin_batch.py ===
#!/usr/bin/env python3
=====
codexRC - PLUGIN-BATCH (v0.42.0)

Modo agente: un solo comando recibe slugs de plugins
WordPress, descarga la ultima version estable, y corre el
arsenal ESTATICO del Hunter (TAINT-TRACE + CVE-MATCH),
con capa VERIFICACION: filtra ruido de librerias y deja
solo hallazgos en codigo propio del plugin.

Uso:
python3 core/plugin_batch.py slug1 slug2 ...
echo "slug1\nslug2" | python3 core/plugin_batch.py
python3 core/plugin_batch.py --vdp cz_hunt/vdp/vdp_matches.json
python3 core/plugin_batch.py slugs.txt --out resultados.json

Salida: resumen en consola ([HALLAZGO] = criticos) + JSON completo.
Nivel de confianza por hallazgo: HIGH (taint confirmado en
codigo propio) / MED (patron con outlier del baseline) /
LOW (ruido ya filtrado, no se reporta).
=====

=== poi_reach.py ===
#!/usr/bin/env python3
"""POI-REACH (v0.96.1): Object Injection con alcance y gadget chain.

CVE-2026-2599 (Contact Form Entries): download_csv() hacia unserialize()
de input del usuario, sin allowed_classes, en handler anonimo -> RCE
9.8. La clase sigue pagando (\$600-\$2.600 segun installs, tabla
Patchstack). Lo que falta en los tools abiertos no es detectar
unserialize: es probar ALCANCE y EXPLOTABILIDAD.

Escalera cero-FP (cada peldano exige el anterior):
POI-DETECTED unserialize recibe taint del usuario sin sanitizador
(hereda el taint estatico de TAIN-TRACE, nivel 1)
POI-UNAUTH el sink vive en handler dictaminado por GATES-AUDIT
como superficie anonima (CANDIDATO-BAC o REST-ABIERTO)
o el archivo registra hooks nopriv con el sink dentro
POI-CHAIN POP-CANDIDATE en el mismo plugin: clase con metodo

```
magico (__destruct / __wakeup / __toString / __call /
__get) con propiedades = gadget alcanzable
POI-REACH    las tres: candidato reportable (Demo en WP-LAB con
chain inerte queda pendiente, nunca en vivo)
```

Superficie priorizada (leccion CVE-2026-2599): handlers y funciones de export / download / csv reciben parametros serializados y casi nunca pasan por gates: se anotan como nota de prioridad, no como veredicto.

Sin alcance anonimo el veredicto honesto es POI-AUTH (logged): Patchstack acepta roles bajos, pero el dictamen es del operador, no del motor.

Uso:

```
python3 core/poi_reach.py <dir_plugin> [--json]
"""
```

```
=== race_proof.py ===
#!/usr/bin/env python3
"""RACE-PROOF (v0.71.0): ejecutor dinamico de carreras TOCTOU.
```

El lado estatico (RACE-TRACE v0.70.0) encuentra leer->decir->escribir sin lock. Este modulo es el brazo: dispara N requests simultaneos contra el endpoint vulnerable y dictamina si el estado cambio mas de lo permitido.

Modos:

- H2-SINGLE-PACKET: una sola conexion HTTP/2, todos los HEADERS en un unico paquete TCP (la tecnica de James Kettle). Requiere libreria pura `h2` (pip install h2) y que el server hable h2.
- BARRERA-HTTP11: N conexiones pre-abiertas que disparan todas al pasar la barrera (fallback universal, sirve en Termux).

Veredictos:

- RACE-DEMO: exitos > permitidos (la carrera gana)
- SIN-RACE: el server resistio (o la ventana no se dejo caer)

Uso:

```
python3 core/race_proof.py --url http://host/wp-admin/admin-ajax.php \
--data "action=apply_coupon&c=BLACK" --cookie "wordpress_logged_in=x" \
--count 20 --success-contains "cupon aplicado" --allowed 1
"""
```

```
=== race_trace.py ===
#!/usr/bin/env python3
# =====
# codexRC - RACE-TRACE (v0.70.0)
# -----
# Detecta TOCTOU (check-then-act) en estado persistente de
# plugins WP: leer -> decidir -> escribir sin lock. Es la clase
# de bug que gana carreras con single-packet HTTP/2 y que casi
# ningun plugin WP aguanta (limites, cupones, stock, saldos).
#
# Filosofia del engine: NIVEL 1 estatico puro (regex + balance
# de llaves), nada se ejecuta, cero dependencias (Termux OK).
#
# Lo que busca, por funcion:
# 1) READ de estado persistente: get_option/get_user_meta/
#    get_post_meta/get_transient/$wpdb->get_(var|row) SELECT
#    ->get_stock_quantity/->get_usage_count/->get_total/
#    ->get_balance/->get_meta(...)
# 2) WRITE de estado persistente: update_option/update_user_meta/
#    update_post_meta/$wpdb->(query|update|insert)/
#    ->set_stock_quantity/->increase_usage_count/...
# 3) EMPAREJA read->write sobre la MISMA clave o variable
#    (flujo lite: $var asignada en el read y usada en el write
#    o en el valor computado del write).
# 4) VENTANA: entre read y write hay un if/guard con la
#    variable => check-then-act clasico.
#
# Veredictos:
# CANDIDATO-RACE    read->write emparejado en estado sensible
#                   (dinero/stock/cupon/limite/credito) sin
#                   mitigacion visible en la funcion
# RACE-ATOMICO      el write es SQL incremento sobre columna
#                   (col = col + 1) => atomico, descartado
# MITIGADO-TRANSIENT lock con get_transient/set_transient en
```

```

#           el flujo => mitigacion de facto WP, blando
#   MITIGADO-LOCK       flock / LOCK_EX / get_lock de MySQL
#   DESCARTADO-LECTURA read y write sin ventana ni variable comun
#
# Severidad del candidato:
#   critica handler nopriv (carrera ANONIMA via ajax)
#   alta     estado de dinero/stock/cupon/billetera
#   media    contador/limite/intentos (enumeracion o bypass)
#
# Uso:
#   python3 core/race_trace.py <dir_plugin> [--json]
# Integrado en PLUGIN-BATCH: rec["race_candidatos"] + anota
# hallazgos existentes con "_race" si comparten funcion.
# =====

=== re_engine.py ===
"""REVERSE: ingenieria inversa nivel 2 (sobre BIN-AUDIT).

Mientras BIN-AUDIT extrae la superficie (secrets, endpoints, imports),
REVERSE abre los formatos y lee la ESTRUCTURA interna del artefacto:

DEX (APK/Android)  parse REAL del formato DEX: string_ids, type_ids,
method_ids y class_defs con uleb128. Lista clases, metodos y todas
las strings del bytecode -> se les pasa el detector de secrets y
endpoints de BIN-AUDIT. Detecta app OFUSCADA (nombres a/b/c estilo
ProGuard/R8) y metodos con keywords de logica sensible
(decrypt/license/sign/verify/token).

CLASS (JAR/Java)   parse del constant pool del .class (CAFEBAFE):
nombre de la clase, superclase y todas las constantes Utf8 ->
secrets y referencias internas.

ELF (nativo)       tabla de simbolos INTERNA (.symtab, no solo
.dynsym): nombres de funciones estaticas que revelan como se
organiza la logica (verify_*, decrypt_*, handle_*). Detecta binario
STRIPPED, secciones .debug_* y ENTROPIA por seccion: una seccion
ejecutable con entropia > 7.0 = empaquetado/ofuscado (UPX-like).

Nivel honesto: parse de formatos + simbolos + entropia. Desensamblado
de instrucciones queda como etapa futura (necesita binutils en el
dispositivo); esto ya revela la logica sin ejecutar nada.

Uso:
python3 core/re_engine.py <archivo_o_dir> [--json] [--top N]
POST /api/re {"path": "..."}                                (backend)
"""

=== recon.py ===
"""
codexRC - Basic Recon Module
Collects headers, status, redirects, cookies, security headers and basic info.
"""

=== reg_bot.py ===
#!/usr/bin/env python3
# =====
# codexRC - REG-BOT (v0.68.0, AB-DIFF universal)
# -----
# PROVEEDOR UNIVERSAL DE IDENTIDADES/SESIONES DE PRUEBA.
# Desacoplado de WordPress: AUTHZ-PROOF consume objetos
# Identity estandarizados y jamas sabe si la cuenta nacio en
# WordPress, Laravel, Django, Express o una app propia.
#
# Submodulos (separacion de responsabilidades):
#   FORM-DISCOVERY       lee el formulario (no lo adivina)
#   CONSTRAINT-SOLVER    resuelve restricciones del servidor
#   IDENTITY-GENERATOR    identidad plausible y unica por sitio
#   REGISTRATION-FLOW     registro multi-paso con reintentos
#   VERIFICATION-FLOW     codigo numerico 0 enlace magico (polling corto)
#   SESSION-HANDLER       captura cookies/csrf/tokens
#   RECIPE-MEMORY         memoria de receta por sitio
#
# Objeto Identity estandarizado:
#   Identity = {
#       "id", "credentials", "cookies", "csrf", "tokens",
#       "verification_state", "registration_recipe", "capabilities"

```

```

# }
#
# Relaciones para A/B (lo que AUTHZ-PROOF realmente necesita):
#   A = propietario del objeto X
#   B = usuario independiente
#   A->X permitido, B->X permitido = evidencia de BAC/IDOR
#
# Reglas de seguridad (permanentes):
#   - pide telefono/documentos/KYC -> DESCARTAR el blanco
#   - CAPTCHA que GHOSTGATE no pasa -> CAPTCHA-PENDING (cola de
#     handoff al operador, NUNCA un sistema para vencer CAPTCHA)
#   - el mailbox ninja es mail.tm (API, sin telefono); en labor
#     se usa MockMailProvider local
#
# Identidades y recetas viven en CODEXRC_HOME (estado privado
# del operador, NO se comparten por git: contienen credenciales).
#
# CLI:
#   python3 core/reg_bot.py https://sitio.com          # registrar 1 ninja
#   python3 core/reg_bot.py https://sitio.com --n 2    # A y B
#   python3 core/reg_bot.py https://sitio.com --recipes # recetas guardadas
# =====

=== regress.py ===
"""CODEX-REGRESS (alcance reducido: Regression Corpus + ejecucion).

Cada defecto REAL confirmado y corregido se guarda como caso reproducible
(sec. 9 de la especificacion). No se eliminan casos aunque pasen: su valor
es impedir que el defecto vuelva. Se omiten differential/metamorphic/
property-based/fuzz testing (fases 5-6-8) por ahora: para una herramienta
de caza orientada a bugs pagables, el regression corpus de defectos reales
ya detectados es lo que protege plata; el resto es inversion de ingenieria
sin payoff claro hoy.

Uso:
    python3 core/regress.py          # corre todos los casos, PASS/FAIL
    python3 core/regress.py --add ... # (manual, ver _CASES)
"""

=== report_export.py ===
"""Reportes exportables del Hunter: TXT, JSON y PDF.

PDF de alta calidad: se arma un documento HTML+CSS y se imprime con el
navegador real en modo headless (Chrome/Chromium/Edge, la misma deteccion
multiplataforma que usa VERITAS). Si no hay navegador disponible, fallback
a fpdf2 (python puro, `pip install fpdf2`, funciona en Termux/armv7l).

Orden del informe (regla del operador): filtraciones de datos de usuarios
ARRIBA y en rojo; hallazgos tecnicos abajo por severidad.
"""

=== repro_engine.py ===
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""REPRODUCTION ENGINE v0.91 – etapa 21 del roadmap
(ultima etapa de MODULO antes de IMPACT / FP-ELIMINATION
/ EVIDENCE PACKAGE).

Escalera de impacto (nunca se salta):
DESYNC OBSERVED -> REPRODUCIBLE (2 genealogias
independientes) -> STATE EFFECT -> CROSS-CONNECTION
-> SECURITY IMPACT.

Este modulo toma un candidato STATE-EFFECT y responde
UNA pregunta: la firma se reproduce a demanda?

Genealogia = estado del servidor INDEPENDIENTE:
* modo lab: instancia FRESCA de labs/crc_lab.py
  (proceso y pool propios, puerto distinto).
* modo vivo: secuencia completa de conexiones nuevas
  contra el blanco (sin estado compartido del lado
  del cliente).

Cada genealogia ejecuta DOS pases del probe subyacente
(por defecto crc_probe, bateria double):
* pase A (deteccion): la corrida original.

```

```

* pase B (replay): MISMA genealogia, corrida nueva
que debe volver a mostrar la firma. Un disparo
unico (flaky) NO escala: NON-REPRODUCIBLE.

Veredictos (conservadores, cero-FP):
REPRODUCIBLE      todas las genealogias muestran
                   STATE-EFFECT con la MISMA firma
                   (k) en pase A y pase B.
NON-REPRODUCIBLE  hubo senal (DETECTED/STATE-EFFECT)
                   en algun pase pero no se sostiene
                   en A+B de todas las genealogias.
                   El candidato NO avanza.
NO-CANDIDATE      todos los pases BENIGN/BENIGN-EDGE:
                   reproducible pero no es nada.
UNSTABLE          instancia muerta o control caido:
                   no se puede juzgar. Sin claim.

Presupuesto: genealogias (2) x pases (2) x 8 reqs
= 32 requests max. Read-only. Python puro.
"""

=== research_memory.py ===
"""RESEARCH-MEMORY (v0.81.1): memoria de investigacion por
target.

Guarda por host, en CODEXRC_HOME/memory/research/<host>/:
dossier.jsonl  una linea por ventana (sesion) observada
latest.json    la ultima ventana completa

REGLAS DE HONESTIDAD:
1. Solo observaciones: se guarda lo visto, lo ejecutado y lo
descartado CON SU NOTA. Nunca inferencias sin contrato.
2. Las conclusiones CADUCAN POR VENTANA: una hipotesis
contradicha ayer no revive sola hoy, pero tampoco es
verdad eterna. Al re-entrar, el grafo se siembra fresco y
lo previo entra como ANOTACION de ventana.
3. El UNICO reuso automatico es con evidencia fuerte:
baseline IDENTICO (mismas huellas) dentro del TTL. Mismo
observable = mismas condiciones observadas; distinto
baseline = ventana nueva, corrida nueva.
"""

=== retro_hunt.py ===
#!/usr/bin/env python3
"""
RETRO-HUNT (v0.51.0) – CodexRC

Caza el codigo PRE-COOLDOWN: plugins >=5k installs cuya ultima
actualizacion es ANTERIOR al gate de revision IA de WordPress.org
(junio 2026). Ese codigo distribuido nunca paso por el escaneo
automatico de WP.org ni por Jetpack Scan de cola, asi que es el
terreno con menos filtros encima.

La auditoria es FULL-CODE (no diff): todo el plugin se taintea,
no solo lo nuevo.

Uso:
python3 core/retro_hunt.py          # cola completa pre-cooldown
python3 core/retro_hunt.py --limit 20      # prueba chica
python3 core/retro_hunt.py --workers 6
python3 core/retro_hunt.py --desde 2026-06-01 # umbral cooldown

Salida:
hechos/retro_results.json.jsonl  (hallazgos por plugin)
hechos/retro_done.txt            (auditados, incremental crash-safe)
"""

=== saml_audit.py ===
#!/usr/bin/env python3
"""SAML-DEFENSE (v0.71.0): auditor SAML/XML de PHP.

Los bypasses novedosos de 2025 ("The Fragile Lock", PortSwigger)
golpean implementaciones SAML: XML Signature Wrapping, entidad
externa (XXE), XInclude y validaciones de firma flojas.

Detecta estaticamente en PHP:

```

- XXE: parseo de XML con input de usuario SIN flags LIBXML_NONET
- Mitigaciones: LIBXML_NONET / entity_loader desactivado
- SAML-WSW: processResponse() SIN chequeo posterior de getErrors()/isValid()/isAuthenticated() en el flujo
- SAML-STRICT: settings con 'strict' => false
- XInclude: ->xinclude() sobre DOM con input usuario

Uso:

```
python3 core/saml_audit.py <root>
```

```
"""
```

```
=== self_tune.py ===
```

```
"""AUTOCORRECCION: el Hunter recuerda que rindio y ajusta su propio gasto.
```

Memoria persistente (tune_state.json, NO se committea):

- por PARAMETRO: cuantas veces ese nombre de parametro produjo hallazgos en cazas anteriores (user_id, cat, q, id...).
- por BLANCO: fingerprint del sitio (WordPress, Angular...) -> si un CMS/Framework rinde historico, se nota.

Uso:

- load_boosts() ANTES de rankear: multiplica el presupuesto de sondas de los parametros con historial exitoso (x1.5 con 2+ hallazgos, x1.2 con 1). Determinista: mismas estadisticas, misma decision.
- record() AL FINAL de cada caza: cuenta los hallazgos por param y por tipo de blanco. La caza N aprende de las N-1.

Anti-sesgo: el boost nunca baja el presupuesto de otros parametros; solo sube el de los probados, con tope. Un parametro nuevo empieza neutro.

```
"""
```

```
=== semantic_channels.py ===
```

```
"""Canales de observacion de SEMANTIC-CACHE (v0.80).
```

Regla del diseno aprobado: dos canales NO son

"independientes" solo porque tienen nombres diferentes.

independence_status:

- VERIFIED solo con evidencia de dispositivos distintos (topologia). v0.80 NO la produce.
- PARTIAL canal_type distinto (mecanismo de observacion distinto) aunque el dispositivo no sea verificable.
- UNKNOWN mismo canal_type, o sin poder establecer nada.

Ningun canal afirma estados internos: cada observacion cita que se midio y como.

```
"""
```

```
=== semantic_core.py ===
```

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
"""SEMANTIC CORE (v0.58.0) – Nivel 0: identidad canonica de recursos.
```

Fundamento del nivel semantico del motor. Un hallazgo solo puede alcanzar DEMOSTRADO-ESTATICO si la evidencia proviene del archivo correcto, verificado por hash de contenido.

Identidad canonica de un archivo (FileId):

- root canonico (absoluto, normalizado)
- + ruta relativa normalizada (separador '/', sin '..')

El basename NO es identidad. Si la resolucion solo puede hacerse por basename, el resultado es AMBIGUO y bloquea el veredicto alto.

Cadena de integridad:

- Finding -> FileId -> ContentHash -> contenido analizado
- Si el hash del contenido analizado != hash esperado:
- INTEGRITY FAILURE -> el finding no puede pasar a DEMOSTRADO-ESTATICO.

Python puro, Termux/armv7l OK.

```
"""
```

```
=== semantic_dimension.py ===
```

```
"""Fase 2 de SEMANTIC-CACHE: localizacion de dimension y
```

descarte de SELF-INDUCED STATE (v0.80).

Solo corre si la fase 1 (v0.79) encontro una inconsistencia observable SIN explicacion legitima. Presupuesto duro: fase2 <= 8, controles <= 4.

Self-induced (regla del diseno aprobado): ninguna observacion producida por una sonda se convierte en evidencia de causalidad si no podemos descartar razonablemente que la propia sonda creo el estado observado. Metodos:

```
VIRGIN_PATH      el patron se reproduce en paths nunca
                  tocados por ninguna sonda
NEGATIVE_CONTROL  sonda que NO debe producir el efecto;
                  si lo produjera -> UNKNOWN
CLEAN_REPRO       repro sin contaminacion previa
MULTI_RUN         patron consistente en >=3 observaciones
"""
```

=== semantic_engine.py ===

"""SEMANTIC-CACHE (v0.80): motor de desacuerdos semanticos.

Flujo:

```
FASE 1  bateria v0.79 INTACTA (17 req) via
        cache_correlation.audit
FASE 2  solo si hay inconsistencia observable SIN
        explicacion legitima (<= 8 req + <= 4 controles)
JUEZ    determinista sobre atributos separados
```

HARD CAP: total <= 30 requests. Si se alcanza el limite sin discriminar: UNKNOWN. No se persigue la hipotesis.

=== semantic_judge.py ===

"""Juez determinista de SEMANTIC-CACHE (v0.80).

Veredicto corto: CONSISTENT / BENIGN / INCONSISTENT / SUSPICIOUS / DEMO / UNKNOWN.

Atributos SEPARADOS del veredicto (nunca mezclados):

```
semantic_state  CONSISTENT | INCONSISTENT
correlation     NONE | OBSERVED | STRONG
impact         NONE | CROSS-CONSUMER | SECURITY
attribution     UNKNOWN | CANDIDATE | SUPPORTED
binding         WEAK | MODERATE | STRONG | UNKNOWN
self_induced    RULED_OUT | UNKNOWN
self_induced_method  VIRGIN_PATH | NEGATIVE_CONTROL |
                    CLEAN_REPRO | MULTI_RUN | UNKNOWN
```

Sin scores, sin confidence, sin sumas: el veredicto se deriva de CONDICIONES sobre atributos.

DEMO exige CONJUNTAMENTE (regla aprobada, ninguna sustituye a otra):

- 1 inconsistencia reproducible
- 2 dimension semantica localizada
- 3 convergencia atribuida (validator 0 provenance)
- 4 controles criticos PASSED
- 5 baseline utilizable
- 6 self_induced RULED_OUT (con metodo citado)
- 7 correlacion/atribucion suficiente (la provenance sola NO sustituye la demostracion de impacto)
- 8 E5 CROSS-CONSUMER o E6 SECURITY reproducible

self_induced=UNKNOWN: no alimenta E5/E6, no produce DEMO, maximo SUSPICIOUS si el resto es fuerte.

=== semantic_models.py ===

"""Modelos de datos de SEMANTIC-CACHE (v0.80).

Regla epistemologica central (diseno aprobado):

Hipotesis != Observacion != Evidencia != Veredicto.

Estos modelos NUNCA representan estados internos del edge/cache/proxy/origin como hechos. Toda propiedad

```

interna no demostrable con observaciones del motor queda
como UNKNOWN o CANDIDATE en su campo dedicado
(attribution), nunca como hecho del reporte.
"""

=== sqli_bait.py ===
"""SQLI-BAIT: deteccion avanzada de inyeccion SQL en la superficie de la app.

4 tiers, del mas barato al mas contundente (solo sobre blancos autorizados):

1. ERROR      rompe sintaxis y captura el error del motor en la respuesta,
               con fingerprint del DBMS (MySQL/MariaDB, PostgreSQL, MSSQL,
               SQLite, Oracle).
2. BOOLEAN    diferencial clasico AND 1=1 vs AND 1=2 contra el baseline
               (respuesta con 1=1 ~ baseline y con 1=2 distinta = inyetable).
3. TIME       SLEEP(4) / pg_sleep(4) / WAITFOR DELAY con confirmacion
               repetida (2 aciertos seguidos contra jitter de red).
4. STACKED    '; SELECT ... -- - detectado por error o timing (apilado).

Tambien sondea cabeceras classicas que terminan en SQL (X-Forwarded-For,
User-Agent, Referer). Todo con presupuesto de sondas por parametro para no
martillar, y con controles anti-falso-positivo (el error debe ser NUEVO
respecto al baseline; el time-based exige repeticion).
"""

=== ssa.py ===
"""SSA-lite: grafo de versiones def-use para Taint-Trace.

Cada asignacion crea una VERSION nueva de la variable (q#1, q#2, ...)
con: kind (SOURCE/SANITIZED/PREPARED/CONCAT/PLAIN), taint, parents
(versiones que aportaron valor) y linea. Un uso resuelve la version
viva mas reciente (asignada en linea <= uso, dentro del scope).

Entrega la CADENA de evidencia de un sink:
    v3 = CONCAT(v2, "%")
    v2 = SANITIZE(v1)
    v1 = SOURCE($_GET['q']) @ L40
en vez de "$q aparece cerca del sink".

Nivel 1 (documentado): intra-funcion, orden lineal (los back-edges de
loops se ignoran: una var reasignada en loop cuenta la version de la
asignacion previa). Closures sin nombre heredan el scope exterior.
"""

=== ssti_bait.py ===
"""SSTI-BAIT: bateria dedicada de inyeccion de plantillas con FINGERPRINT.

Identifica el MOTOR de plantillas, no solo el hecho de que computa:
Jinja2 / Twig      {{7*7}} -> 49          (Python/PHP)
Jinja2 confirmado  {{7*'7'}} -> 7777777 (multiplicacion de string Python)
FreeMarker/Java    ${7*7} -> 49
Velocity           #set($x=7*7)$x -> 49
ERB / EJS (Ruby)   <%= 7*7 %> -> 49
Ruby interp        #{7*7} -> 49
Smarty (PHP)       {math equation="7*7"} -> 49

SSTI = RCE en potencia: paga CRITICO en la tabla. Canarios 100% inertes
(aritmetica 7*7, cero comandos, cero datos ajenos).
"""

=== state.py ===
"""Estado individual por operador/IA.

Un solo repositorio de codigo (compartido via git) puede ser usado por
varias IAs u operadores en paralelo sin pisarse: todo archivo de
ESTADO (avance de caza, ya-auditados, resultados) vive en el
workspace del operador. Por defecto es el propio repo (compatibilidad
total con lo existente); exportando CODEXRC_HOME cada IA tiene el
suyo:

    export CODEXRC_HOME=~/.codexrc_hermes    # cada IA, distinto

    python3 core/hunt_wide.py --shard 0/2    # ademas: repartir corpus
    python3 core/hunt_wide.py --shard 1/2

Regla: el repo guarda CODIGO y datos publicos (data/:
```

vdp_mapa.json, wide_corpus.json); CODEXRC_HOME guarda HECHOS propios (hechos/, resultados). Dos IAs con distintos CODEXRC_HOME no comparten la cola de "ya auditados" ni se duplican hallazgos.

```
=== state_correlation.py ===
#!/usr/bin/env python3
"""STATE-CORRELATION (v0.78.0): un veredicto por target, no por sonda.
```

No agrega payloads. Toma lo que v0.75 (contrato del edge), v0.76 (representacion) y v0.77 (estado de conexion) observan sobre el MISMO target y lo une en una ficha de estado unica.

Cuatro resultados:

STABLE	sin nada observable fuera de varianza
PIPE-BENIGN	desplazamiento observado PERO la conexion inocente quedo limpia: pipelining legitimo del front (edge honra TE, RFC 7230). SELLADO: re-observarlo mil veces jamas escala
STATE-CHANGED	cambio de estado reproducible sin evidencia cross-connection
CROSS-CONNECTION-MISMATCH	la conexion inocente recibio respuesta ajena: desacuerdo real de cadena. UNICO que alimenta la escalera

EVIDENCE QUALITY (E0-E5), anotacion transversal:

- E0 observacion nula
- E1 reproducible
- E2 diferencial
- E3 downstream observado (ECO)
- E4 cross-connection
- E5 impacto de seguridad (respuesta ajena entregada a una conexion inocente, reproducible)

El vector va a un JUEZ determinista, no a una suma: DEMO exige E4 + E5 SIEMPRE (E3 sube confianza, no es requerido). La confianza sube porque llego evidencia independiente que reduce incertidumbre, no porque una anomalia se repitio.

REGLA v0.77 (leccion del falso DEMO): un desplazamiento en conexion unica NO demuestra desync; hay que demostrar contaminacion o desacuerdo entre conexiones independientes.

```
=== stealth_bait.py ===
"""STEALTH-BAIT: caza de superficie oculta para blancos blindados.
```

Los escaneres genericos prueban siempre lo mismo en los mismos sitios. Esta bateria ataca por donde nadie mira, 100% MODO LECTURA (canarios inertes, cero payloads de explotacion, estandar A->B):

1. HIDDEN-PARAMS parametros secretos que el frontend nunca muestra (debug, is_admin, role, internal...): si el backend reacciona a uno, hay logica no publicada a revisar.
2. PATH-BAIT path traversal en params file/path/tpl/lang con firma de deteccion (passwd / wp-config). LFI PAGA en la tabla de Patchstack.
3. SSTI-BAIT canario de plantilla {{7*7}} / \${7*7} / #{7*7}: si la respuesta contiene 49, el servidor COMPUTA (RCE paga).
4. CRLF-BAIT inyeccion de cabeceras: marcador %0d%0a en params reflejados, verificado SOLO en headers de respuesta.
5. METHOD-BAIT method tampering: X-HTTP-Method-Override, _method, y verbos alternativos sobre endpoints de datos.
6. CACHE-BAIT deteccion PASIVA de entradas unkeyed (X-Forwarded-Host reflejado): candidata a cache poisoning, sin envenenar.
7. GRAPHQL introspeccion abierta y endpoints /graphql ocultos.

Todo con presupuesto de sondas por objetivo (respeta el CEREBRO) y anti-FP por comparacion contra baseline.

```
"""
=== taint_trace.py ===
"""TAINT-TRACE: analisis de flujo de datos (taint) sobre codigo fuente PHP.
```

Sigue el valor de un parametro desde la FUENTE (\$_GET/\$_POST/\$_REQUEST/\$_COOKIE/php://input/\$_SERVER) a traves de asignaciones y concatenaciones hasta el SINK peligroso:

```
base de datos : $wpdb->query/get_var/get_row/get_results, mysql_query
navegador     : echo/print/printf (XSS)
objetos       : unserialize (object injection)
archivos      : include/require/file_get_contents/file_put_contents (LFI)
comandos      : system/exec/passthru/eval (RCE)
red           : wp_remote_get/curl (SSRF)
```

Si el sink recibe un valor que NACE del usuario SIN sanitizador en el camino, es hallazgo: la vulnerabilidad existe aunque la respuesta HTTP no muestre nada (SQLi ciego, XSS reflejado en otra pagina, etc).

Sanitizadores que cortan el taint: intval/absint/sanitize_*/esc_sql/esc_attr/esc_html/esc_url/wpkses. \$wpdb->prepare con placeholders y el taint SOLO en los valores (no en la cadena de formato) = SEGURO.

Nivel 1 (heuristico, intra-archivo): analisis por linea con punto fijo hasta que el taint deja de propagarse. Sin dependencias externas, Python puro, sirve en Termux. Para nivel interprocedural sigue siendo mejor Semgrep (SINK-SCAN); este tracer es el complemento vivo.

Uso:

```
python3 core/taint_trace.py <archivo_o_directorio> [--json] [--top N]
POST /api/taint {"path": "..."} (desde el backend)
"""
```

```
=== tech_detect.py ===
"""
codexRC - Basic Technology Detection (html.parser)
"""
```

```
=== universal_engine.py ===
#!/usr/bin/env python3
"""
UNIVERSAL-ENGINE (v0.52.0) – CodexRC
```

Orquestador: descubre que es un blanco, modela su superficie y decide que analizadores activar. Convierte los modulos especializados (BIN-AUDIT, REVERSE, DECOMPILE, TAINT-TRACE, GATES-AUDIT, COV-BAIT, EVIDENCE-CHAIN, WP-LAB) en componentes de un motor comun.

Arquitectura (metodologia OWASP: mapear -> auth -> logica -> impacto):

1. UNIVERSAL-RECON fingerprint del blanco (que tecnologia/es)
2. descubrir superficie (que entradas existen)
3. elegir analizadores segun perfil (no 500 pruebas, las que aplican)
4. correr componentes (estatico + dinamico + navegador)
5. UNIVERSAL-EVIDENCE una sola cadena con FISCAL/DEFENSA/JUEZ
6. ledger de cobertura: ANALIZADO / DESCUBIERTO / NO ACCESIBLE / VERIFICADO

Uso:

```
python3 core/universal_engine.py <path> # fuente (dir/zip/apk/bin)
python3 core/universal_engine.py <url> # app viva (GHOSTGATE si CF)
python3 core/universal_engine.py <path> --json
"""
```

```
=== vdp_fresh.py ===
#!/usr/bin/env python3
"""v0.61.0 VDP-FRESH: cazar los VDP RECIEN AGREGADOS a Patchstack.
```

Palanca de corpus fresco: los plugins que entran NUEVOS al directorio VDP (patchstack.com/database/vdp) aun no fueron revisados por nadie. Este modulo mantiene snapshots del mapa VDP y detecta:

```
ALTAS   slugs con VDP nuevo (blanco de caza inmediata)
BOUNTY  plugins ya conocidos que ahora si pagan (sube prioridad)
BAJAS   VDP retirado (dejar de cazar para reportar)
```

```
CLI:
python3 core/vdp_fresh.py --snapshot          # guarda linea base
python3 core/vdp_fresh.py --diff data/vdp_mapa.json
        compara el mapa NUEVO contra el snapshot -> vdp_nuevos.json
python3 core/vdp_fresh.py --estado           # resumen del snapshot
```

Flujo semanal: extraer mapa (navegador, GOLDEN LIST) -> --diff -> vdp_nuevos.json -> hunt_wide --vdp-nuevos (prioridad maxima).

```
"""
=== vdp_watcher.py ===
"""VDP-FRESH WATCHER (v0.62.6).
```

Detecta plugins PAGABLES (VDP con bounty) que sacaron version nueva desde nuestra ultima auditoria y los manda a DIFF-HUNT solos: el parche fresco es el mejor momento para cazar parches incompletos.

Lo despierta un workflow cada 6 horas; el script es determinista (Python puro, sin LLM) y el agente solo triaga al final.

```
Uso:
python3 core/vdp_watcher.py          # detecta y caza los que movieron
python3 core/vdp_watcher.py --check  # solo listar, sin cazar
python3 core/vdp_watcher.py --limit 20 # prueba rapida (primeros N pagables)
"""
```

```
=== vendor_farm.py ===
"""VENDOR-FARM: descubre familias de plugins de UN MISMO vendor con
instalaciones en rango pagable, para farmear el vendor completo (el
patron que dio el 0-day de AFFI en VillaTheme).
```

- Consulta la API publica de wordpress.org (HTTPS, python puro, Termux OK)
- Agrupa por vendor (autor) y filtra familias con N+ plugins en el rango
- Excluye slugs ya auditados (pasa el directorio del corpus o una lista)
- Estima la paga por plugin con la tabla Patchstack (BTC/ETH) y marca el precio del hallazgo mas valioso de la familia

```
Uso (CLI):
python3 core/vendor_farm.py --tag woocommerce --pages 5 \
    --min 10000 --max 200000 --family 2 --exclude-dir cz_hunt
python3 core/vendor_farm.py --browse popular --pages 8
```

```
API del server:
POST /api/vendor_farm {"tag": "woocommerce", "pages": 5,
                      "min": 10000, "max": 200000,
                      "family_min": 2, "exclude_slugs": [...]}
"""
```

```
=== veritas.py ===
"""VERITAS: verificacion de XSS con navegador real (Chrome headless).
```

La reflexion puede mentir: el servidor refleja el marcador, pero el navegador real lo sanea, lo escapa, lo mata el CSP o nunca llega a un sink ejecutable. VERITAS no cree en reflexiones: re-lanza cada candidato con un CANARIO INERTE (solo cambia document.title, nada visible, nada que exfiltre, no modifica datos) dentro de Chrome headless y mira si el DOM final contiene el canario en <title>. Si ejecuto -> confirmado. Si no -> era un falso positivo y baja de gravedad.

Herramienta general de auditoria con autorizacion previa del operador."""

```
=== vision_gate.py ===
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""VISION-GATE (v0.69.0): clasificador visual de challenges via Gemini.
```

Filosofia (mismas reglas del JUEZ): el LLM clasifica, NUNCA conduce. Vision-Gate recibe screenshot opcional + fragmento de DOM redactado y devuelve EXCLUSIVAMENTE el JSON del esquema. REG-BOT decide que hacer con el veredicto; el modelo jamas toca el navegador.

```
Seguridad:
- hacia el modelo via solo imagen + texto redactado (regex tapa emails/telefonos/tokens largos). Cero credenciales, cookies o secretos de sesion.
- sin GEMINI_API_KEY o con API caida -> action=ERROR y REG-BOT
```

conserva el comportamiento determinista actual (CAPTCHA-PENDING en cola de handoff). Nunca cuelga la caza.

Estados (taxonomia del diseño):

```
no_challenge | supported_checkpoint | unsupported_checkpoint
| ambiguous | requires_human
```

Acciones ejecutables por REG-BOT:

```
CONTINUE      seguir solo (falso positivo de la heuristica)
GHOSTGATE     delegar a navegador real (sugerencia; REG-BOT loguea)
DISCARD       descartar blanco (solo con confidence >= 0.8)
REQUIRES_HUMAN cola de handoff al operador
ERROR         no se pudo clasificar -> comportamiento por defecto
```

CLI de prueba:

```
python3 core/vision_gate.py --html page.html
python3 core/vision_gate.py --image shot.png --url http://...
"""
```

```
=== waf_guard.py ===
```

```
# -*- coding: utf-8 -*-
"""
```

codexRC - GHOST-SHIELD (evasión WAF con memoria, v0.24.0)

=====

Capa de evasión del lado del COMPORTAMIENTO (no del payload). Los WAF modernos castigan patrones de tráfico antes que payloads sueltos; esta capa hace tres cosas:

1. JITTER: ninguna sonda sale con el mismo intervalo. Cada request duerme $\text{delay} * \text{uniforme}(0.5, 1.8)$ antes de salir, rompiendo el patrón de bot perfecto de intervalo fijo.
2. DETECCIÓN DE BLOQUEO: cada respuesta pasa por el clasificador. Firma dura (403/503 + WAF conocido o página de challenge) o blanda (429 rate-limit). Ante bloqueo duro se sugiere GHOSTGATE en el log.
3. COOLDOWN CON MEMORIA: el origen bloqueado entra en cooldown exponencial ($30s * 2^n$, tope 10 min) y el estado SE PERSISTE EN DISCO: la memoria de quien nos bloquea sobrevive jobs, reinicios y días. Mientras un origen está en cooldown, sus sondas se saltan en vez de seguir martillando y quemando la IP. Cuando el WAF nos deja pasar de nuevo (respuesta sana), el exponente se resetea.

Arquitectura: la capa vive DENTRO de la sesión HTTP (GuardedSession), no en cada batería. Cualquier sonda que use la sesión (arana, GET, forms, headers, XSS-PRO, IDOR, BLIND...) queda cubierta sin tocar su código. El carril async (SLIPSTREAM) engancha el mismo WafGuard en sus sondas httpx.

Solo lectura: esta capa no altera payloads ni escribe en el blanco.

```
"""
```

```
=== wcd404_probe.py ===
```

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
"""WCD-404-PROBE (v0.96.0): el oráculo de la cache que guarda 404s.
```

Pregunta: cuando el status miente (4xx cuyo CUERPO renderiza datos de la cuenta), el cache del edge lo guarda y se lo sirve a OTRO usuario?

Escalera cero-FP, monotona (cada peldano exige el anterior):

```
BENIGN          el disfraz responde 4xx con cuerpo LIMPIO:
                 no hay status-lie
STATUS-LIE-DETECTED 4xx cuyo cuerpo contiene marcadores DECLARADOS
                 de la cuenta propia (contrato pre-registrado:
                 el operador declara que marcadores son PII,
                 ej. email ninja propio; el probe no adivina)
WCD-404-LEAK      cebado con repeticiones -> el mismo URL con
                 la sesión de OTRO usuario (o anon) devuelve el
                 CUERPO DEL PROPIO: cross-user leak observable
```

Sin HIT de cache el veredicto honesto es LEAK-NO-CACHE (caso linktr.ee): la señal existe pero no contamina; NO reportable.

Regla de la casa (cache-semantic): cache key interna es HIPOTESIS; lo observable son huellas externas (headers de HIT / swap de cuerpo). Presupuesto: ≤ 8 requests, read-only. Sin blanco vivo sin 'LestGo'.

Uso:

```
from core.wcd404_probe import probe
probe({"url": "http://.../app/settings/profile/x.css",
      "marker_self": "ninja-A@maxxspace.com",
      "cookie_self": "acct=A", "cookie_victim": "acct=B"})
"""
```

=== wcd_bait.py ===

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

v0.86.0 WCD-CACHE-KEY (Web Cache Deception)

=====

Familia WCD con la disciplina de la linea DESYNC: contratos pre-registrados, juez determinista, presupuesto bajo y cero-FP por diseno.

Filosofia heredada (v0.72-v0.85):

- Solo evidencia observable; nada inferido queda fuera de 'unknown'.
- Un endpoint abierto (anonimo ya ve los datos SIN disfraz) NO es WCD: es un candidato BAC, familia distinta, veredicto honesto.
- Sin almacenamiento positivo observable en el edge (HIT/Age) NO es WCD reportable: es LEAK-NO-CACHE (probable, no reportable).
- DEMO exige contaminacion: una conexion NUEVA y SIN sesion recibe el cuerpo autenticado desde el cache, con controles PASSED. (Vary sobre Cookie o Set-Cookie en la respuesta descalifican.)

Contratos (pre-registrados en el ExperimentGraph):

baseline: A0 sesion@base debe contener el marcador autenticado (si no, NO-AUTH-DATA: no hay linea base de que es 'dato autenticado'); D0 anon@base NO debe verlo (si lo ve: OPEN-ENDPOINT, no es WCD).

por variante:

A	sesion + disfraz	-> leak? cache-positive?
B	sesion + disfraz (re)	-> HIT reproducido?
C	anon + disfraz	-> contaminacion observable?

Veredictos (escalera determinista):

BENIGN -> LEAK-NO-CACHE -> WCD-CACHE -> WCD-DEMO
(+ OPEN-ENDPOINT / NO-AUTH-DATA / UNREACHABLE / UNKNOWN)

Presupuesto: 2 baseline + 2 variantes x 3 = max 8 requests.

"""

=== wide_corpus.py ===

```
#!/usr/bin/env python3
"""v0.48.0 WIDE-CORPUS: corpus de caza = TODOS los plugins wordpress.org
con installs >= umbral (por defecto 5k), sin depender del filtro VDP.
```

Filosofia EVIDENCE-CHAIN: como los abogados cierran solos los falsos positivos, el cuello de botella ya no es el ruido sino el ancho de banda. El VDP de Patchstack ya no filtra a quien CAZAMOS; solo decide a quien REPORTAMOS (VDP activo = paga; sin VDP = CVE credit directo al vendedor).

CLI:

```
python3 core/wide_corpus.py          # >=5k installs -> wide_corpus.json
python3 core/wide_corpus.py --min 10000 # umbral a gusto
python3 core/wide_corpus.py --refresh  # refresca desde la API
"""
```

=== zombies.py ===

```
#!/usr/bin/env python3
"""ZOMBIE-KILL - mata procesos zombis y duplicados del backend CodexRC.
```

Leccion aprendida (27/09/2026, supervisor del bot de trading): pgrep/pkill -f matchea el PROPIO bash del comando (falso zombi). Este modulo NO usa pgrep: lee /proc directamente y excluye su propio PID y el de su padre.

Criterios:

1. Estado Z (zombi real: no consume CPU, el kernel los reapa con su padre; si el padre es nuestro server muerto, quedan colgando).
2. DUPLICADOS: dos+ procesos python ejecutando backend/app.py. El mas viejo (menor PID, no, mayor tiempo de inicio -> el de menor PID suele ser el primero) SOBREVIVE; el resto muere por PID con kill -9, nunca pkill -f.

```
Uso:
python3 core/zombies.py          # listado (no mata nada)
python3 core/zombies.py --kill   # mata duplicados (deja 1 vivo)
"""
```

10. ANEXO C — Metodología anti-falsos-positivos (texto íntegro)

CodexRC — Historial completo de falsos positivos, bugs del motor y lecciones

Versión del documento: 1.1 (03/10/2026) **Motor:** CodexRC / UNIVERSAL-ENGINE v0.58.0 **Actualización 1.1:** tras revisión experta se inició el SEMANTIC CORE (v0.58.0, nivel 0: identidad canónica + integridad). Decisión arquitectónica adoptada: si un FP nuevo se puede expresar como propiedad del CFG/data-flow/types, NO se crea una nueva regla RC de texto; se mejora el Semantic Core (jerarquía de 4 niveles: textual -> estructural -> semántico -> interprocedural; una señal de nivel inferior nunca contradice una prueba de nivel superior). Fase 1 COMPLETA (v0.58.1-v0.58.2): dominancia en producción en loose-auth-cmp (cmp_router, RC-000137) y en el patrón BAC (all_protected, RC-000132 resuelto). Pendiente Fase 1.5: SSA-lite en Taint-TRACE. **Autor:** elaborado por el agente de IA del operador (identidad del operador omitida por política) **Propósito:** presentar a expertos por área el historial real de falsos positivos y defectos del motor, con la causa raíz de cada uno, el fix aplicado y preguntas abiertas. Cada área es autocontenida: se puede enviar a un experto solo su sección.

Contexto mínimo para cualquier experto: CodexRC es un motor de caza de vulnerabilidades en plugins de WordPress escrito en Python puro (corre en Android/Termux armv7l, sin librerías compiladas). Tiene capas: Taint-TRACE (flujo de datos fuente->sink, intra-archivo, punto fijo), PATRON/CVE-MATCH (patrones estructurales ponderados, sin firmas fijas), GATES-AUDIT (mapeo de hooks AJAX/REST a sus callbacks y sus gates), EVIDENCE-CHAIN (cadena de evidencia evaluada por agentes deterministas: FISCAL, DEFENSA, JUEZ; nunca un LLM como juez) y DIFF-HUNT (solo código nuevo de los últimos 90 días). Filosofía declarada: preferir falsos negativos a falsos positivos (camino Infer/CodeQL). Regla de oro operativa: cada patrón de FP recurrente se codifica UNA vez como regla; nunca se tria a mano dos veces la misma clase.

ÁREA 1 — Análisis de taint / flujo de datos (TAINT-TRACE)

Para expertos en: taint analysis, análisis estático de PHP, abstract interpretation.

FP-1.1 · Sinks que matchean wrappers de WordPress (RC-000128)

- **Qué pasó:** el sink SQLi `->query( matcheaba new WP_Query , WP_Term_Query , WP_User_Query` . El core de WordPress sanitiza internamente; no es SQL crudo.
- **Impacto medido:** 19 de 39 SQLi del backlog wide-hunt eran esta familia (mayoría en unlimited-elements, que usa `$query->query($args)` en vez de `$wpdb->query`).
- **Fix (v0.57.5):** el sink exige `$wpdb->` o `$this->` con contexto de tabla real; los wrappers conocidos de WP se excluyen por nombre de clase.
- **Pregunta al experto:** ¿conviene una lista negativa de wrappers o resolver el tipo del receptor (interprocedural light) antes de decidir si es sink?

FP-1.2 · `prepare` multilínea autorreferencial (RC-000129)

- **Qué pasó:** el patrón de "prepare seguro" no limpiaba el taint cuando el formato era construido iterativamente: `$q = $wpdb->prepare($q, $args)` y acumulación `.=` . El punto fijo re-infectaba la variable en cada iteración, y el formato concatenado con `$wpdb->prefix` rompía el reconocimiento de placeholder.

- **Fix (v0.57.5):** reconocimiento de la forma autorreferencial + el prefijo de tabla tratado como constante segura en el format-string.
- **Pregunta al experto:** en análisis intra-archivo con punto fijo, ¿cuál es la forma correcta de modelar "acumulador que pasa por prepare en un loop" sin análisis de reachability completo?

FP-1.3 · Taint a nivel archivo, sin scopes de función (RC-000130)

- **Qué pasó:** una variable local reutilizada como texto no-SQL en OTRA función del mismo archivo (`$jsst_query = implode` de términos de búsqueda en js-support-ticket) contaminaba los queries de todas las demás funciones del archivo.
- **Fix (v0.57.5):** scopes de función: el taint vive y muere dentro del cuerpo de cada función.
- **Pregunta al experto:** obvio en retrospectiva, pero el diseño original elegía granularidad de archivo por simplicidad. ¿Qué granularidad mínima recomendarían para un motor de Python puro: función, bloque, o SSA light?

Controles positivos del área (para calibrar)

- Lab vulnerable 3/3 detectado (SQLi ciego, XSS, unserialize), 0 FP en control sanitizado.
- Sanitizadores reconocidos: `intval/absint/sanitize_*/esc_*`, `prepare` con placeholders, `extract($_GET)` flag directo, `implode(array_fill(%d))` ya no se taintea (FP de format-string).

ÁREA 2 — Patrones estructurales / BAC (PATRON + CVE-MATCH)

Para expertos en: static analysis rules, WordPress security, authorization logic.

El detector de patrones combina señales estructurales débiles ponderadas (hook nopriv +3, escritura +3, input +2, protección -4) y cita la familia CVE al disparar. Sin firmas fijas: la misma regla "re-descubre" fallos conocidos. Redescubrió solo, por puro patrón, el 0-day real de RegistrationMagic (`validate_ipn`, paypal.php:193, salto `validate_ipn->callback->paypal_ipn` nopriv, sin autenticación) — validación de que el enfoque funciona.

FP-2.1 · BAC: handler con gate interno (RC-000131)

- **Qué pasó:** hooks `wp_ajax_nopriv` registrados de forma INTENCIONAL (formularios front-end públicos) pero con gate completo dentro del handler: whitelist de métodos + nonce obligatorio + capability check. El patrón disparaba por el HOOK, sin mirar el cuerpo. Caso real: pods `admin_ajax`: 20 sinks reportados, todos FP.
- **Fix (v0.57.7):** `finalize()` postergado: los hallazgos esperan el mapa de TODAS las funciones, resuelven el callback del hook (`'fn'`, `array($this, 'método')`) y escanean el CUERPO del handler en busca de gates. Handler gateado = descartado.

FP-2.2 · BAC: gate en el caller, no en la función (RC-000133)

- **Qué pasó:** el patrón disparaba sobre funciones que ni siquiera tienen hook nopriv: solo "escribe datos + usa input + bonus de nonce público en el archivo". El gate vivía en el CALLER (pods `admin_save`, gate en l18n.php:123: `pods_is_admin` + `wp_verify_nonce`).
- **Fix (v0.57.7):** se exige que la señal nopriv esté en el cuerpo escaneado.

FP-2.3 · Ramas públicas por diseño (RC-000132) — SIN RESOLVER COMPLETO

- **Qué pasó:** handler nopriv cuyas ramas públicas son features intencionales (render de bloques de ads, ads.txt público) y las ramas sensibles están gated por option default-off o constante "pro". El write real (`update_option`) vive en la rama gated.
- **Estado:** parcial. El gate interno se cubre; la separación de ramas por diseño SIGUE REQUIRIENDO triaje. **Pregunta abierta al experto:** ¿cómo distinguir automáticamente "rama pública por diseño" de "rama pública alcanzable por salto de flujo"? ¿Análisis de dominance en el CFG alcanza?

FP-2.4 · "Verificación de pago degradable" que es diseño del vendor (RC-000134)

- **Qué pasó:** wc-multivendor-membership completa la suscripción sin contactar gateway. Suena a IPN-downgrade (familia CVE-2026-9242), pero: el paymode se valida contra allow-list doble estricta (`enabled n offline, in_array` con `true`), los gateways online SOLO se completan tras IPN verificado, y `is_valid_member_id` exige `get_current_user_id() == $member_id` (self-scoped, no IDOR). El propio vendor dejó comentario citando CVE-2026-12967: ya parcheó esta clase.
- **Lección:** "completa pago sin gateway" no es bug cuando el método es offline habilitado por el admin (conciliación manual). La regla para automatizar: exigir AUSENCIA de `in_array` estricto sobre el paymode antes de reportar. Aún manual.

FP-2.5 · Loose-cmp: comparación floja sin contexto de autorización (RC-000135)

- **Qué pasó:** el patrón `loose-auth-cmp` (familia CVE-2023-6875, Post SMTP) disparaba con CUALQUIER `== / !=` sobre input de usuario con score 6/6 de una sola señal: filtros de query (`$_POST['term_id'] != 0`), flags (`checkbox == 1`), presencia de parámetros (`$_GET['code'] != ''`), routers de flujo OAuth. 23 de 29 hallazgos N1 n VDP de esta familia eran esta clase.
- **Matices que costó aprender (sub-fixes):**
 - El token `auth` matcheaba `author` en queries de posts -> lookbehind anti-author.
 - El token `token` mantenía vivo cualquier handler OAuth por contexto -> tokens débiles solo cuentan en el snippet de la comparación (± 80 chars), no en contexto.
 - `"nonce was verified"` en un COMENTARIO contaba como gate -> comentarios despojados del contexto antes de evaluar tokens.
 - El snippet se toma de la LÍNEA SEÑALADA, no del primer `==` del cuerpo (FP: checkbox de newsletter dentro de una función de registro llena de password/nonce en jsst-hooks.php:175).
- **Fix (v0.57.9):** tokens AUTH fuertes (password/passwd/user_pass/nonce/capability/current_user_can/login/role/privilege/cookie/manage_options) en snippet o contexto ± 5 líneas; tokens débiles (auth/token/secret) solo en operandos.
- **Control crítico preservado:** el caso real 6875 (`==` sobre password) sigue vivo tras todas las guardas. El filtro de flag en el mismo archivo muere.

FP-2.6 · Loose-cmp: router detrás de gate adyacente (RC-000136)

- **Qué pasó:** quedaban 6 vivos de la forma `$_GET['auth']==$ntInfo['lcode']` (social-networks-auto-poster, 6 archivos clonados): llevan "auth" en el operando (pasaban la guarda de tokens) pero en la misma línea o la anterior hay un capability check real (`nxs_snap_user_can_access()`). Esa llamada ES el control de acceso; la comparación floja es solo el router del flujo.
- **Fix (v0.57.10):** `_GATE_ADJ` generalizada (sin hardcodear funciones de plugin):
`current_user_can|user_can(|is_user_logged_in|_can_access\b|wp_die|die(|exit` en ± 1 línea -> FP. Se evalúa ANTES que los tokens (si no, los 6 volvían por el token del operando).
- **Residuo conocido:** 2 hallazgos fuera del backlog con el gate a 2-3 líneas (`nxs_class_snap.php:177`, filtro magic-quotes; `nxs_functions_adv.php:10`, dispatcher ajax con `current_user_can` 2 líneas arriba). ¿Ensanchamos la ventana a ± 2 o mantenemos ± 1 para no matar gates lejanos bypassables? **Pregunta abierta al experto.**

ÁREA 3 — Cadena de evidencia / resolución de rutas (EVIDENCE-CHAIN)

Para expertos en: program analysis infrastructure, data structures, verificación.

BUG-3.1 · Resolución de ruta por basename (RC-000127) — bug CRÍTICO real del motor

- **Qué pasó:** dos archivos del mismo plugin compartían basename (`wpil/Error.php` VS `wpil/Table/Error.php`). El fallback de resolución de ruta en `build_chain` no rompía el loop externo de `os.walk`, así que el motor analizaba el archivo EQUIVOCADO: el ÚLTIMO visitado, no el correcto.
- **Impacto:** cualquier hallazgo en un archivo con basename duplicado podía llevar evidencia de otro archivo. Cuantificación del alcance en corridas previas: no determinado. Caso testigo real: link-whisper SQLi pasó de falso positivo "DEMOSTRADO-ESTATICO" a correctamente CERRADO tras el fix.
- **Fix inicial (v0.49.1):** match por ruta relativa exacta primero; el basename quedó como último recurso.
- **Fix definitivo (v0.58.0, SEMANTIC CORE nivel 0):** el basename dejó de ser identidad. Identidad canónica `FileId` = `root` + ruta relativa normalizada + hash de contenido (sha256-16) como testigo. Dos archivos con basename

compartido -> INTEGRITY FAILURE: NO se elige ninguno. El JUEZ bloquea DEMOSTRADO-ESTATICO si integrity != OK: la evidencia de un archivo equivocado nunca alcanza veredicto alto. El bug se convirtió en invariante verificable, no en fallback.

- ~~Pregunta al experto~~ -> **RESUELTA por revisión experta (02/10/2026)**: se adoptó exactamente la identidad canónica FileId -> ContentHash -> contenido analizado, con INTEGRITY FAILURE como respuesta a cualquier desajuste.

Diseño del área (para opinión general)

- Cada hallazgo recibe cadena SOURCE/FLOW/AUTH/SANITIZATION/SINK/CORRELATION.
- FISCAL prueba 4 requisitos; DEFENSA argumenta refutaciones (gate, sanitizador, prepare, admin-only); JUEZ determinista emite veredicto: CONFIRMED / DEMOSTRADO-ESTATICO / PROBABLE / CONTESTADO / DESCARTADO. Nunca un LLM como juez.
- Los DESCARTADO por la DEFENSA se filtran de los vivos en DIFF-HUNT.

ÁREA 4 — Mapeo de gates / hooks (GATES-AUDIT)

Para expertos en: WordPress internals, hooks AJAX/REST.

FP-4.1 · Notices cosméticos con nonce (validación de campo)

- **Qué pasó:** los primeros candidatos BAC de wpdatatables y ai-engine resultaron notices cosméticas (updates de UI) con nonce válido. El veredicto REVISAR-AUTH se disparaba por mapear bien el hook pero no ponderar qué HACÍA la acción.
- **Lección incorporada:** CANDIDATO-BAC requiere además write con impacto (options de otro scope, datos de terceros), no solo handler sin gate aparente.

FP-4.2 · Hooks dinámicos no resueltos (nivel honesto declarado)

- **Qué pasó:** CALLBACK-NO-RESUELTO para hooks registrados con nombres dinámicos (`add_action('wp_ajax_'.$this->prefix.'_'.$action, ...)`). El motor los marca nivel 1 (no los resuelve) en vez de inventar.
- **Pendiente histórico:** deep-dive de ai-engine `files.php` (candidates LFI `file_get_contents $path` en `classes/modules/files.php:140,167` + `rest.php SIN-HANDLER`). Nunca se cerró formalmente. **Pregunta al experto:** ¿vale la pena resolver prefijos de hook por inferencia de constantes de clase, o el ratio FP/TP no lo paga?

Registros auditados y muertos en triaje manual (muestra del ruido que producen los handlers ajax sin gate aparente)

- cartflows v3.2.1: el "código sospechoso nuevo" era el parche de seguridad del propio vendor.
- captcha-code-authentication v3.33: comparación floja HEREDADA solo en registro (categoría no pagada); el login está blindado con `===`.
- simple-local-avatars v2.8.6: nonce + whitelist mime estricta.
- ht-mega theme-builder v3.3.2: refleja constantes internas, no input.
- wpzoom instagram settings v2.4.0: imprime flags internos del plugin.

ÁREA 5 — Triage de hallazgos dinámicos / web apps (no-WordPress)

Para expertos en: web security, DOM security, SPAs.

FP-5.1 · Reflexión en puntos RSC/Next.js (xenpaid.com)

- **Qué pasó:** CODEX-OBSERVE marcó puntos de reflexión que en Next.js con RSC no ejecutan script (los datos viajan por el protocolo RSC, no por HTML directo).
- **Lección:** la superficie XSS efectiva depende de la arquitectura de render; un motor que no modela RSC/hidratación va a reportar reflexiones inertes.

- **Pregunta al experto:** ¿qué heurística mínima distingue un sink DOM real de una reflexión RSC inert, sin renderizar?

FP-5.2 · postMessage handlers sin origen verificado... que sí lo verifican (xenpaid)

- **Qué pasó:** EVIDENCE-CHAIN + TAINT-TRACE marcaron hallazgos postMessage; la DEFENSA los refutó como falsos positivos (checks de origen presentes).
 - **Estado:** cerrados por refutación de la DEFENSA. La cadena funcionó como debe.
-

ÁREA 6 — Infraestructura, operación y tooling propio

Para expertos en: pipelines de análisis, reliability, LLM ops.

BUG-6.1 · Workers del sandbox se niegan SIEMPRE a análisis de seguridad

- **Qué pasó:** los sub-agentes del sandbox (workers de IA) rechazaron TODO análisis de seguridad sobre código concreto, probado con 3 framings (directo, contexto defensivo completo, micro-tarea neutra de trazabilidad): 6 negativas.
- **Decisión:** Semgrep v1.178.0 en sandbox como capa SINK-SCAN determinista (nunca se niega); los workers SOLO para tareas no-security (descargas, inventarios, formateo). El LOGIC-AUDIT lo hace el agente principal.
- **Pregunta al experto:** patrones conocidos para enrutar tareas a modelos que no se niegan sin perder cobertura?

BUG-6.2 · Arsenal: selección de payload atascada en UNA opción (v0.57.1)

- **Qué pasó:** el módulo de pruebas mantenía un solo payload elegido manualmente y no ciclaba la suite. Causa: la validación de UI bloqueaba la auto-selección y el loop no escalaba por tipo de hallazgo.
- **Fix (v0.57.1):** auto-selección por tipo de hallazgo conocido; default a la matriz LAB completa para tipos desconocidos. Se eliminó el estado "atascado".

BUG-6.3 · Regresión de resolución de URLs absolutas (UNIVERSAL-API, v0.57.x)

- **Qué pasó:** la refactorización a UNIVERSAL-API-MATRIX rompió `_armar_url_api`: endpoints absolutos se concatenaban mal; y un payload sin el campo `login` que el backend esperaba causaba rechazo silencioso.
- **Fix:** manejo de absolutos + mecanismo de payload adaptativo que aprende los campos que el servidor exige.
- **Lección:** cada refactorización de URL-building necesita tests con URLs absolutas, relativas y con base combinada, ANTES de tocar el motor.

BUG-6.4 · Endpoints no-responsivos consumían el presupuesto del lote

- **Qué pasó:** en caza en lote, un blanco con API colgada congelaba la ejecución (timeout largo por intento).
- **Fix:** presupuesto duro de 60 intentos por blanco + timeout por intento + aislamiento (un blanco fallido no tumba el lote).

BUG-6.5 · Filtro de ruido de librerías: rutas sin barra inicial (v0.42.x)

- **Qué pasó:** `is_noise` no matcheaba rutas de librerías sin `/` inicial, así que vendor/ y lib/ enteros entraban al análisis y generaban hallazgos de terceros.
- **Fix:** normalización con prepend de `/`.

BUG-6.6 · Type-confusion en la medición del propio agente (metodología, v0.57.8)

- **Qué pasó:** al comparar hallazgos viejos vs nuevos por UBICACIÓN (archivo+línea±5), el agente contó como "sobreviviente" un hallazgo de OTRA familia que casualmente caía en la misma línea. El "15 loose-cmp vivos" real era 29, y 2 de los "sobrevivientes" eran de otra clase.
- **Fix:** el match exige misma familia de patrón, no solo ubicación. Lección de método: medir supervivencia exige igualdad de (ubicación, tipo), nunca ubicación sola.

Limitaciones estructurales honestas (declaradas, no bugs)

- Python puro en armv7l/Termux: no hay curl_cffi ni librerías compiladas; se usan fallbacks (cloudscraper). Los análisis interprocedurales se delegan a Semgrep.
- Taint-Trace intra-archivo: para interprocedural está Semgrep (capa separada).
- El punto fijo es heurístico (nivel 1); documentado como tal.

Los dos controles que validan el sistema (verdaderos positivos)

1. **RegistrationMagic validate_ipn (paypal.php:193):** 0-day real, DEMOSTRADO-ESTATICO por la cadena completa, salto `validate_ipn->callback->paypal_ipn` nopriv sin autenticación. Redescubierto por el patrón de CVE-MATCH SIN firma del fallo, y confirmado por EVIDENCE-CHAIN. Después de 10 clases de FP cerradas, el caso sigue vivo.
2. **CVE-2023-6875 lab (Post SMTP family):** `==` flojo sobre password, sin gate adyacente, sin tokens de comentario. Sigue detectado tras TODAS las guardas (135, 136). Es el test de no-regresión de la familia loose-cmp.

Métrica de madurez del corpus: 10 clases de FP codificadas (RC-000127 a RC-000136). Última familia cerrada (loose-cmp N1 n VDP): 29 hallazgos -> 0 vivos, 23 muertos por RC-000135 y 6 por RC-000136, 0 muertos a mano en la última pasada. Las ventanas de texto (± 1 , ± 5 , ± 80) se declaran pre-filtros transitorios: serán reemplazadas por dominancia real del CFG a medida que avance el Semantic Core.

Preguntas transversales para cualquier experto

1. Con filosofía "preferir FN a FP", ¿dónde está el punto óptimo de endurecimiento de reglas antes de empezar a perder TP reales que no conocemos?
2. Las guardas actuales son regex sobre contexto local (± 1 a ± 5 líneas). ¿Qué caso justificaría pasar a análisis de CFG real (dominance, reachability) y cuál es el costo esperable?
3. Corpus de regresión: hoy son 10 casos con repro mínima. ¿Qué tamaño y forma de corpus consideran suficiente para refactors del motor sin miedo (estilo Soundness testing)?
4. Segunda etapa de juicio: con las restricciones (sin LLM externo confiable para security, Python puro), ¿reglas deterministas + cola de triaje batch es el techo razonable, o hay una capa automática que estamos dejando en la mesa?

11. ANEXO D — Dossier de investigación Q4 2026 (texto íntegro)

Dossier de investigación — Q4 2026

Investigación de clases de vulnerabilidad recientes, de alto valor y ausentes del motor. Criterio de admisión: paga en Patchstack/bounties, es novedosa (no está en tools abiertos) y encaja en los oráculos honestos del pipeline. Invariante: nada de esto se ejecuta en vivo sin orden del operador.

1. MAGIC-CONFUSION — extensión vs contenido en rutas de upload

Fuente: CVE-2026-65640, WordPress Core <= 7.0.3, Author+ RCE (CVSS 9.1, publicado 12/08/2026, hallado por pwn.ai, parche 7daaa50).

La lección estructural: ImageMagick decide el formato por magic bytes (`%! , \x04%!`, `\xC5\xD0\xD3\xC6`, formato prefijado tipo `EPS:file.png`), WordPress confía en la extensión. Cualquier ruta de upload que llegue a Imagick/Ghostscript SIN `wp_check_filetype_and_ext()` es RCE potencial. Las rutas que ya lo esquivan en core: XML-RPC `wp.uploadFile` y extracción de cover-art de MP3s, ambas vía `wp_upload_bits()` que jamás inspecciona contenido.

Por qué es calibre: es una familia NUEVA (agosto 2026) y aplica a cualquier PLUGIN que procese media con Imagick en rutas de upload propias. Nadie publica un hunter estático de esta clase todavía.

Integración propuesta (estática): - `pattern_match`: familia MAGIC-CONFUSION. Señal: sink `readImage/readImageBlob/Imagick()` alcanzado desde una ruta de upload donde NO domina `wp_check_filetype_and_ext` ni sniffing de contenido propio. - `gates_audit`: el gate aquí es el check de contenido, no un nonce: extender el modelo de gates para tratar validadores de contenido (`finfo`, `magic bytes`, `getimagesize`) como gates de sink. - Escalera honesta: MAGIC-CONFUSIBLE (estático) -> verificación WP-LAB con payload PostScript inerte y control benigno (un PNG real). Nunca Ghostscript ejecutable en blanco de terceros: PoC con archivo inerte que DEMUESTRE el routing, no la ejecución.

Precio: RCE pagable; Author+ = rol aceptado por Patchstack.

2. POI-REACH — Object Injection unauth con gadget chain

Fuente: CVE-2026-2599, "Contact Form Entries" <= 1.4.7, CVSS 9.8, unauthenticated PHP Object Injection -> RCE. `download_csv()` hace `unserialize()` de input del usuario sin `allowed_classes`.

La lección estructural: la clase sigue pagando (Patchstack la lista explícitamente). Lo que falta en los tools abiertos no es detectar `unserialize` — es probar **alcance:** unauth + gadget chain disponible en el propio plugin. La superficie ganadora: handlers de export/CSV/download, que reciben parámetros serializados y casi nunca pasan por gates.

Por qué es calibre: `unserialize($input)` unauth con POP chain propia = \$600–\$2.600 según installs. Nuestro Taint-TRACE ya detecta `unserialize`; falta la parte que cobra.

Integración propuesta (estática): - `taint_trace`: sink `unserialize` con línea `SOURCE` (ya existe); etiquetar handlers `export/download/csv` como superficie prioritaria. - `gates_audit`: veredicto AUTH del handler que contiene el `unserialize` (unauth/nonce-less = candidato POI-REACH). - Gadget chain: `deep_scan` indexa clases del plugin con métodos mágicos (`__destruct`, `__wakeup`, `__toString`) y propiedades llamables -> POP-CANDIDATE como evidencia de explotabilidad (demostrable en WP-LAB con harmless chain). - Escalera honesta: POI-DETECTED (sink unauth) -> POI-CHAIN (gadget en el mismo plugin) -> POI-DEMO (WP-LAB, chain inerte).

3. WCD-404-LEAK — la cache que guarda cadáveres con datos

Fuente: reporte real 2025 (System Weakness, ValidByAccident, marcado duplicado = confirmando que el programa lo paga): endpoint autenticado con payload de extensión falsa devuelve **404 con el cuerpo lleno de datos privados**, y el cache guarda el 404 como clave única. Usuario B visita la misma URL -> datos del usuario A.

La lección estructural: el status code miente. La pregunta correcta no es "¿cachea páginas privadas?" sino "¿la respuesta — incluso de error — contiene datos de sesión y es cacheable?".

Por qué es calibre: es una variante 2025 de la clase que ya casi cobramos en linktr.ee (allí el edge no cacheaba nada: LEAK-NO-CACHE fue el veredicto honesto). Con este lente el motor pregunta más.

Integración propuesta (dinámica, sobre cache_semantic): - Nuevo escenario: enviar disfraz de extensión, leer STATUS + BODY. Oráculo: body contiene marcadores de datos de cuenta (email, nombre, tokens) con status 404/4xx -> STATUS-LIE-DETECTED. - Segundo paso (el que paga): esperar/forzar cache HIT de la URL disfrazada y verificar como usuario B distinto -> cross-user leak real. Presupuesto declarado, read-only, cuentas ninja propias. - Escalera honesta: STATUS-LIE (señal) -> WCD-404-LEAK (cache HIT cross-user) -> impacto PII. Sin HIT: LEAK-NO-CACHE honesto, igual que el test de linktr.ee.

Contexto de mercado (verificado 05/10/2026)

- Patchstack 2025: 7.966 vulnerabilidades nuevas en WP; 33% de plugins vulnerables sin fix. XSS = 47,7% del volumen pero severidad baja; lo que paga es lo de siempre: RCE, Object Injection, PrivEsc, BAC sobre secretos.

- WordPress Core 7.0.4 parcheó la familia Imagick: los plugins que duplican el patrón (procesan media por su cuenta) quedaron expuestos a la misma clase. Corpus de 3.260 plugins >=5k installs = superficie real para MAGIC-CONFUSION.
- USENIX Sec'25 (Zhang et al.): minería precisa de gadget chains por taint tracking a nivel de llamada — metodología aplicable al POP-CANDIDATE de POI-REACH.

NOTA DE CAZA (05/10/2026, primera corrida LestGo con v0.96.0)

Calibración sobre contact-form-entries 1.4.7 (CVE-2026-2599): el motor NO dispara (poi=0/0/0) porque el flujo real del CVE es SEGUNDO ORDEN: el payload serializado entra por el formulario, se almacena en DB, y el unserialize ocurre al leer la entrada (download_csv -> maybe_unserialize SIN allowed_classes, alcanzable sin login vía vx_crm_key en \$_GET, líneas 76-83 y 3017). TAINT-TRACE level-1 no modela ida-vuelta por DB. Gap honesto documentado: la clase POI-REACH hoy caza unserialize DIRECTO (input del usuario al sink sin persistencia). Extension futura: taint de segunda orden (source = handler que guarda, sink = unserialize de lectura DB). No es falso negativo del corpus: el caso de prueba cubre el flujo directo, como fue diseñado.

ESTADO DE INTEGRACIÓN (05/10/2026): COMPLETO v0.96.0

Las TRES clases fueron integradas y certificadas (139/139 PASS): POI-REACH (core/poi_reach.py, casos RC-000261..263), MAGIC-CONFUSION (core/magic_confusion.py, RC-000264..265) y WCD-404-LEAK (labs/wcd404_lab.py + core/wcd404_probe.py, RC-000266..268). Ver CHANGELOG v0.96.0. Lo que sigue (caza con blancos reales) requiere 'LestGo'.

Orden de integración propuesto (histórico)

1. POI-REACH (paga ya, motor casi listo, solo faltan gates + gadgets)
2. MAGIC-CONFUSION (clase fresca, sin competencia, RCE)
3. WCD-404-LEAK (upgrade barato de cache_semantic, cierra la pregunta que linktr.ee dejó a medias)

Ninguna acción en vivo sin "LestGo". Todo nace en LAB con corpus.

Evaluado y descartado: HTTP/2 Bomb (CVE-2026-49975)

Bomb HPACK + retención Slowloris contra nginx <=1.29.7, Apache httpd <=2.4.67, IIS, Envoy, Pingora. DoS por amplificación de memoria.

Por qué NO entra al motor: es clase de disponibilidad. Los programas de bounty excluyen DoS del scope (probarlo = tumbar el servicio de un tercero: destructivo, contra nuestras reglas read-only). No paga, no se reporta, no se ejecuta en vivo jamás.

Valor residual: la mecánica (amplificación HPACK en la traducción edge->origin) confirma que nuestra h2_lab/h2_probe modela la capa correcta. Si algún día un vendor paga por DoS con autorización escrita, el lab ya sabe hablar H2 real. Nada más que hacer aquí.

Decisión: documentado, descartado. El foco sigue en POI-REACH -> MAGIC-CONFUSION -> WCD-404-LEAK (las clases que pagan).

12. Documentos relacionados (fuente original de cada anexo)

- [README.md](#)
- [ARCHITECTURE.md](#)
- [AGENTS.md](#)
- [CHANGELOG.md](#) — volcado íntegro en el Anexo A.
- [modulos_index.csv](#) — tabla resumen de los 102 módulos.

- [AUDITORIA_FALSOS_POSITIVOS.md](#) — volcado íntegro en el Anexo C.
- [DOSSIER_INVESTIGACION_2026Q4.md](#) — volcado íntegro en el Anexo D.

Tabla resumen - Indice de los 102 modulos

Modulo	Version	Descripcion
__init__		codexRC core package
ab_diff	v0.68.0	AB-DIFF universal (v0.68.0)
adaptive_hunt	v0.81.0	ADAPTIVE-HUNT (v0.81.0): la capa epistemica.
arsenal	v0.55.0	ARSENAL v0.55.0 — dos niveles: BASICO (verde, sin candado) y EXTREMO (rojo, contraseña).
async_lane	v0.23.0	SLIPSTREAM (carril async, v0.23.0)
auth		Authentication Module (Termux friendly)
bac_proof	v0.67.0	BAC-PROOF (v0.67.0, AUTHZ-PROOF capa 4)
bin_audit		BIN-AUDIT: analisis binario/nativo, mas alla del navegador.
brain		CEREBRO: la capa de inteligencia determinista del Hunter.
cache_bait		CACHE-BAIT: bateria dedicada de cache poisoning y cache deception.
cache_baseline	v0.79.0	CACHE-BASELINE (v0.79.0): clasifica el baseline del target.
cache_controls	v0.79.0	CACHE-CONTROLS (v0.79.0): explicaciones alternativas ANTES
cache_correlation	v0.79.0	CACHE-CORRELATION (v0.79.0): el orquestador.
cache_fingerprint	v0.79.0	CACHE-FINGERPRINT (v0.79.0): huella SOLO de senales
cache_semantic	v0.79.0	CACHE-SEMANTIC (v0.79.0): relacion entre Request A y Request B.
cfg	v0.58.1	SEMANTIC CORE (v0.58.1) — Nivel 1: CFG intra-funcion + dominancia.
chain		ENCADENAR HALLAZGOS: el valor real esta en las CADENAS, no en piezas.
cl0_probe	v0.87.0	v0.87.0 CL.0-SINGLE-TIER (Request Smuggling CL.0)
connection_state	v0.77.0	CONNECTION-STATE AUDIT (v0.77.0): ¿deja una peticion ambigua
cov_bait		COV-BAIT v2: caza guiada por COBERTURA DE CODIGO + MOTOR DE MUTACION.
crc_probe	v0.90.0	v0.90.0 CROSS-REQUEST CORRELATION
cross_layer	v0.84.0	v0.84.0 CROSS-LAYER CORRELATION
crown_chain	v0.95.0	v0.95.0 CROWN CHAIN — INTEGRACION AL HUNTER

cspt_scan	v0.71.0	CSPT-SCAN (v0.71.0): Client-Side Path Traversal.
cve_matcher		CVE Matcher Module
decompile	v0.36	DECOMPILE: descompilacion parcial nivel 3 (bytecode DEX -> pseudocodigo).
deep_scan		Deep Scan Module
desync_hunt	v0.83.0	DESYNC-HUNT (v0.83.0): investigacion DESYNC sobre el
diff_hunt	v0.45.0	DIFF-HUNT (v0.45.0)
dossier_hunt	v0.96.1	DOSSIER-HUNT (v0.96.1) — caza con las clases Q4-2026
edge_profile	v0.75.0	EDGE-PROFILE (v0.75.0): caracteriza el CONTRATO del edge.
edgesync	v0.74.0	EDGESYNC-HUNT (v0.74.0): SONDA-DE-CORRELACION.
escalada	v0.28	ESCALADA: que hacer DESPUES del aviso critico.
evidence	v0.47.0	EVIDENCE-CHAIN + ABOGADOS (v0.47.0)
evidence_package	v0.94.0	v0.94.0 EVIDENCE PACKAGE
experiment_catalog	v0.81.0	CATALOGO DE EXPERIMENTOS (v0.81.0): sondas HTTP de solo
experiment_graph	v0.83.0	EXPERIMENT-GRAPH (v0.83.0): grafo dirigido de
experiment_selector	v0.82	SELECTOR DE EXPERIMENTOS (v0.82): grafo de experimentos.
fp_autoclose	v0.44.0	FP-AUTO-CLOSE (v0.44.0)
fp_elimination	v0.93.0	v0.93.0 FP-ELIMINATION
fp_memory	v0.66.0	FP-MEMORIA (v0.66.0, AUTHZ-PROOF capa 5)
gates_audit	v0.43.0	GATES-AUDIT (v0.43.0)
ghostgate		Modulo GHOSTGATE (Cloudflare relay, Termux friendly)
graphql_bait		GRAPHQL: bateria dedicada contra endpoints GraphQL ocultos.
h2_probe	v0.88.0	v0.88.0 H2-TRANSLATION (Parser Differential)
hunt_wide	v0.48.0	v0.48.0 WIDE-HUNT: caza sobre el corpus COMPLETO wordpress.org (>=5k installs).
hunter	v0.57.5	Hunter (FACHADA, v0.57.5)
hunter_base		Motor de descubrimiento (arana + inventario de parametros) y corpus de
hunter_blind		from typing import Any, Callable, Dict, List, Optional, Set, Tuple
hunter_deep	v0.57.2	Caza profunda multi-vector. Extraida de hunter.py (v0.57.2).

hunter_spider	v0.57.2	Arana de descubrimiento: mapa paginas/params/endpoints. Extraida de
hunter_xss		from typing import Any, Callable, Dict, List, Optional, Set, Tuple
hunter_xsspro	v0.20.0	from typing import Any, Callable, Dict, List, Optional, Set, Tuple
hypothesis_graph	v0.81.0	HYPOTHESIS-GRAPH (v0.81.0): hipotesis explicativas de una
impact_probe	v0.92.0	v0.92.0 IMPACT CORRELATION
lab_ab_site	v0.68.0	LAB-AB-SITE (v0.68.0): sitio NO-WordPress de prueba
magic_confusion	v0.96.0	MAGIC-CONFUSION (v0.96.0): extension vs contenido en rutas de media.
mc_probe	v0.89.0	v0.89.0 MULTI-CONNECTION (Efecto de Segundo Orden)
normalization_audit	v0.76.0	NORMALIZATION-AUDIT (v0.76.0): que representacion recibe el
observe		CODEX-OBSERVE (Fase 1, alcance reducido).
path_bait		PATH-BAIT: bateria dedicada de LFI / path traversal (PAGA en Patchstack).
pattern_match		CVE-MATCH: aprendizaje de patrones globales sobre codigo PHP.
pipeline		Pipeline Orchestrator
plugin_batch	v0.42.0	PLUGIN-BATCH (v0.42.0)
poi_reach	v0.96.1	POI-REACH (v0.96.1): Object Injection con alcance y gadget chain.
race_proof	v0.71.0	RACE-PROOF (v0.71.0): ejecutor dinamico de carreras TOCTOU.
race_trace	v0.70.0	RACE-TRACE (v0.70.0)
re_engine		REVERSE: ingenieria inversa nivel 2 (sobre BIN-AUDIT).
recon		Basic Recon Module
reg_bot	v0.68.0	REG-BOT (v0.68.0, AB-DIFF universal)
regress		CODEX-REGRESS (alcance reducido: Regression Corpus + ejecucion).
report_export		Reportes exportables del Hunter: TXT, JSON y PDF.
repro_engine	v0.91	REPRODUCTION ENGINE v0.91 — etapa 21 del roadmap
research_memory	v0.81.1	RESEARCH-MEMORY (v0.81.1): memoria de investigacion por
retro_hunt	v0.51.0	RETRO-HUNT (v0.51.0) — CodexRC
saml_audit	v0.71.0	SAML-DEFENSE (v0.71.0): auditor SAML/XML de PHP.
self_tune		AUTOCORRECCION: el Hunter recuerda que rindio y ajusta su propio gasto.

semantic_channels	v0.80	Canales de observacion de SEMANTIC-CACHE (v0.80).
semantic_core	v0.58.0	SEMANTIC CORE (v0.58.0) — Nivel 0: identidad canonica de recursos.
semantic_dimension	v0.80	Fase 2 de SEMANTIC-CACHE: localizacion de dimension y
semantic_engine	v0.80	SEMANTIC-CACHE (v0.80): motor de desacuerdos semanticos.
semantic_judge	v0.80	Juez determinista de SEMANTIC-CACHE (v0.80).
semantic_models	v0.80	Modelos de datos de SEMANTIC-CACHE (v0.80).
sqli_bait		SQLI-BAIT: deteccion avanzada de inyeccion SQL en la superficie de la app.
ssa		SSA-lite: grafo de versiones def-use para Taint-Trace.
ssti_bait		SSTI-BAIT: bateria dedicada de inyeccion de plantillas con FINGERPRINT.
state		Estado individual por operador/IA.
state_correlation	v0.78.0	STATE-CORRELATION (v0.78.0): un veredicto por target, no por
stealth_bait		STEALTH-BAIT: caza de superficie oculta para blancos blindados.
taint_trace		TAINT-TRACE: analisis de flujo de datos (taint) sobre codigo fuente PHP.
tech_detect		Basic Technology Detection (html.parser)
universal_engine	v0.52.0	UNIVERSAL-ENGINE (v0.52.0) — CodexRC
vdp_fresh	v0.61.0	v0.61.0 VDP-FRESH: cazar los VDP RECIEN AGREGADOS a Patchstack.
vdp_watcher	v0.62.6	VDP-FRESH WATCHER (v0.62.6).
vendor_farm		VENDOR-FARM: descubre familias de plugins de UN MISMO vendor con
veritas		VERITAS: verificacion de XSS con navegador real (Chrome headless).
vision_gate	v0.69.0	VISION-GATE (v0.69.0): clasificador visual de challenges via Gemini.
waf_guard	v0.24.0	GHOST-SHIELD (evasión WAF con memoria, v0.24.0)
wcd404_probe	v0.96.0	WCD-404-PROBE (v0.96.0): el oraculo de la cache que guarda 404s.
wcd_bait	v0.86.0	v0.86.0 WCD-CACHE-KEY (Web Cache Deception)
wide_corpus	v0.48.0	v0.48.0 WIDE-CORPUS: corpus de caza = TODOS los plugins wordpress.org
zombies		ZOMBIE-KILL — mata procesos zombis y duplicados del backend CodexRC.